

System Design Documents

Table of Contents · 9 documents · 132 pages

1.	Designing a Multi-Tenant CI/CD Platform (Cloud-Native)	2
2.	Designing a Sandboxed Cloud IDE / Notebook (Colab-like)	16
3.	Designing a Cloud DevBox Platform (Remote Dev Environments)	31
4.	Designing an Instagram-like Photo/Video Sharing App (Feed-Focused)	48
5.	Designing an In-Memory Key–Value Database (Redis-like)	64
6.	Designing a Distributed Rate Limiter	78
7.	Distributed Rate Limiter — One-Page Cheat Sheet	89
8.	Designing a Slack-like Team Messaging System (Real-Time Focus)	92
9.	Designing an Online Chess Platform (Real-Time, Timed Games)	112

Click any entry to jump to that document.

Designing a Multi-Tenant CI/CD Platform (Cloud-Native)

A complete, interview-ready walkthrough: requirements → estimates → architecture/components → **multi-tenancy & security** → **execution semantics** → **scaling** → **billing** → **observability/compliance** → trade-offs → rollout plan. Use the headings as your whiteboard agenda. The hard parts are **strong tenant isolation while running untrusted build code, fair scheduling, and correct DAG execution semantics** — spend your time there.

Think GitHub Actions / GitLab CI / CircleCI / Buildkite. Tenants connect their repos; pushes/PRs trigger pipelines that run as DAGs of jobs on ephemeral runners; artifacts, caches, and logs flow back — all isolated per tenant, metered per tenant, and scaled elastically.

0. How to drive the interview (talk track)

1. **Clarify** functional + non-functional requirements and assumptions.
2. **Estimate** scale (tenants, builds/day, concurrent jobs, runner fleet, storage).
3. **Split control plane vs data plane (runners)** — the key architectural cut.
4. **Walk the trigger→schedule→execute→report flow.**
5. **Pick the isolation boundary** for untrusted build code (the security core).
6. **Define execution semantics:** DAG, conditionals, matrices, retries, timeouts, cancel, idempotency.
7. **Scale:** autoscale runners, shard the scheduler/control plane, multi-region.
8. **Meter & bill** per tenant; **observe & comply**; **trade-offs**; **rollout plan**.

Keep saying *"here's the trade-off..."* — that's what's being graded.

1. Problem & motivation

A managed platform where many independent tenants run **continuous integration and delivery** pipelines triggered by source-control events, on a shared elastic fleet, without seeing or harming each other.

What makes it hard:

- **Untrusted code execution** — a build runs arbitrary tenant code (make, npm, Docker builds). Multi-tenant on shared infra → must isolate like a hostile workload.
- **Bursty, spiky load** — a monorepo push or a "merge train" can spawn thousands of jobs in seconds; nights/weekends idle. Elastic from zero.
- **Heterogeneous jobs** — seconds-long lint to hour-long integration suites; Linux default, plus Windows/macOS pools.
- **Stateful-ish caching** — dependency/build caches make or break build speed but must stay tenant-scoped.
- **Correctness under failure** — webhooks retry, runners die mid-job, networks flake → need idempotency, retries, and exactly-one logical run per trigger.
- **Fairness** — one tenant's 5,000-job burst must not starve everyone else (noisy neighbor).

The central cut: a **durable, sharded control plane** (orchestration, state, APIs) vs an **ephemeral, elastic data plane** (runners that execute untrusted jobs in disposable sandboxes).

2. Requirements

Functional

- **Integrate with source control** (GitHub/GitLab/Bitbucket): OAuth/Apps, **webhooks**, repo read/clone, commit-status/checks write-back.
- **Trigger pipelines** on push/PR/tag/schedule/manual/API.
- **Execute pipelines as DAGs** of jobs/steps with conditionals, matrices, dependencies.
- **Runners/agents** execute jobs in isolated environments (Linux default; Windows/macOS pools).
- **Artifacts & images** — store/retrieve build outputs and container images.
- **Caching** — source, dependency, and build caches to speed builds.
- **Secrets** — securely inject tenant secrets into jobs, scoped.
- **Logs** — stream live job logs to the UI; persist them.
- **Retries, timeouts, cancellation, re-runs.**

Non-functional

- **Strong tenant isolation** (security is primary — untrusted code).
- **Low scheduling latency** — a job starts in seconds (warm), not minutes.
- **High availability** — control plane HA; a runner/host loss doesn't lose pipeline state.
- **Scalable** — 100k+ tenants, millions of jobs/day, tens of thousands of concurrent jobs.
- **Fair** — no noisy-neighbor starvation; per-tenant quotas.
- **Observable & compliant** — logs/metrics/traces, **audit**, retention, DR.
- **Cost-attributable** — per-tenant/project metering and billing.

Assumptions (state them)

- Managed cloud + Kubernetes available. Linux runners default; Windows/macOS via dedicated pools (often macOS = bare-metal/VM, can't containerize).
- At-least-once webhooks (must dedup). Eventual consistency fine for logs/metrics; **strong** for pipeline state transitions.
- Server-side encryption (not customer-managed E2E unless asked).

Clarifying questions

- **Tenant trust** — fully untrusted public (OSS forks running on our infra) or enterprise tenants? (Public PRs = the hardest threat model.)
- **Self-hosted runners** — allow tenants to bring their own compute? (Yes — big enterprise need.)
- **Max concurrency / build minutes** per plan tier?
- **Caching guarantees** — best-effort (fine) vs correctness-critical?
- **Multi-region / data residency** requirements?
- **Deployment (CD) scope** — do we also deploy to tenant environments, or stop at build/test/artifact?

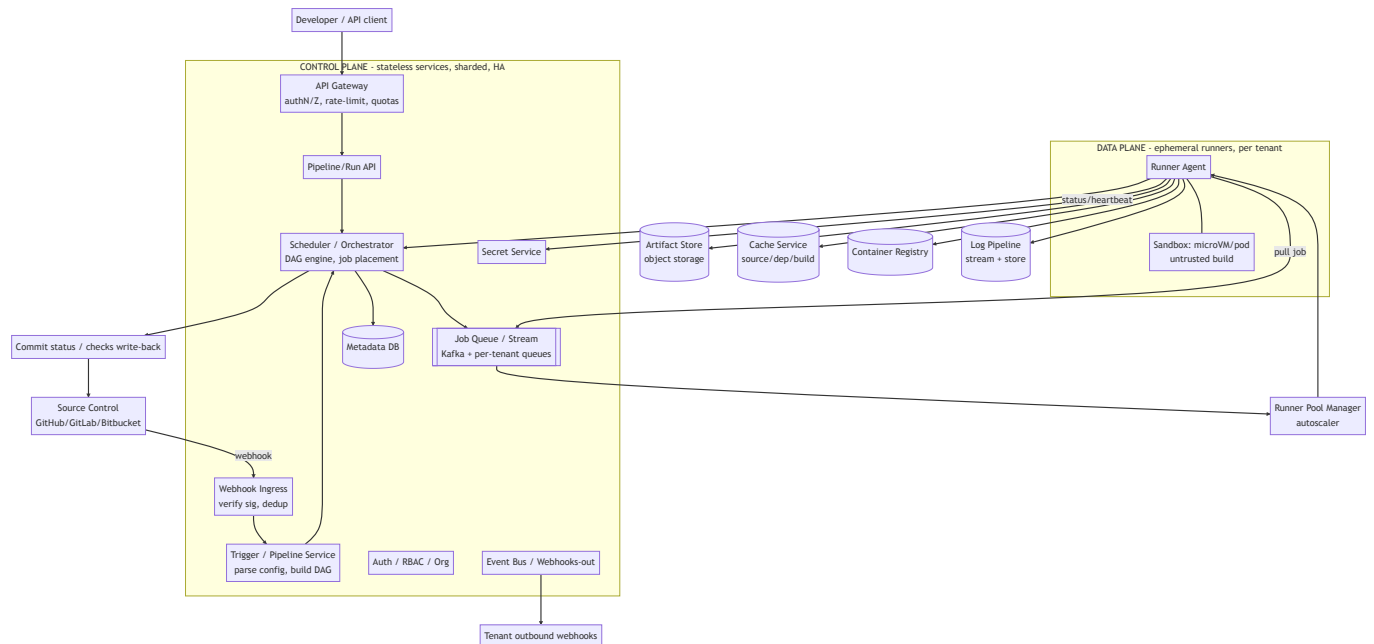
3. Back-of-the-envelope estimation

Quantity	Assumption	Result
Tenants / repos	—	100k tenants, ~1M repos
Builds/day	~10 pushes/active repo subset	10M pipeline runs/day → **120/sec avg, ~1–2K/sec peak**
Jobs per pipeline	DAG avg ~8 jobs (matrices inflate)	80M jobs/day → **1K/sec avg, ~10K+/sec peak**
Concurrent jobs	peak bursts	~20k–50k concurrent runners
Avg job	2 vCPU, 4–8 GB, ~5 min	30k × 2 = ~60k vCPU live at peak
Runner host packing	32 vCPU hosts, ~8 jobs each	~few thousand hosts at peak (scale to ~0 off-peak)
Log volume	~1 MB/job	~80 TB/day logs (hot→cold tiered)
Artifacts/cache	~100 MB/build avg	tens of PB; cache hit-rate is the speed lever
Webhook ingress	every SCM event	~thousands/sec peak , must dedup

Takeaways that drive the design:

1. **10k+ jobs/sec peak, untrusted** → ephemeral sandboxed runners + warm pools; isolation is non-negotiable.
2. **Spiky 0→huge** → autoscale-from-zero; queue + fair scheduler absorb bursts.
3. **Tens of PB artifacts/cache** → object storage + CDN; per-tenant scoping + GC.
4. **At-least-once webhooks** → idempotency/dedup at ingress.

4. Architecture & key components



Component responsibilities & choices

Component	Responsibility	Choice / notes
Source-control integration	OAuth/ App install, webhook receipt, clone/mirror, status write-back	Use a GitHub/GitLab App (fine-grained, per-install tokens) not a personal token; short-lived installation tokens to clone
Webhook ingress	Verify HMAC signature, dedup (delivery id), enqueue	At-least-once → idempotency key = delivery id / (repo, sha, event)
Trigger/Pipeline service	Fetch pipeline config (.ci.yml), parse → DAG , expand matrices	Config-as-code in the repo; validate + version
Scheduler/Orchestrator	Walk the DAG, decide ready jobs, place on runners, track state	The brain; sharded by tenant/run ; durable state machine
Job queue/stream	Decouple schedule↔execute, absorb bursts, fair dispatch	Kafka (durable log) + per-tenant logical queues for fairness
Runner Pool Manager	Autoscale runners, manage warm pools & OS pools	K8s + cluster-autoscaler / Karpenter; pools per OS/size/tenant-class
Runner agent	Pull job, set up sandbox, run steps, stream logs, upload artifacts	Stateless; dials out ; least-privileged
Artifact & image storage	Build outputs + container images	Object store (S3) + CDN; OCI registry for images; per-tenant prefixes
Secret management	Store + inject tenant secrets, scoped	Vault/KMS; short-lived , scoped per repo/env; never in logs/images
Cache service	Source mirror, dependency cache, build cache	Object store + CDN; tenant-scoped keys; restore/save steps
Metadata DB	Tenants, repos, pipelines, runs, jobs, RBAC, quotas	Postgres (HA, sharded); strong consistency for state transitions
APIs / eventing	External REST + webhooks, internal gRPC, outbound events	REST/GraphQL external; gRPC internal; outbound webhooks + event bus

Control plane vs data plane (the key cut)

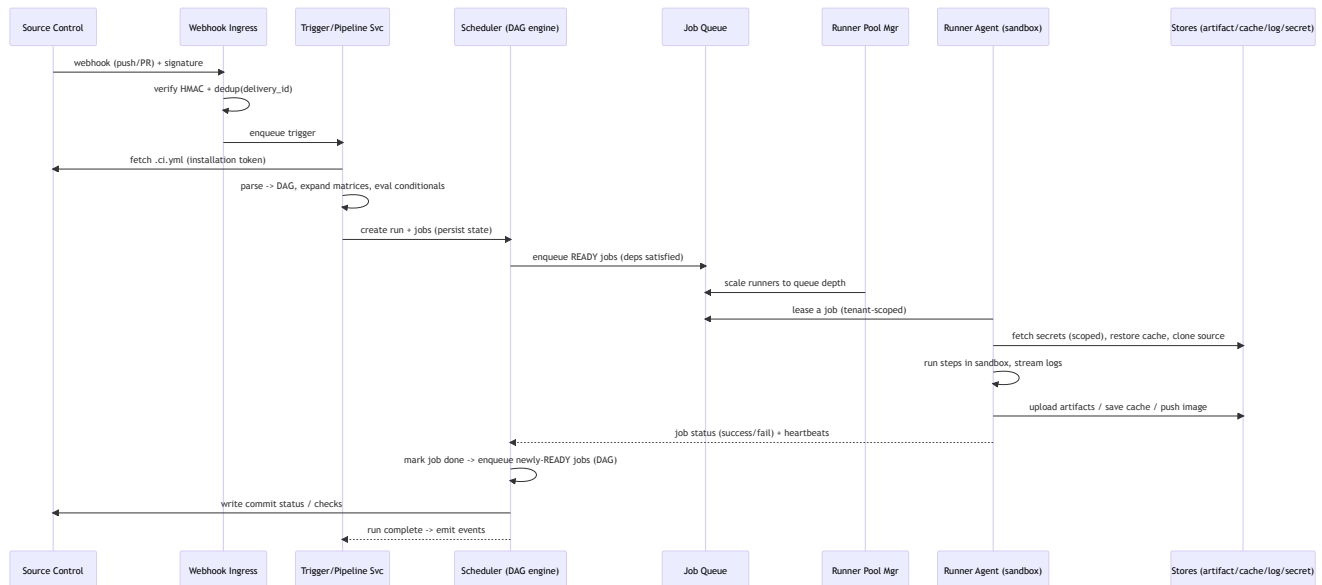
- **Control plane** — durable, HA, **sharded** services holding *pipeline state* (runs, jobs, DAG progress) in Postgres; schedules and reconciles. Survives runner churn.
- **Data plane** — **ephemeral runners** that execute untrusted jobs in disposable sandboxes, **pull** work from the queue, **dial out** (no inbound), and report status. Cattle: a runner dies → reschedule the job.
- **Logs/artifacts bypass the control plane** — they flow runner → log pipeline / object store directly, so the orchestrator isn't a data bottleneck.

Internal & external APIs (REST / gRPC / webhooks / eventing)

```
# External REST (tenant-facing)
POST /v1/repos/{id}/pipelines:run      {ref, inputs}      → {run_id}
GET  /v1/runs/{run_id}                  → {status, jobs[], dag}
POST /v1/runs/{run_id}:cancel
GET  /v1/runs/{run_id}/jobs/{jid}/logs?follow=true      # live log stream (SSE/WS)
POST /v1/webhooks (inbound from SCM)    # signature-verified
```

- **Internal: gRPC** between control-plane services and **agent**↔**scheduler** (lease, heartbeat, status) — typed, low-latency, streaming for logs.
- **Eventing**: every state change (`run.started`, `job.failed`, `run.succeeded`) publishes to an **event bus**; tenants subscribe via **outbound webhooks**; internal consumers drive notifications, metering, and the UI.

5. Data flow: trigger → schedule → execute → report



Key points:

1. **Verify + dedup at ingress** — HMAC signature (authenticity) + delivery-id dedup (at-least-once → exactly-one run).
2. **Config-as-code** — pipeline defined in the repo; parsed into a DAG; **matrices expanded, conditionals evaluated** at plan time (and per-job at runtime for dynamic ones).
3. **Persist run/job state before dispatch** — durable state machine; a control-plane restart resumes the DAG.
4. **Lease-based dispatch** — runners *pull* and **lease** jobs (visibility timeout); if a runner dies, the lease expires and the job is redelivered (with idempotency to avoid double-effect).
5. **DAG progression** — finishing a job unlocks dependents; the scheduler enqueues newly-ready jobs until the DAG completes.
6. **Status write-back** — commit statuses/checks update the PR; events fire to tenant webhooks.

6. Multi-tenancy & security (focus area — the core)

We run **untrusted tenant code** (including public-PR forks) on shared infra. Isolation is the product's foundation.

6.1 Tenant isolation

Compute isolation — pick the boundary by threat model:

Boundary	Isolation	Startup	Density	Use
Container/K8s pod (namespaces/cgroups)	Shared kernel — weak for untrusted	~ms	High	Trusted/enterprise tenants
gVisor (userspace kernel)	Medium-strong	~tens ms	High	Hardened pods
microVM (Firecracker/Kata) ✅	Own kernel — strong	~125 ms	High	Untrusted / public PRs
Dedicated VM/host pool	Strongest	~slow	Low	High-security / Windows/macOS

Recommendation: one job = one fresh, ephemeral sandbox, destroyed after the job (no reuse → no state leakage between tenants). Use **microVMs** for untrusted/public workloads; pods (optionally gVisor-hardened) for trusted enterprise tenants; dedicated VM pools for Windows/macOS and high-security tiers.

Network & data isolation:

- **Per-tenant network policy** — default-deny egress + allowlist; **block the cloud metadata endpoint** (169.254.169.254) so jobs can't steal node IAM creds (SSRF); per-tenant subnets/namespaces; jobs can't reach each other.
- **Per-tenant data scoping** — artifacts/caches/logs keyed by `tenant_id` with **per-tenant encryption keys** (KMS); object-store IAM scoped so a job's credentials can only touch its tenant's prefix.
- **Per-tenant runner groups** — dedicated pools for tenants who require it (and for self-hosted runners they own).
- **K8s namespaces per tenant** (or vCluster) for control-plane-side resources, with NetworkPolicies + ResourceQuotas.

6.2 RBAC, role scopes & secret scoping

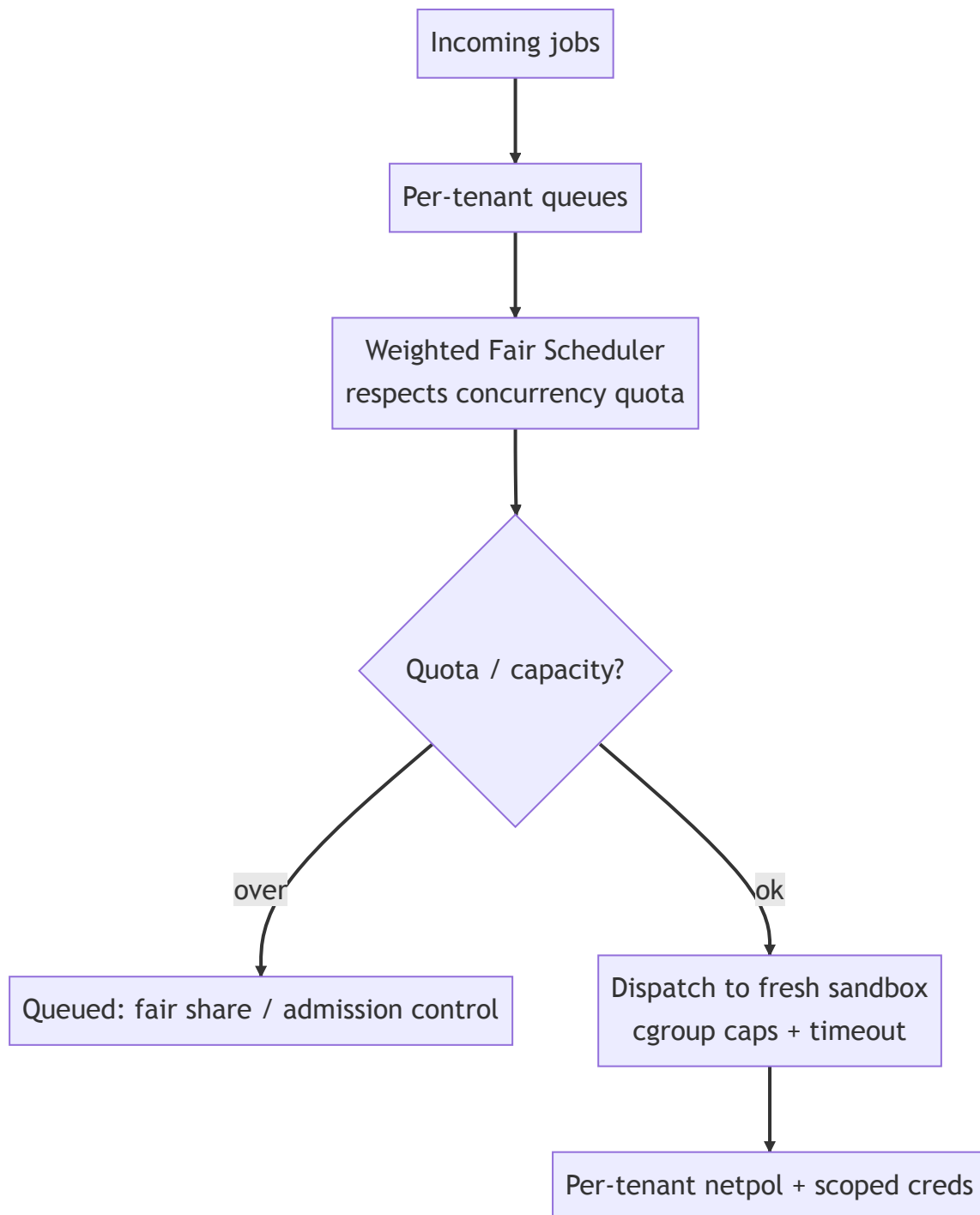
- **Hierarchy**: Org → Project/Repo → Pipeline; **roles** (Owner/Admin/Maintainer/Developer/Viewer) with scoped permissions (run pipeline, manage secrets, approve deploy, view logs).
- Enforced at the API gateway on **every** call and at **job dispatch** (can this pipeline use this runner group / this secret?).
- **Secret scoping** — secrets bound to a scope (org / repo / environment); a job only gets secrets for **its** scope, injected as **short-lived** env/files in the sandbox, **masked in logs**, never persisted to artifacts/images. **Protected/production secrets** gated by environment approvals and **not exposed to fork PRs** (the classic supply-chain attack — untrusted PR exfiltrating prod secrets).

6.3 Rate limiting & quotas

- **API rate limits** per tenant/token (gateway).
- **Concurrency quotas** — max concurrent jobs per tenant/plan (the main fairness lever).
- **Storage quotas** — artifact/cache GB per tenant; **build-minutes** budget per tier.
- **Enforced transactionally** (atomic reserve) at dispatch so two bursts can't both exceed the cap.

6.4 Noisy-neighbor protection (fair scheduling)

- **Per-tenant queues + weighted fair queuing** — round-robin/WFQ across tenants so one tenant's 5,000-job burst can't monopolize runners; each tenant gets a fair share, bursts drain as capacity frees.
- **Resource limits per job** — cgroup CPU/mem/PID/IO caps; hard timeouts; OOM-kills the job, not the host.
- **Admission control** — queue beyond quota; surface "queued (concurrency limit)".
- **Isolation of the build cache** so a poisoned/huge cache from one tenant can't affect others.



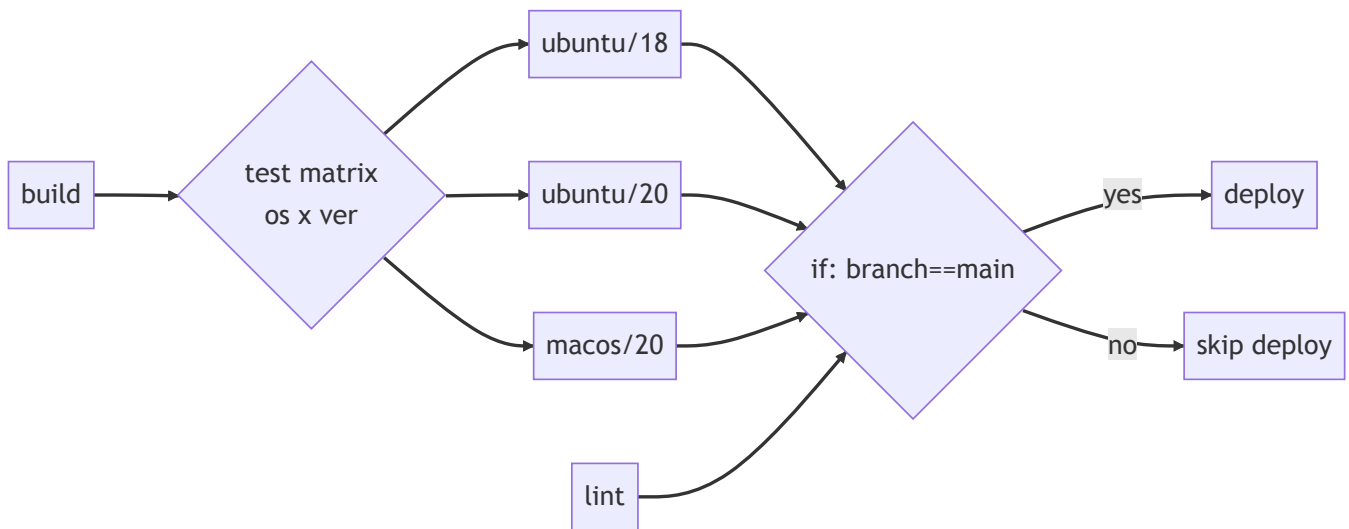
One-liner: "One job runs in one **fresh microVM** with default-deny egress (metadata endpoint blocked), tenant-scoped encrypted storage, and short-lived scoped secrets that never reach fork PRs — and a **weighted-fair, per-tenant-quota scheduler** keeps any single tenant's burst from starving the fleet."

7. Pipeline execution semantics (focus area)

7.1 DAG model, conditionals, matrices

- **DAG of jobs**, each with `needs:/depends_on`: edges → topological execution; independent branches run in parallel; a job starts only when **all** dependencies succeed (or per policy).
- **Steps** within a job run sequentially in the same sandbox (share workspace).

- **Conditionals** — `if`: expressions on branch, event, file paths, prior results, or inputs; evaluated at plan time for static gates and at runtime for dynamic ones (skip → mark `skipped`, propagate to dependents).
- **Matrices** — expand a job template over a parameter grid (`os × version`) into N parallel jobs; `fail-fast` cancels siblings on first failure (configurable); `include/exclude` to tune the grid.
- **Fan-in/fan-out** — fan-out a matrix, fan-in a "deploy" job that `needs`: all of them.



7.2 Retries/backoff, timeouts, cancellations

- **Retries** — per-step/job retry on failure with **exponential backoff + jitter**; distinguish **infra failures** (runner died, spot reclaimed → auto-retry, free) from **user failures** (test failed → don't auto-retry). Cap attempts.
- **Timeouts** — per-step, per-job, per-run wall-clock limits; on expiry, **cancel** (SIGTERM→SIGKILL), mark `timed_out`, free the runner.
- **Cancellations** — user/API cancel, or **superseded** (new push to the same PR auto-cancels the in-flight run — "concurrency group: cancel-in-progress"). Propagate cancel to running sandboxes (signal agent) and dependents (mark `cancelled`).

7.3 Idempotency & deduplication

- **Trigger dedup** — webhook delivery id / (`repo, sha, event`) idempotency key → one logical run per trigger despite at-least-once webhooks/retries.
- **Job lease idempotency** — at-least-once dispatch (a redelivered job after a dead runner) must not double-apply side effects: jobs get a **unique attempt id**; artifact/cache/registry writes are **idempotent** (content-addressed or attempt-scoped paths); "publish"/deploy steps guarded by idempotency keys.
- **Exactly-once effect, at-least-once execution** — the realistic guarantee: a job may *run* more than once (retry), but its observable result is reconciled to one (last-writer/attempt-scoped).
- **Deterministic run id** so UI/API/events refer to one canonical run.

One-liner: "Pipelines are DAGs with matrices and conditionals; infra failures auto-retry with backoff while user failures don't; new pushes supersede in-flight runs; and at-least-once dispatch is made safe with attempt-scoped, content-addressed, idempotent writes so a re-run never double-publishes."

8. Scaling (focus area)

8.1 Autoscaling runners/agents

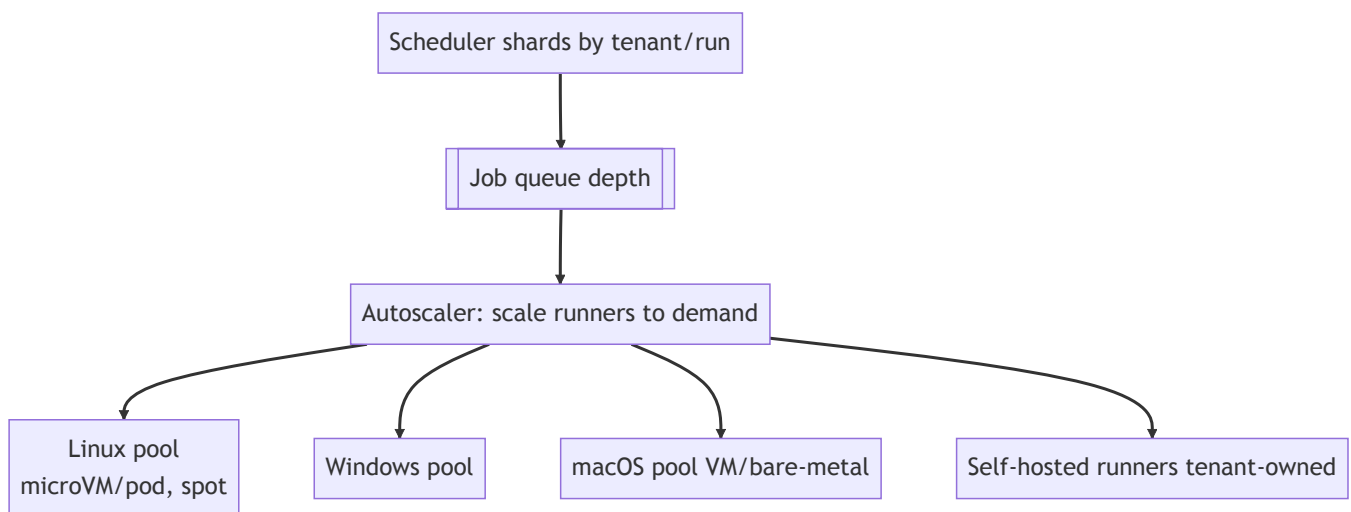
- **Scale-to-zero + warm pools** — keep a small buffer of pre-booted runners per OS/size for fast starts; autoscale the rest on **queue depth** (jobs waiting) and per-pool utilization.
- **K8s cluster-autoscaler / Karpenter** (or cloud ASGs) provision nodes; bin-pack jobs; **spot/preemptible** for cheap, retry-safe jobs (infra-retry covers reclaim), on-demand for critical/long jobs.
- **OS pools** — separate autoscaled pools for Linux (containers/microVMs), Windows, macOS (VM/bare-metal, slower to provision → larger warm buffer).
- **Self-hosted runners** — tenants register their own agents (dial-out, labeled); the scheduler routes labeled jobs to them; tenant owns that capacity/cost.

8.2 Horizontal scaling & sharding of control plane + scheduler

- **Stateless services** scale horizontally behind LBs.
- **Shard the scheduler by tenant/run** (consistent hashing) so each shard owns a subset of runs → no global lock; the DAG engine for a run lives on one shard (ownership + failover).
- **Queue partitioned by tenant** for fairness + parallelism.
- **Metadata DB** sharded by `tenant_id`; read replicas for UI/API reads; strong consistency on the write path for state transitions.
- **Idempotent, lease-based** job handling so any scheduler replica can recover an orphaned run.

8.3 Multi-region

- **Regional control planes + runner fleets**; route a tenant/repo to a **home region** (data residency, latency).
- **Webhook ingress is global/anycast**, forwards to the repo's home region.
- **Artifacts/caches regional** with cross-region replication for DR; registry geo-replicated.
- **Self-hosted runners** can pin builds to a tenant's own region/VPC.
- Keep a run **within one region** (DAG locality) to avoid cross-region chatter; replicate only state needed for failover.



9. Cost tracking & billing per tenant (focus area)

Meter everything a job consumes, attribute to tenant/project, aggregate, bill.

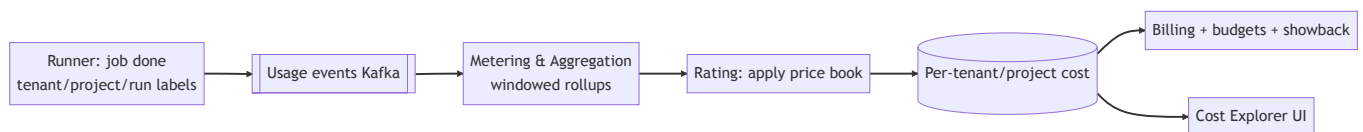
- **Meters (emitted as usage events):**
 - **Compute** — runner **vCPU-seconds**, **GB-seconds**, **GPU-seconds**, by OS/instance type (macOS/Windows priced higher).
 - **Build minutes** — wall-clock per job × a multiplier per runner class.
 - **Storage** — artifact/cache **GB-month**, registry storage.
 - **Network egress** — artifact/log/image download bytes.

- **API/requests** — if metered.
- **Attribution** — every job carries `tenant_id/project_id/pipeline_id/run_id/attempt_id` labels; the agent emits a usage event on job completion (and periodically for long jobs) tagged with these → cost attributes precisely down to project/pipeline.
- **Pipeline:**

```

Runner agent → usage events → Kafka → Metering/Aggregation service
    → rated usage (apply price book) → per-tenant rollups (hourly/daily)
    → Billing (invoices, budgets, showback/chargeback) + Cost Explorer UI
  
```

- **Aggregation** — stream-aggregate usage events into per-tenant/per-project counters (windowed); reconcile against cloud bills (the platform's own cost) for margin.
- **Budgets & controls** — per-tenant budgets, alerts, and **hard caps** (stop scheduling when build-minutes/budget exhausted) — also a cost-abuse guard (crypto-mining in CI).
- **Idempotent metering** — usage events keyed by `attempt_id` so retries/redeliveries don't double-bill.



One-liner: "Each job emits idempotent, attempt-keyed usage events tagged with `tenant/project/run`; a streaming metering pipeline rates them against a price book and rolls them up per tenant for billing, budgets, and hard caps — and we reconcile against our own cloud bill for margin."

10. Observability & compliance (focus area)

Logs, metrics, traces, correlation IDs

- **Logs** — live job logs **streamed** runner → log pipeline → UI (SSE/WebSocket) and persisted (hot store for recent, object store + tiering for old). **Secret-masked**. Separate **system logs** (scheduler/agent) from **user build logs**.
- **Metrics** — queue depth, scheduling latency, runner utilization, job success/failure/retry rates, p50/p99 **time-to-first-byte** (queued→running) and build duration, autoscaler lag → TSDB + SLOs/alerts.
- **Traces** — distributed trace across webhook → trigger → schedule → dispatch → run; find the slow stage.
- **Correlation IDs** — a `run_id` (+ `job_id`, `attempt_id`, `trace_id`, `webhook_delivery_id`) propagated through every service, log line, metric, and event → one-click "everything about this run."

Audit logging & retention

- **Append-only, tamper-evident audit** of security-relevant actions: secret create/access, RBAC/role changes, runner-group changes, pipeline config changes, manual approvals, run cancel/override, login. Per-tenant, queryable, exported to SIEM, with **configurable retention** (compliance: SOC2/ISO/GDPR). PII handling + data-deletion workflows.

Disaster recovery

- **Backups** — Postgres PITR (point-in-time recovery), cross-region replicas; object store + registry cross-region replication; periodic restore drills.
- **RPO/RTO targets** — e.g., RPO minutes (replicated state), RTO < 1h (stand up control plane in DR region, runners autoscale from zero).
- **Graceful degradation** — if a region fails, fail over the home region; in-flight runs may need re-run (idempotent triggers make that safe); queued work survives in replicated Kafka.

Zero-downtime deploys & backward-compatible migrations

- **Rolling / blue-green / canary** deploys of control-plane services behind LBs; **drain** runners (finish in-flight jobs, stop leasing new) before node replacement.
- **Expand-contract (backward-compatible) DB migrations** — add columns/tables first (expand), deploy code that writes both/reads new, backfill, then drop old (contract) — never a breaking schema change in one step. **Versioned APIs + versioned agent protocol** (new control plane supports old agents during rollout). Feature-flag risky changes.

11. Trade-offs (focus area — the money section)

Shared vs dedicated runner pools

	Shared pool	Dedicated pool (per tenant/group)
Cost / utilization	✓ High (bin-packed, scale-to-zero)	✗ Lower (idle reserved capacity)
Isolation	● Relies on sandbox boundary	✓ Physical/pool separation
Noisy neighbor	● Needs fair scheduling	✓ None (tenant owns the pool)
Compliance	● Harder to argue	✓ Easy (dedicated, even single-tenant nodes)
Best for	Most tenants, OSS, cost-sensitive	Enterprise, regulated, high-security

Guidance: shared by default (microVM isolation + fair scheduling) for cost/utilization; offer **dedicated pools** as a premium/enterprise tier for compliance and predictable performance.

Managed vs self-hosted agents

	Managed (our fleet)	Self-hosted (tenant's compute)
Ops burden	✓ We run it	✗ Tenant runs it
Cost model	We meter & bill compute	Tenant pays their own cloud
Network access	● Public/our VPC	✓ Inside tenant VPC (private resources, on-prem)
Security/trust	We isolate untrusted code	✓ Tenant trusts their own; ● they own patching
Best for	Fast onboarding, most tenants	Private networks, custom hardware (GPU), data residency

Guidance: managed by default; self-hosted runners for tenants needing private-network access, special hardware, or strict data residency — they register dial-out agents we route labeled jobs to.

One-liner to say out loud: "I'd split a **sharded, HA control plane** (trigger, DAG scheduler, metadata in Postgres, per-tenant queues on Kafka) from an **ephemeral runner data plane** where **one job runs in one fresh microVM** with default-deny egress and short-lived scoped secrets. **Weighted-fair per-tenant scheduling** kills noisy neighbors, **autoscale-from-zero + warm pools** handle spiky load, **idempotent attempt-scoped writes** make at-least-once dispatch safe, and **per-job usage events** drive tenant billing. Shared pools by default for cost, dedicated/self-hosted for enterprise isolation and private networks."

12. Deployment plan (phased rollout, validation & guardrails)

Phase	Scope	Validation & guardrails
0. Foundations	Multi-tenant data model, authN/Z, metadata DB, API skeleton, IaC	Threat model review; tenancy isolation tests; everything behind feature flags
1. Core CI (private beta)	SCM App + webhooks, DAG scheduler, Linux microVM runners , logs, artifacts	Internal dogfooding; load-test webhook ingress + scheduler; isolation pentest (try to escape sandbox / read another tenant) before any external code
2. Caching + secrets + matrices	Dependency/build cache, secret service, matrices/conditionals, retries/timeouts/cancel	Correctness suite (DAG edge cases, idempotency, dedup); secret-leak scans; chaos test (kill runners → jobs reschedule, no double-effect)
3. Scale & fairness (GA)	Autoscaling, per-tenant quotas, weighted fair scheduling , rate limits	Soak/burst tests (one tenant 5k jobs ≠ starvation); quota-enforcement tests; SLO dashboards + alerts; gradual tenant onboarding (% rollout)
4. Billing & observability	Metering pipeline, cost explorer, audit logs, tracing	Reconcile metered cost vs cloud bill (±% accuracy gate); audit completeness review; retention policies
5. Multi-OS + dedicated/self-hosted	Windows/macOS pools, dedicated pools, self-hosted runners	Per-OS isolation validation; self-hosted security guidance; pin-to-region tests
6. Multi-region + DR	Regional fleets, replication, DR runbooks	DR game-day (fail a region, hit RTO/RPO); restore drills; data-residency verification

Cross-cutting guardrails:

- **Feature flags + canary + % rollout** for every phase; instant rollback.
- **Progressive delivery** — new scheduler/agent versions canaried on a small tenant cohort first.
- **Backward-compatible (expand-contract) migrations** and **versioned agent protocol** so rollouts never break in-flight runs.
- **Guardrail metrics** — error rate, scheduling latency, isolation-violation alerts, cost-anomaly detection; auto-halt rollout on breach.
- **Security gates** — isolation pentest before untrusted code; secret-handling review; fork-PR secret policy enforced from day one.

13. Networking, security & performance best practices

Networking

- **Webhook ingress**: verify **HMAC signature** + dedup delivery id; **anycast/global** ingress forwarding to the repo's home region.
- **Runners dial out** (no inbound); **self-hosted runners** reach the tenant's private VPC/on-prem.
- **Per-tenant/per-job network policy** — default-deny egress + allowlist; **block the metadata endpoint** (fork-PR credential theft).
- **gRPC** internal + agent↔scheduler; **artifact/log/image uploads go direct** to object store / registry / CDN (not through the control plane).

Security

- **One job = one fresh microVM** (untrusted), destroyed after — no reuse, no leak.
- **SCM App + short-lived installation tokens** (least privilege); never personal/long-lived tokens.

- **Secret scoping** — only the job's scope, **masked in logs, never to fork PRs**; protected/prod secrets gated by environment approvals.
- **Supply-chain security** — **signed images (cosign/Sigstore)**, SBOM, **SLSA provenance**, pinned action versions; **OIDC keyless** auth to cloud (no static secrets).
- **Per-tenant KMS** for artifacts/cache; **tamper-evident audit**.

Performance

- **Autoscale-from-zero + warm pools**; **spot/preemptible** for retry-safe jobs; bin-packing.
- **Source/dependency/build cache** (CDN-backed) — the single biggest build-speed lever.
- **DAG parallelism + weighted-fair per-tenant scheduling**; **layer cache + lazy pull** for images.

14. Staying current — modern & emerging approaches

- **Reference systems**: GitHub **Actions**, GitLab CI, CircleCI, Buildkite; **Argo Workflows**, **Tekton**, Drone for K8s-native pipelines.
- **Runners**: **Actions Runner Controller (ARC)** on K8s, **Firecracker/Kata** isolation, **Karpenter** autoscaling.
- **Build/cache**: **BuildKit**, **Bazel** remote cache/RBE, sccache, **Nix** for reproducible builds.
- **Supply chain**: **SLSA** framework, **Sigstore/cosign**, in-toto attestations, SBOM (Syft/Trivy), **OIDC** federation (kill static cloud secrets).
- **Artifacts**: OCI images + **OCI artifacts**, Harbor registry.
- **How I stay current**: GitHub/GitLab changelogs, SLSA & Sigstore projects, KubeCon CD talks, the CNCF landscape.

15. Likely follow-up questions (rehearse these)

- Shared vs dedicated runner pools — the trade-off? (*cost/utilization vs isolation/compliance*)
- How do you stop a fork PR from exfiltrating prod secrets? (*scope secrets, none to forks, env approvals*)
- Runner dies mid-job — double side-effects? (*lease + idempotent, attempt-scoped, content-addressed writes*)
- New push while a build runs — what happens? (*concurrency group: cancel-in-progress / supersede*)
- One tenant fires 5,000 jobs — noisy neighbor? (*per-tenant queue + weighted fair scheduling + quota*)
- Untrusted build stealing node IAM creds? (*microVM + block metadata endpoint + no host creds*)
- At-least-once webhooks — idempotency? (*delivery-id / (repo, sha, event) dedup → one run*)
- Accurate per-tenant billing? (*idempotent, attempt-keyed per-job usage events → rating → rollups*)

16. Summary checklist (whiteboard recap)

- **Control plane vs data plane** — sharded HA orchestrator + ephemeral runners; control plane owns durable DAG state, runners are cattle.
- **SCM integration** — App-based OAuth, signature-verified + deduped webhooks, installation-token clone, status/checks write-back.
- **Isolation** — **one job = one fresh microVM**, default-deny egress (block metadata endpoint), tenant-scoped encrypted storage, short-lived scoped secrets **never exposed to fork PRs**.
- **Fairness** — per-tenant queues + **weighted fair scheduling** + concurrency/storage/API quotas + cgroup caps.
- **Execution semantics** — DAG + matrices + conditionals; infra-retry vs user-fail; timeouts; supersede-on-new-push cancel; **idempotent attempt-scoped** writes for exactly-once *effect*.
- **Scaling** — autoscale-from-zero + warm pools, spot for retry-safe jobs, **scheduler sharded by tenant/run**, DB sharded by tenant, multi-region home-region.
- **Billing** — idempotent per-job **usage events** (vCPU-s, GB-s, minutes, storage, egress) → metering → rating → per-tenant rollups + budgets/hard caps.

- **Observability/compliance** — streamed secret-masked logs, metrics, traces, **run_id correlation**, append-only audit + retention, PITR/cross-region **DR**, **expand-contract** migrations + versioned agent protocol for zero-downtime.
- **Trade-offs** — shared pools (cost) vs dedicated (compliance); managed (easy) vs self-hosted (private network/residency).
- **Rollout** — phased: foundations → core CI → cache/secrets → scale/fairness → billing/obs → multi-OS/dedicated → multi-region/DR, gated by isolation pentests, chaos tests, billing reconciliation, and DR game-days, all behind flags + canaries.

Designing a Sandboxed Cloud IDE / Notebook (Colab-like)

A complete, interview-ready walkthrough: assumptions → requirements → estimates → APIs → architecture → **compute substrate, isolation, output streaming, lifecycle, persistence** → failure handling → trade-offs. Use the headings as your whiteboard agenda. **Security (isolation) is primary**, and the signature feature is **near-real-time output/log streaming** of running cells — spend your time there.

Think Google Colab / Jupyter / Deepnote / Replit / Hex. A browser-based, multi-tenant IDE where a user edits code and **runs cells/commands in an isolated sandbox**, watching stdout/stderr and rich output stream back live.

0. How to drive the interview (talk track)

1. **State assumptions** + clarify functional/non-functional requirements.
2. **Estimate** scale (users, concurrent sessions, start rate, output throughput).
3. **Split control plane vs data plane**; identify the **kernel/sandbox** as the core unit.
4. **Pick the compute substrate** (VM vs container vs microVM) — security-first.
5. **Define the isolation model**: filesystem, network, process, credentials.
6. **Design the output/log streaming path** (the signature feature).
7. **Define the session lifecycle**: create → run → idle → suspend/resume → terminate.
8. **Persist** workspace files + checkpoints.
9. **Failure + trade-offs**.

Keep saying *"here's the trade-off..."* — that's what's being graded.

1. Assumptions (state these up front)

- **Workspace = project/notebook**; a **session = one running sandbox** bound to a workspace.
 - Code is **untrusted and arbitrary** (`pip install`, network calls, fork bombs, crypto-miners) — adversarial multi-tenant is the threat model.
 - **Notebook/REPL model**: a long-lived **kernel** holds in-memory state; cells execute against it and return stdout/stderr + **rich output** (HTML, images, plots, tables).
 - **Eventual consistency** is fine for files; **ordering** of streamed output within a cell must be preserved.
 - A session is **more ephemeral** than a full dev box — minutes to hours, often idle, frequently recycled. **Workspace files** persist; **kernel memory** generally does not (unless we add suspend/resume).
 - Single cloud, multi-region later. Free tier (small, capped) + paid tiers (bigger, GPU).
 - Collaboration is **optional** — called out separately in §13.
-

2. Requirements

Functional

- **Provision an isolated compute environment** per workspace/session.
- **Execute arbitrary user code safely** (sandboxed), cell-by-cell or as commands.
- **Stream execution output/logs** (stdout, stderr, rich MIME output) to the browser in **near real time**.

- **File operations:** upload/download, browse, and **persisted workspace state**.
- **Interrupt / restart** the kernel; **install packages**.
- (Optional) **Real-time collaboration** — co-editing + shared kernel.

Non-functional (the stated priorities)

- **Strong tenant isolation** — *security is primary*. VM-grade boundary.
- **Reasonable startup latency** — new session ready in **a few seconds** (warm), not minutes.
- **Autoscaling + fair resource sharing** — many concurrent sandboxes; no tenant starves others.
- **Observability** — metrics, tracing, **audit logs**.
- **Reliability** — a crashed kernel/host shouldn't lose the user's *files*.

Clarifying questions to ask the interviewer

- **Isolation bar** — adversarial multi-tenant on shared hosts (→ microVMs) or trusted/dedicated?
- **Session persistence** — must kernel **memory** survive idle/suspend (true resume), or only files?
- **GPU** needed (ML notebooks)? Affects scheduling/pools/cost.
- **Egress policy** — can user code reach the internet (`pip install`, API calls)? Usually yes-but-policed.
- **Max runtime / idle limits**, resource caps per tier?
- **Languages** — Python-first (Jupyter kernels) or polyglot?
- **Collaboration** in scope now or later?

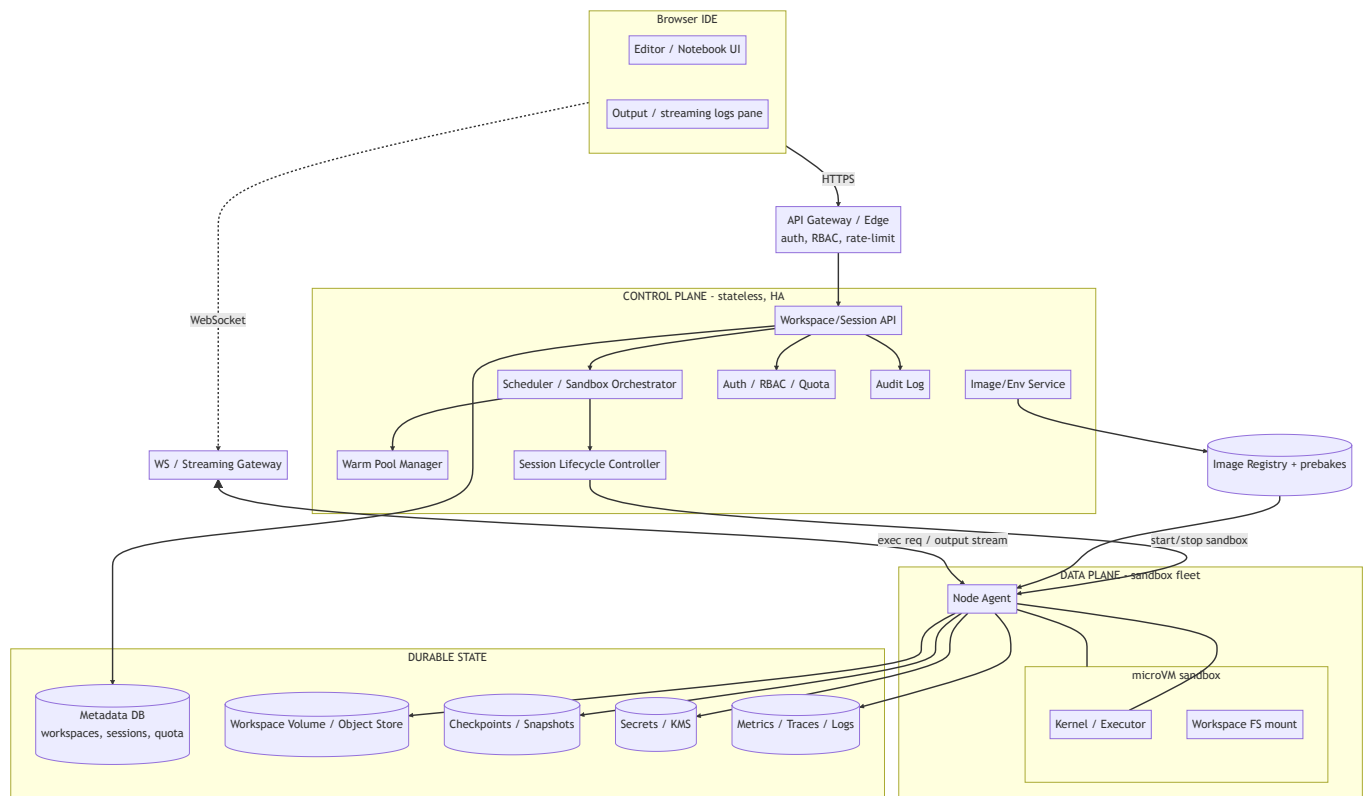
3. Back-of-the-envelope estimation

Quantity	Assumption	Result
MAU	—	5,000,000
Concurrent sessions	~2% online at peak	~100,000 sandboxes live
Session starts/day	~3 per active user	40–15M/day ~**150/sec avg, ~1–2K/sec peak**
Avg session	2 vCPU, 4–8 GB	100k × 2 = ~200k vCPU , ~600 TB RAM live
Host packing	64 vCPU / 256 GB, ~16 small sandboxes/host	~6,000+ hosts at peak
Output streaming	active cells emit logs/plots	bursty KB/s–MB/s per session; fan to 1+ viewers
Workspace storage	~1–5 GB/workspace, 10M workspaces	tens of PB (tiered, mostly cold)
Idle ratio	most sessions idle between runs	idle-cull + suspend = top cost lever

Takeaways that drive the design:

1. **1–2K session starts/sec peak** → cold container/VM boot is too slow → **warm pools + prebaked images**.
2. **100K live sandboxes, adversarial code** → **microVM isolation** + strict resource caps for fair sharing.
3. **Output is bursty and ordered** → a **streaming pipeline** (kernel → broker → WebSocket) with backpressure, not request/response.
4. **Files must persist; memory usually not** → decouple durable **workspace volume/object store** from ephemeral kernels.

4. Architecture overview — control plane vs data plane



Why this split (control vs data plane)?

- **Control plane** (stateless, HA): holds *desired state* in a metadata DB, schedules sandboxes, manages warm pools and lifecycle — reconciler/operator pattern. Restart/scale freely without killing running sessions.
- **Data plane**: worker hosts running each sandbox as a **microVM** with a **node agent** that launches kernels, mounts the workspace, and proxies the **exec** ↔ **output** stream.
- **Streaming bypasses the control plane**: cell-run requests and output flow client ↔ **WS/streaming gateway** ↔ agent ↔ kernel, so live output latency never depends on the orchestrator.

Core components

- **Workspace/Session API** — CRUD workspaces, start/stop sessions, validate quota/RBAC.
- **Scheduler/Orchestrator** — place sandboxes (bin-pack, GPU/zone affinity), manage **warm pools**, autoscale nodes, enforce **fair-share**.
- **Session Lifecycle Controller** — drives the session **state machine**; reconciles desired vs actual; idle-culls.
- **Node Agent** — per-host daemon: launch/stop microVMs + kernels, mount FS, stream output, enforce cgroup caps, heartbeat.
- **Kernel/Executor** — runs user code (Jupyter kernel for Python, etc.), emits stdout/stderr + rich MIME output.
- **WS/Streaming Gateway** — terminates browser WebSockets, routes messages to the right sandbox, fans out output (and to collaborators).
- **Image/Env Service** — prebaked base images + per-workspace dependency layers.
- **Auth/RBAC/Quota, Audit, Observability** — multi-tenant control surfaces.

5. API sketch

REST/JSON for control; **WebSocket** for the live exec + output stream (the important one).

```
# --- Workspaces & files (control) ---
POST   /v1/workspaces                               {name, template}      → {workspace_id}
GET     /v1/workspaces/{id}/files?path=/            → file tree
PUT     /v1/workspaces/{id}/files?path=/a.py        (body = bytes)        # upload/save
GET     /v1/workspaces/{id}/files?path=/a.py        → bytes                # download

# --- Sessions (sandbox lifecycle) ---
POST    /v1/workspaces/{id}/sessions {resources:{vcpu,mem,gpu}} → {session_id, ws_url}
GET      /v1/sessions/{sid}           → {state, host, started_at}
POST     /v1/sessions/{sid}/suspend   # snapshot + free compute
POST     /v1/sessions/{sid}/resume    # restore
DELETE   /v1/sessions/{sid}           # terminate
POST     /v1/sessions/{sid}/interrupt # SIGINT the kernel
POST     /v1/sessions/{sid}/restart   # fresh kernel, keep files
```

WebSocket protocol (Jupyter-style message bus)

Client opens `wss://stream/v1/sessions/{sid}` and exchanges typed messages:

```
// client → server: run a cell
{ "type":"execute_request", "msg_id":"c1", "cell_id":"42", "code":"print(df.head())" }

// server → client: ordered stream for msg_id "c1"
{ "type":"status", "msg_id":"c1", "state":"busy" }
{ "type":"stream", "msg_id":"c1", "name":"stdout", "text":" a b\n0 1 2\n", "seq":1 }
{ "type":"display_data", "msg_id":"c1", "mime":"image/png", "data":"<base64>", "seq":2 }
{ "type":"error", "msg_id":"c1", "ename":"ValueError", "evalue":"...", "traceback":[...] }
{ "type":"execute_reply", "msg_id":"c1", "status":"ok", "exec_count":7 }
{ "type":"status", "msg_id":"c1", "state":"idle" }
```

Design notes:

- Every output carries `(msg_id, seq)` → the UI **orders** and **associates** output with the right cell, and can **resume** after a reconnect by requesting `seq > last_seen`.
- **Rich output** via MIME bundles (`text/plain`, `text/html`, `image/png`, `application/vnd.*`) → plots, tables, widgets.
- **Interrupt/restart** are control messages, not in-band, so they work even when stdout is flooding.

6. Compute substrate — VMs vs containers vs microVMs (focus area)

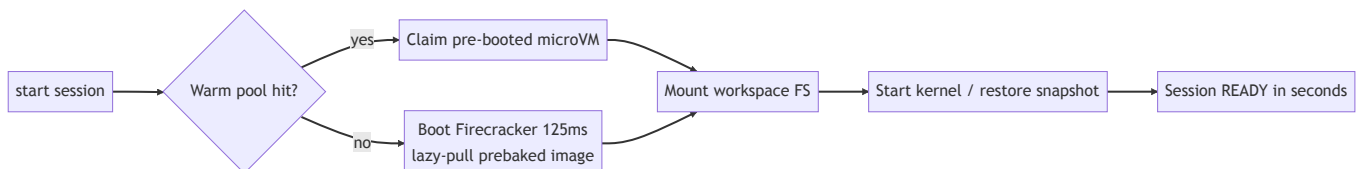
We execute **untrusted**, **adversarial** code from many tenants on shared hosts. The substrate choice **is** the security decision.

Substrate	Boundary	Cold start	Density	Security (untrusted)	Verdict
Plain container (ns/cgroups)	Shared host kernel	~ms	Highest	Weak — one kernel 0-day = full escape	❌ not for adversarial multi-tenant
gVisor (userspace kernel)	Syscall interception	~tens ms	High	Medium-strong (smaller attack surface, some perf cost)	🕒 good middle ground
Kata / microVM (Firecracker) ✅	Own guest kernel + hypervisor	~125 ms	High	Strong (VM-grade)	✅ recommended
Full VM	Hypervisor	~10s+	Low	Strong	for GPU/heavy/dedicated

Recommendation: microVMs (Firecracker/Kata). Each sandbox runs in its **own minimal VM with its own kernel** — VM-grade isolation at near-container speed and density. This is precisely why Colab-like products and serverless runtimes (AWS Lambda, Fly.io) use Firecracker. gVisor is an acceptable fallback where microVMs aren't available; **plain containers are not acceptable** for arbitrary untrusted code.

Managing the substrate:

- **Warm pools** of pre-booted microVMs per common size/image → claim-and-personalize instead of cold boot (kills startup latency).
- **Prebaked images** (base OS + interpreter + popular libs) cached node-local + regional pull-through; **lazy-pull** layers so the kernel starts before the whole image lands.
- **Snapshot/restore** of a pre-initialized VM (kernel imported, libs warm) → resume in well under a second.
- **Bin-packing + autoscaler** scales nodes with demand; **fair-share scheduler** caps per-tenant footprint.



7. Isolation model — filesystem, network, process, credentials (focus area)

Defense in depth *around* the microVM (security is primary):

Filesystem

- **Read-only base image**; user writes land on a **per-session overlay** + the mounted **workspace volume** (only the user's own files).
- No host paths exposed; the guest sees only its rootfs + /workspace. Per-tenant **encryption at rest** (tenant-scoped KMS keys).
- **Disk quota** per session (stop a notebook from filling the host); scratch in tmpfs with caps.

Network

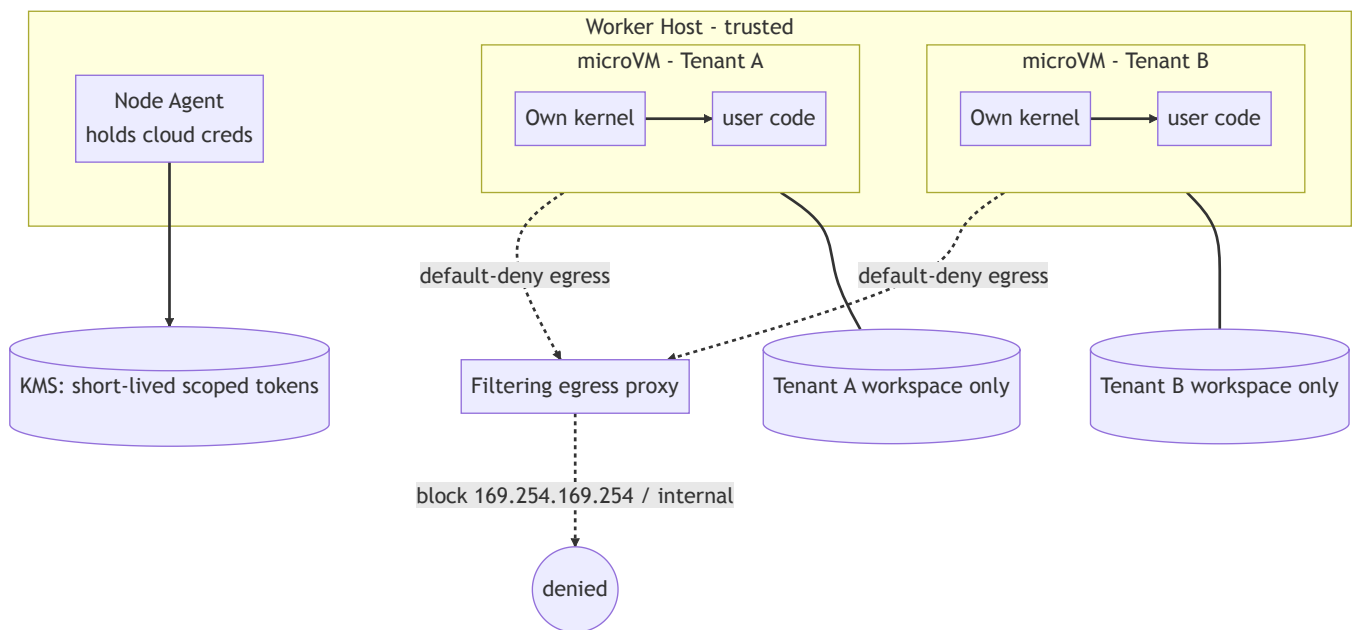
- **Default-deny egress**; allowlist what's needed (e.g., the package mirror). Block the **cloud metadata endpoint** (169.254.169.254) and internal ranges → prevents SSRF to IAM/credential theft.
- Each sandbox on its **own virtual NIC / namespace**; **sandboxes cannot see each other** (no lateral movement).
- Egress through a **filtering proxy** for pip/apt (policy + audit); per-tenant bandwidth caps.

Process / runtime

- **microVM = own kernel**; inside it, drop privileges, **seccomp**/syscall filtering, no `CAP_SYS_ADMIN`.
- **cgroup caps**: CPU shares, memory limit (OOM-kill the sandbox, not the host), **PID limit** (fork-bomb guard), CPU-time/wall-clock limits, IO throttling → **fair sharing** + DoS protection.
- **Hard timeouts** on cells/sessions; kill runaway kernels.

Credentials

- **No host/cloud credentials inside the guest**. The **node agent (host-trusted)** performs any privileged cloud calls; the sandbox receives only **short-lived, narrowly-scoped** tokens (e.g., a signed URL to its own workspace bucket).
- **Secrets** (user-provided API keys) injected at runtime into **tmpfs/env**, never baked into images or persisted in clear; wiped on terminate.
- Every privileged action and connection is **audited**.

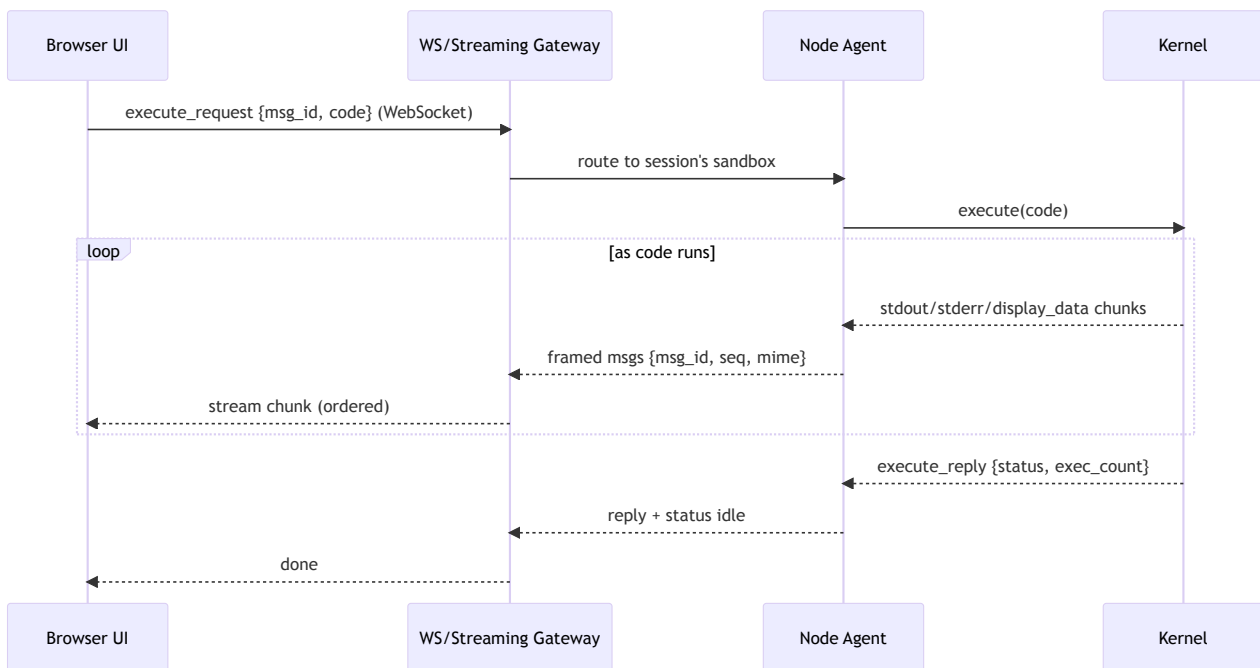


One-liner: "Each sandbox is a Firecracker microVM with its **own kernel**, a read-only base + per-tenant workspace mount, **default-deny egress** that blocks the metadata endpoint, cgroup CPU/mem/PID caps, and **zero host credentials** inside — privileged cloud calls happen host-side via the agent with short-lived scoped tokens."

8. Output / log streaming architecture (the signature feature)

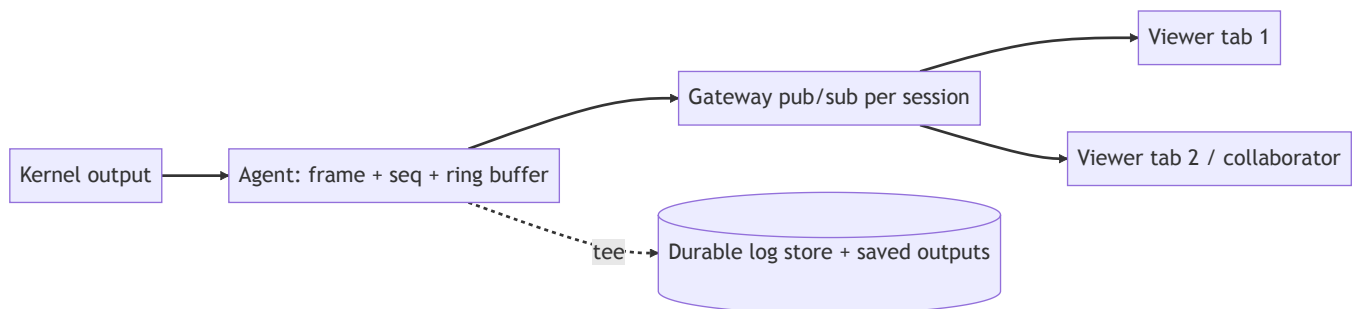
The product *feels* like Colab because output appears **live, ordered, and resumable**. This is a streaming problem, not request/response.

Path



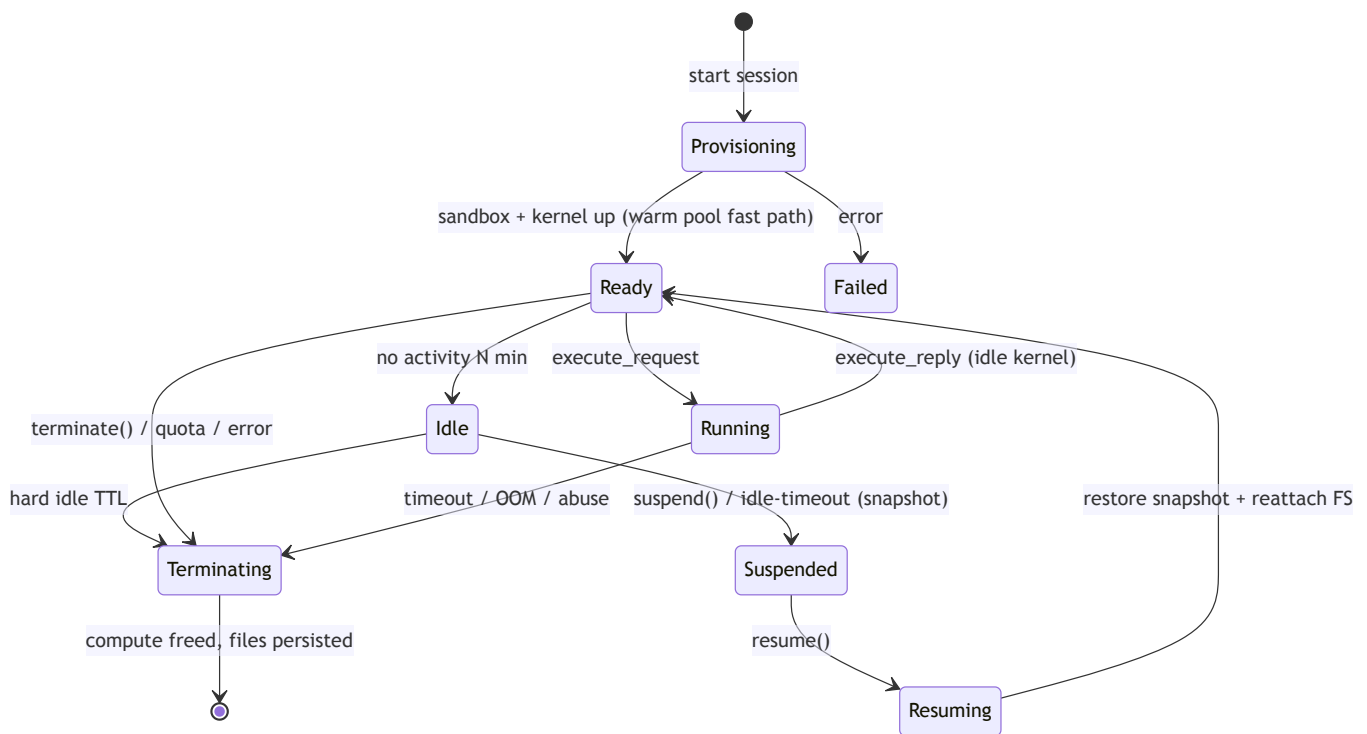
Key design decisions

- **WebSocket (bidirectional, low-latency)** for the live channel — not long-polling. SSE is a fallback for output-only.
- **Ordering & resume** — each chunk has `(msg_id, seq)`. The agent keeps a small **ring buffer** of recent output per cell; on reconnect the UI sends `last_seq` and replays the gap → **no lost or duplicated** output across flaky networks.
- **Backpressure** — `while True: print(x)` can emit MB/s. Apply flow control: **rate-limit/coalesce** chunks (batch small writes every ~50 ms), **cap total output per cell** (truncate with "output limit reached"), and respect WS buffer high-water marks so a slow client can't OOM the gateway or kernel.
- **Rich output** — MIME bundles streamed the same way; large blobs (images, files) can be **offloaded to object storage** and streamed as a URL reference instead of inline base64.
- **Fan-out** (collaboration / multiple tabs) — the gateway **publishes** a session's output to all subscribers via an in-memory pub/sub (or Redis pub/sub across gateway nodes), so N viewers see the same live stream.
- **Durable logs** — tee the stream to a **log store** (and the notebook's saved outputs) for audit, post-hoc viewing, and the "reopen and see last run's output" experience. Separate **system logs** (kernel lifecycle, errors) from **user output**.
- **Stateful routing** — output is tied to a specific sandbox, so the WS gateway uses **sticky routing** (`session_id` → `sandbox/agent`) rather than round-robin.



One-liner: "Output streams over a WebSocket as ordered `(msg_id, seq)` chunks with an agent-side ring buffer for gap-free reconnect, backpressure + per-cell caps so a runaway `print` loop can't melt the gateway, rich output offloaded to object storage, and a per-session pub/sub fan-out so multiple viewers (or collaborators) share one live stream — tee to a durable log store for replay and audit."

9. Lifecycle management — create, run, idle, suspend/resume, terminate (focus area)



Phase	Compute	Kernel memory	Workspace files	Cost	Resume
Running	Active	In RAM	Mounted, live	Full	—
Idle	Active (briefly)	In RAM	Mounted	Full	instant
Suspended	Freed	Snapshotted (optional)	Persisted	Storage only	sub-sec—seconds
Terminated	Freed	Gone	Persisted	Storage only	cold start

- **Create** — fast path via warm pool (§6); mount workspace; start kernel; return WS URL.
- **Run** — exec messages over WS; kernel holds REPL state between cells.
- **Idle** — kernel goes quiet; after N minutes start reclaiming. **Idle culling is the #1 cost + fair-share lever** (most sessions idle).
- **Suspend/resume** — two options:
 - *Cheap*: **terminate the kernel, keep files**; resume = cold kernel (memory lost). Default for free tier.
 - *Premium*: **memory snapshot** (Firecracker snapshot to object store) → resume restores variables/imports exactly. Costs storage; great UX.
- **Terminate** — graceful kernel shutdown, **flush files + save outputs**, detach volume, free compute, GC overlay/scratch. Always persist the workspace.
- **Self-healing** — agent heartbeats; lifecycle controller reschedules sandboxes off dead hosts and reaps orphans.

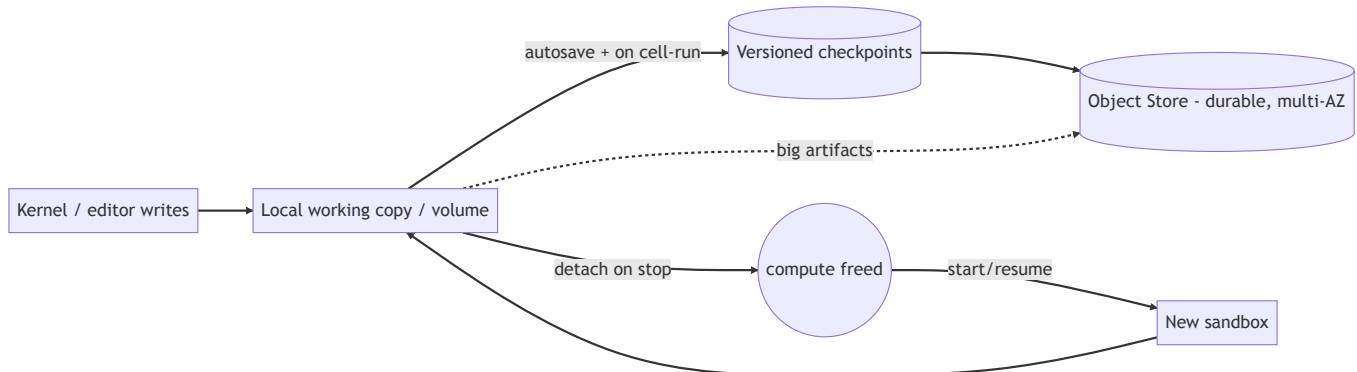
10. Data persistence — workspace files & checkpoints (focus area)

Principle: separate durable state from ephemeral compute. Kernels are cattle; the workspace is the pet.

Data	Store	Notes
Workspace files (notebooks, code, data)	Network volume (per workspace) <i>or</i> object store + local cache	Durable, detach/reattach across hosts; survives kernel/host loss
Saved notebook outputs	With the notebook (+ object store for big blobs)	"Reopen and see last run"
Checkpoints / autosave	Versioned object store	Periodic + on-edit; crash recovery, undo
Memory snapshots (suspend)	Object store	Optional true resume
Large artifacts/datasets	Object store, streamed	Don't bloat the volume; signed-URL access
Metadata (workspaces, sessions, quota)	Postgres (HA)	Source of truth

- **Two models:**

- **Network block volume per workspace** (EBS-like) — POSIX FS, mount on start, detach on stop, **snapshot** for checkpoints. Simple mental model; great for "it's just a filesystem."
- **Object store + local working copy** — files live in S3; sandbox syncs a working copy on start and **writes back** (autosave/on-close). Cheaper, infinitely scalable, naturally versioned; sync adds complexity.
- *Common hybrid:* object store as source of truth + a fast local/volume cache for live editing.
- **Autosave + checkpoints** — periodically and on cell-run, snapshot the notebook (content + outputs) to versioned storage → crash never loses more than a few seconds; users can restore.
- **Durability** — replicated (multi-AZ) volume/object store; checkpoints give point-in-time recovery.
- **Crash consistency** — `fsync`/journaling; flush on graceful terminate; filesystem-consistent snapshots.
- **GC** — reap scratch/overlay on terminate; expire orphaned volumes and old anonymous workspaces after retention.



One-liner: "Files live on a replicated per-workspace volume (or object store) decoupled from the kernel, with periodic + on-run **checkpoints** to versioned storage — so a kernel crash, OOM, or host failure costs you a restart, never your notebook."

11. Autoscaling & fair resource sharing

- **Warm-pool autoscaling** — keep a buffer of pre-booted microVMs per size; scale predictively (weekday/class-hour surges) so starts stay fast under load.
- **Node autoscaler + bin-packing** — add/remove hosts with demand; pack small sandboxes densely; **scale to zero** off-hours.
- **Fair-share scheduler** — per-tenant/per-org caps on concurrent sandboxes and total vCPU/RAM/GPU; **weighted fair queuing** so a heavy user can't starve others; **admission control** + queue under saturation (graceful "waiting for a runtime...").

- **Resource caps per session** — cgroup CPU/mem/PID/IO limits (also a security control) guarantee one notebook can't hog a host.
 - **Cost levers** — **idle culling**, session TTLs, suspend-instead-of-run-idle, **spot/preemptible** hosts for free-tier (with checkpoint so preemption only costs a restart), GPU time-slicing/pools.
-

12. Observability (a stated requirement)

- **Metrics** — start latency p50/p99, **warm-pool hit rate**, sandboxes live, scheduler queue depth, kernel exec latency, output throughput, OOM/timeout rates, per-tenant usage → TSDB + SLOs/alerts.
 - **Tracing** — trace the **start path** (API → schedule → boot → kernel → ready) and the **exec path** (request → kernel → first-output → done) to find slow stages. Propagate `session_id/msg_id`.
 - **Logs** — separate **user output** (streamed/saved) from **system logs** (kernel/agent/control plane); centralized, per-tenant scoped.
 - **Audit logs** — append-only, tamper-evident: session create/terminate, **code execution events**, file up/download, package installs, secret access, egress decisions, role/quota changes → SIEM + retention. Critical for a system running untrusted code.
 - **Golden signal: time-to-ready** (start → first cell runnable) and **time-to-first-output** (run → first stream chunk).
-

13. Collaboration (optional — called out)

If included: **co-editing** via **CRDT/OT** (e.g., Yjs) over the same WebSocket layer for conflict-free shared text, plus a **shared kernel** so collaborators see the **same live output** (the §8 pub/sub already fans output to multiple subscribers). Add presence/cursors and per-user RBAC on the workspace. **Trade-off**: a shared kernel means shared mutable state and a shared blast radius (one user's `os.system` affects all) — fine within a trusted team, gated by workspace membership. Call it out as **v2** to keep the core scope focused.

14. Failure scenarios — "what if X fails?"

Failure	Impact	Mitigation
Kernel crashes / OOM	Cell fails, memory lost	OOM-kill sandbox not host; surface error; files + last checkpoint safe ; one-click restart
Worker host dies	Running sandboxes lost	Workspace is network-attached/object-store → reschedule elsewhere; memory lost unless snapshotted
Control plane down	Can't start/stop sessions	CP stateless + HA; running sandboxes keep streaming (data-plane independent)
WS/streaming gateway dies	Output stream drops	Multiple gateways; client reconnects , replays from <code>last_seq</code> via agent ring buffer
Output flood (<code>while True: print()</code>)	Gateway/client overload	Backpressure, coalescing, per-cell output cap , truncate
Image registry slow	Cold starts stall	Node-local + regional cache, lazy-pull, warm pool absorbs
Runaway / abusive code (miner, fork bomb)	Host degraded / abuse	cgroup CPU/mem/PID/IO caps, timeouts, egress policy, anomaly detection → auto-suspend
Tenant escape attempt	Cross-tenant breach	microVM (own kernel) + default-deny egress + no host creds + seccomp
Storage/AZ failure	Risk to files	Multi-AZ replication + versioned checkpoints (DR restore)
Start storm (class begins)	Latency spike	Predictive warm-pool scaling + admission queue + fair-share

Guiding principle: the **data plane outlives the control plane**, and **durable files outlive the kernel**. A crash should cost a *restart*, never the user's work.

15. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Isolation vs speed/density	Full VM (strong, slow)	Container (fast, weak)	microVM — VM-grade isolation at near-container speed ✅ (gVisor as fallback; never bare containers for untrusted code)
Startup vs cost	Big warm pool (instant)	No pool (cheap, slow)	Pool sized to predicted demand + snapshots; scale-to-zero off-hours
Suspend: files-only vs memory	Terminate kernel (cheap, lose RAM)	Memory snapshot (instant resume, storage cost)	Files-only default; memory snapshot premium
Streaming: WS vs polling	WebSocket (live, bidirectional)	SSE/long-poll (simpler)	WS for the live exec channel; SSE fallback for output-only
Output: inline vs offload	Inline base64 (simple)	Object-store + URL (scalable)	Inline small, offload large blobs
Persistence: volume vs object store	Block volume (POSIX, simple)	Object store (cheap, versioned)	Hybrid: object store source-of-truth + local/volume cache
Consistency vs availability	Strong everywhere	Eventual reconcile	Strong for quota/RBAC/billing; eventual for fleet; available data plane under CP partition

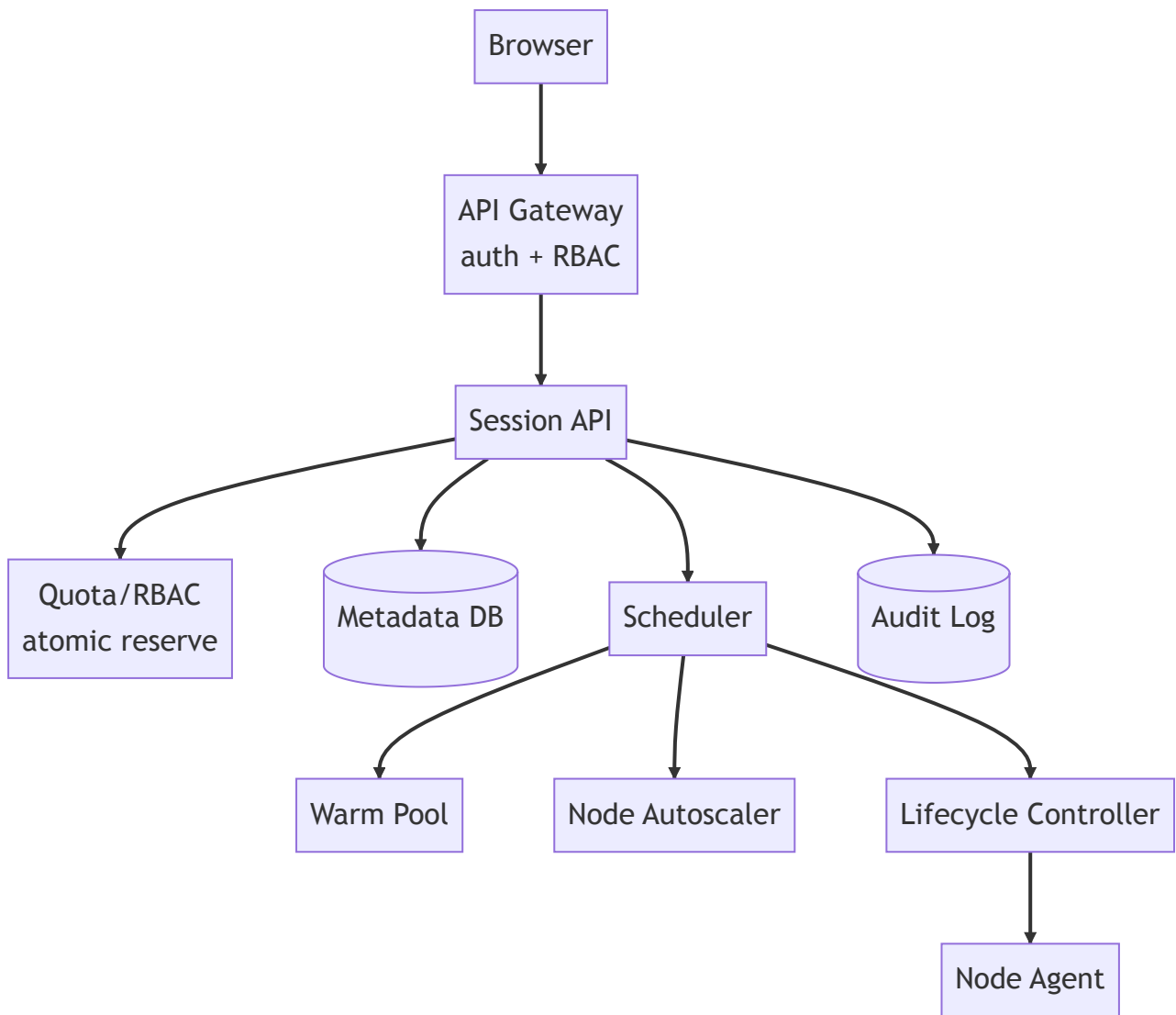
CAP/PACELC framing: **quota/RBAC/metadata** is **CP** (no double-spend, no privilege bypass). **Fleet state** is eventually consistent (controllers reconcile desired→actual). The **streaming data plane** is designed to **stay available** even when the control plane is partitioned — a running notebook keeps streaming.

One-liner to say out loud: "I'd run each session as a **Firecracker microVM** (own kernel, default-deny egress, cgroup caps, no host creds) for security-first multi-tenant isolation, fronted by a **stateless HA control plane** that schedules from **warm pools + prebaked images** so starts take seconds. Cell execution and output ride a **WebSocket** as ordered (`msg_id`, `seq`) chunks with an **agent-side ring buffer** for gap-free reconnect, backpressure + per-cell caps to survive output floods, and per-session **pub/sub fan-out** for collaborators — teed to durable logs. **Files live on a replicated volume/object store with checkpoints**, decoupled from the kernel, so crashes cost a restart, not the user's work. Idle-culling, fair-share scheduling, and spot hosts keep it economical."

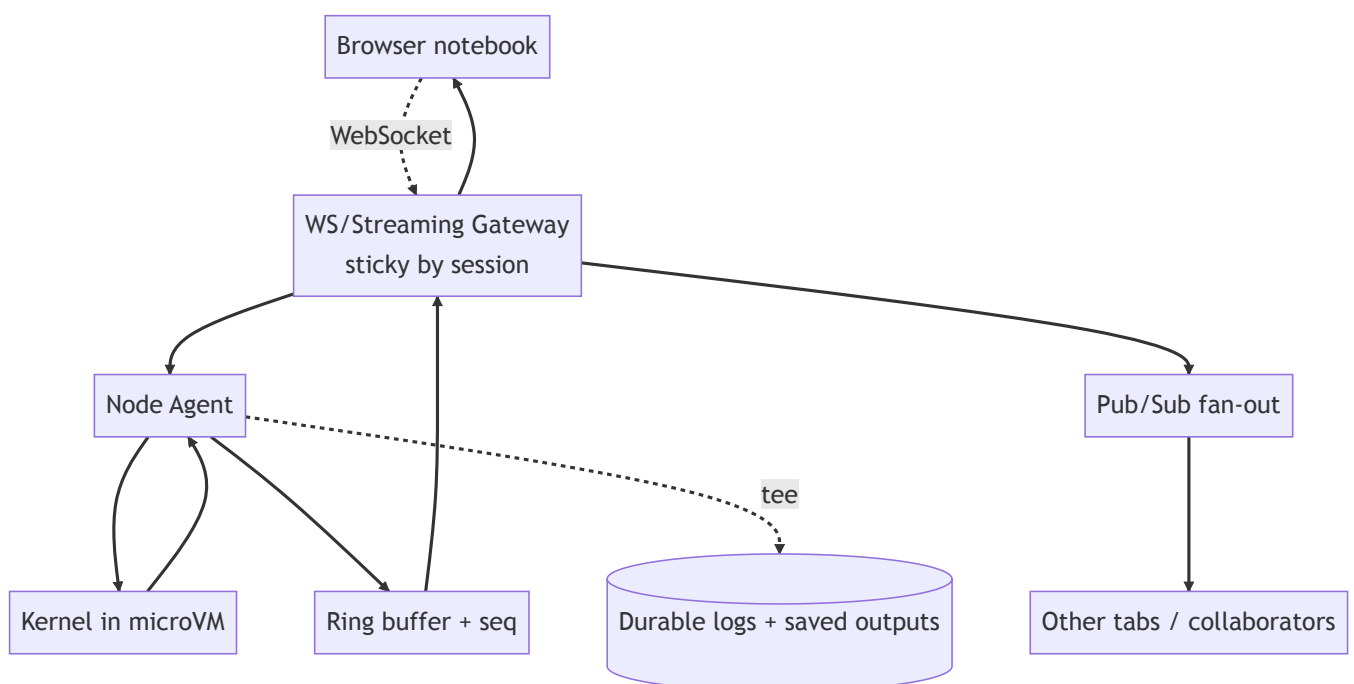
16. Full system design (detailed)

End-to-end, split into **(A)** the control/start path, **(B)** the exec + output streaming data path, and **(C)** persistence & lifecycle.

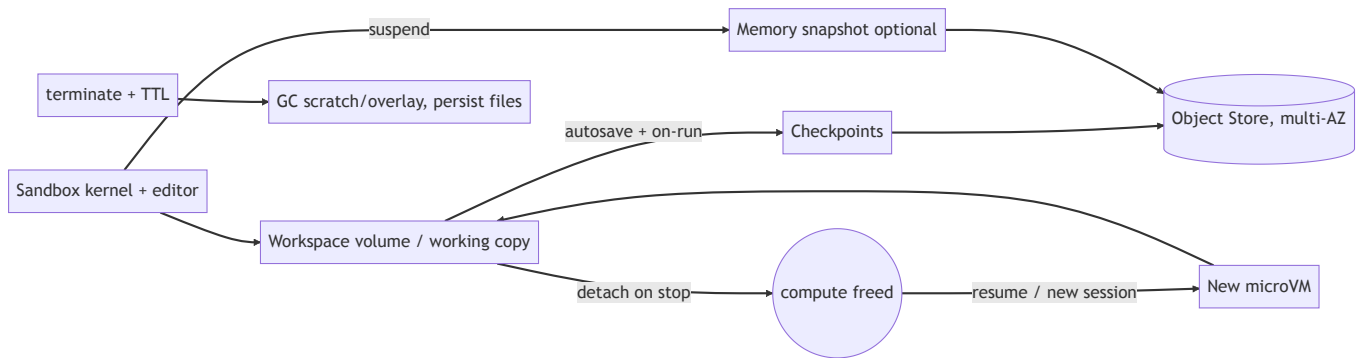
16A. Control path — start a session



16B. Data path — execute & stream output



16C. Persistence & lifecycle



17. Networking, security & performance best practices

Networking

- **WebSocket** for the live exec/output channel (bidirectional, low-latency); **SSE fallback** for output-only clients.
- **Sticky routing** `session_id` → sandbox at the streaming gateway; **TLS at edge**, HTTP/2/3 for the UI, **gRPC** agent↔control.
- **Per-sandbox network namespace**; **default-deny egress** through a filtering proxy; **block the metadata endpoint** (IAM/SSRF).
- **Per-gateway fan-out** of a session's output to multiple viewers via pub/sub.

Security

- **microVM per session** (own kernel); **one session = one fresh sandbox, destroyed after** → no cross-tenant state leak.
- **No host creds in the guest**; short-lived scoped tokens (signed URL to the session's own bucket).
- **Read-only base + per-tenant overlay/workspace**; per-tenant **KMS encryption**.
- **seccomp, dropped caps, cgroup CPU/mem/PID/IO caps** (fork-bomb/miner guard); hard cell/session timeouts.
- **Secrets in tmpfs, masked in logs**; **audit code-execution events** (running untrusted code makes this essential).

Performance

- **Warm pools + prebaked images + snapshot/restore** → seconds to first cell.
- **Output backpressure + per-cell caps + coalescing**; **offload large blobs** to object storage (stream a URL, not base64).
- **Agent-side ring buffer** for gap-free reconnect; **idle-cull + scale-to-zero**.

18. Staying current — modern & emerging approaches

- **Isolation**: **Firecracker / Kata / gVisor**; **WebAssembly (WASI, Pyodide)** for ultra-light or in-browser sandboxing.
- **Reference systems**: Google **Colab**, **Jupyter/JupyterHub + BinderHub**, **Deepnote**, **Hex**, **Replit** (Nix + Firecracker).
- **Protocols**: **Jupyter messaging protocol** (kernels), **Language Server Protocol**, **CRDTs** (Yjs/Automerger) for collaboration.
- **Streaming**: **WebTransport/HTTP3**, gRPC streaming for output.
- **GPU**: time-slicing / **MIG** / fractional GPUs for ML notebooks.
- **How I stay current**: Jupyter & Firecracker releases, Replit/Colab engineering blogs, KubeCon/PyCon talks.

19. Likely follow-up questions (rehearse these)

- Why a microVM over a container for arbitrary code? (*own kernel; bare containers are unsafe for untrusted code*)
 - Output floods (`while True: print`) — how do you survive? (*backpressure + per-cell caps + coalescing*)
 - Client reconnects mid-cell — no lost/duplicated output? (*(`msg_id`, `seq`) + agent ring buffer replay*)
 - Does kernel memory survive idle? (*terminate-and-keep-files by default; memory snapshot for premium resume*)
 - How do you start a session in seconds? (*warm pool + prebaked image + snapshot/restore*)
 - Stop a notebook stealing node IAM creds? (*block the metadata endpoint; no host creds in guest*)
 - Fair sharing across tenants? (*weighted fair queuing + cgroup caps + admission control*)
 - Collaboration consistency? (*CRDT co-editing + shared kernel — call out the shared-blast-radius trade-off*)
-

20. Summary checklist (whiteboard recap)

- **Control plane vs data plane** — stateless HA orchestrator + sandbox fleet; data plane keeps streaming even if CP is down.
- **microVM substrate** (Firecracker) — own kernel per session; security-first for adversarial code (gVisor fallback, never bare containers).
- **Isolation** — read-only base + per-tenant FS, default-deny egress (block metadata endpoint), cgroup CPU/mem/PID caps, **no host creds in guest**, audited.
- **Output streaming** — WebSocket, ordered (`msg_id`, `seq`), agent ring buffer for gap-free reconnect, backpressure + per-cell caps, rich-output offload, pub/sub fan-out, teed to durable logs.
- **Fast start** — warm pools + prebaked images + snapshot/restore = seconds ("restore, not build").
- **Lifecycle** — create → run → idle → suspend/resume (optional memory snapshot) → terminate; **idle-culling** is the top cost/fair-share lever.
- **Persistence** — files on replicated volume/object store + **checkpoints**, decoupled from the kernel; crash = restart, not data loss.
- **Autoscaling + fair-share** — predictive pools, bin-packing, weighted fair queuing, admission control, spot for free tier.
- **Observability** — time-to-ready / time-to-first-output SLOs, exec tracing, append-only audit of code execution.
- **CAP** — CP for quota/RBAC/billing; eventual fleet reconcile; available streaming data plane under partition.

Designing a Cloud DevBox Platform (Remote Dev Environments)

A complete, interview-ready walkthrough: requirements → estimates → APIs → data model → lifecycle → **provisioning, isolation, persistence, connectivity** → multi-tenancy (quota/RBAC/audit/cost) → failure handling → trade-offs. Use the headings as your whiteboard agenda. The hard parts are **fast provisioning, strong isolation, and reliable persistence** — spend your time there.

Think GitHub Codespaces / Gitpod / Coder / AWS Cloud9 / Replit. A managed service that hands a developer a ready-to-code remote machine in seconds, reachable from a **browser, SSH, or an IDE plugin** (VS Code Remote, JetBrains Gateway).

0. How to drive the interview (talk track)

1. **Clarify** product goals + functional/non-functional requirements.
2. **Estimate** scale (devs, concurrent boxes, provisioning rate, storage) to justify choices.
3. **Split control plane vs data plane** — the single most important architectural cut.
4. **Define the lifecycle** (create/start/stop/pause/resume/delete) as a state machine.
5. **Pick the isolation boundary** (VM vs container vs microVM) and justify.
6. **Make provisioning fast** (prebuilds, warm pools, snapshots, image caching).
7. **Persist reliably** (home dir volumes, snapshots, what survives stop/pause).
8. **Connect** the browser/SSH/IDE through a gateway/tunnel.
9. **Multi-tenancy**: quota, RBAC, audit, cost controls.
10. **Failure + trade-offs**.

Keep saying *"here's the trade-off..."* — that's what's being graded.

1. Problem & motivation

Give developers an **instant, reproducible, powerful** development machine that lives in the cloud instead of on their laptop.

Why teams want this:

- **Zero onboarding friction** — "works on my machine" dies; a new hire is coding in minutes from a template, not days of setup.
- **Reproducibility** — environment is defined as code (image + toolchain + dotfiles), identical for everyone.
- **Power & elasticity** — attach 32 vCPUs or a GPU for a build/ML task, then give it back.
- **Security** — source code and secrets stay in the cloud, never on an unmanaged laptop; centrally revocable.
- **Cost efficiency** — pause idle boxes; pay for what's used.

What makes it hard:

- **Fast provisioning** — a cold "create from template + clone repo + install deps" can take minutes; users expect **seconds**.
- **Strong isolation** — untrusted code (arbitrary `npm install`, Docker-in-Docker) from many tenants on shared hardware → must not let one box read/attack another or the host.
- **Reliable persistence** — a developer's uncommitted work and home dir **must survive** stop/pause/restart and host failures.

- **Stateful, long-lived, bursty** — unlike stateless web services, each box is a pet-ish stateful workload that can be paused for days and resumed.
-

2. Requirements

Functional

- **Lifecycle**: create, **start**, **stop**, **pause**, **resume**, delete a dev box.
- **Templates**: predefined images (OS + toolchain) users instantiate from; org-custom templates.
- A dev box bundles: **OS image, language/toolchain deps, repo checkout, user home dir, secrets, allocated CPU/mem/storage, optional GPU**.
- **Access** via browser (web IDE), **SSH**, and **IDE plugins** (VS Code Remote, JetBrains Gateway).
- **Orgs & teams, quota enforcement, RBAC, audit logs, cost controls**.
- Resize / reconfigure resources; snapshot/restore.

Non-functional (the stated optimization targets)

- **Fast provisioning** — warm start in **seconds**, cold create in well under a minute.
- **High availability** — control plane HA; a host failure must not lose a developer's data.
- **Scalability** — tens of thousands of **concurrent** boxes, thousands of provisions/min at peak.
- **Strong isolation** — VM-grade boundary between tenants.
- **Reliable persistence** — durable home dir / workspace; survives pause/stop/host loss.
- **Operational observability** — metrics, logs, traces, audit, cost attribution per box/team.

Out of scope (name then park)

Source-control hosting itself, CI/CD pipelines, the IDE's editing features. We integrate with Git providers and IDEs; we don't rebuild them.

Clarifying questions to ask the interviewer

- **Disposable vs persistent** — both? (Yes — ephemeral PR-review boxes *and* long-lived personal boxes.)
 - **Isolation bar** — multi-tenant on shared hosts (need microVMs) or dedicated per-org hosts?
 - **Pause semantics** — preserve **RAM state** (true hibernate/resume) or just stop the compute and keep disk?
 - **GPU** — required? (Affects scheduling, pools, cost.)
 - **Cloud** — single cloud, multi-cloud, or on-prem (Coder-style)? Affects the infra abstraction.
 - **Max idle / cost policy** — auto-stop after N minutes idle? Hard org budgets?
 - **Self-hosted runners vs our fleet** — do we run the compute or does the customer (BYO-cloud)?
-

3. Back-of-the-envelope estimation

Assume a large enterprise/SaaS scale:

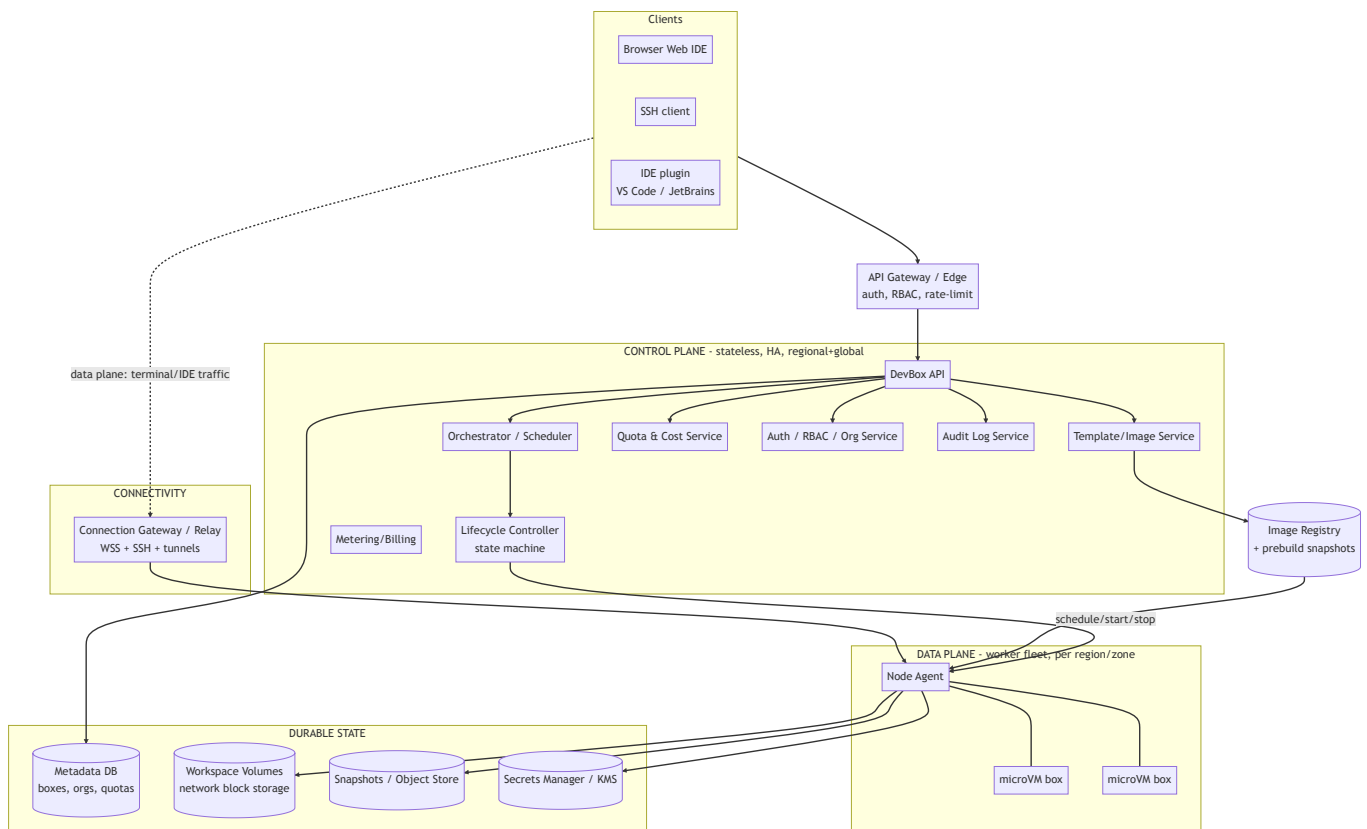
Quantity	Assumption	Result
Developers (MAU)	—	1,000,000
Concurrent active boxes	~5% online at peak	~50,000 concurrent
Provisions/day	each dev starts ~2x/day	2M/day ~**25/sec avg, ~200–500/sec peak**
Avg box size	4 vCPU, 8–16 GB RAM	50k × 4 = ~200k vCPU , ~600 TB RAM live
Host packing	64 vCPU / 256 GB hosts, ~8 boxes/host	~6,000–8,000 hosts at peak
Workspace storage	~20 GB/box, 2M total boxes	~40 PB persistent volumes (tiered)
Image/prebuild cache	hundreds of templates × layers	TBs of cached images per region/node
Idle ratio	most boxes idle most of the time	auto-pause is the #1 cost lever

Takeaways that drive the design:

1. **Hundreds of provisions/sec at peak** → cold-build is too slow → **warm pools + prebuilt snapshots**.
2. **600 TB live RAM, 40 PB disk** → separate **compute (ephemeral)** from **storage (durable)**; never co-fate them.
3. **Most boxes idle** → aggressive **auto-pause/hibernate** + bin-packing reclaims huge cost.
4. **Many tenants per host** → **microVM isolation** is non-negotiable.

4. Architecture overview — control plane vs data plane

The single most important cut: a **stateless, globally-aware control plane** that orchestrates, and a **regional data plane** of worker hosts that actually run boxes.



Why this split?

- **Control plane is stateless & HA** — it holds *desired state* in a metadata DB and reconciles the fleet (Kubernetes-operator pattern). It can restart/scale without touching running boxes.
- **Data plane runs the workloads** — worker hosts with a **node agent** that launches microVMs, mounts volumes, fetches images, and reports health.
- **Data-plane traffic bypasses the control plane** — terminal/IDE bytes flow client ↔ **connection gateway** ↔ box, so user latency doesn't depend on (and isn't bottlenecked by) the orchestrator.

Core components

- **DevBox API** — CRUD + lifecycle commands; validates against quota/RBAC.
- **Orchestrator/Scheduler** — places boxes on hosts (bin-packing, GPU/zone affinity), manages **warm pools** and node autoscaling.
- **Lifecycle Controller** — drives the box **state machine**; reconciles desired vs actual.
- **Template/Image Service** — builds/stores templates, **prebuilds**, and snapshots.
- **Node Agent** — per-host daemon: start/stop microVMs, attach storage, stream health/metrics, enforce cgroups.
- **Connection Gateway/Relay** — authenticated tunnel terminating browser WSS, SSH, and IDE connections; routes to the right box (even behind NAT).
- **Quota/Cost, RBAC/Org, Audit, Metering/Billing** — the multi-tenant control surfaces.

5. Data model & storage choices (polyglot persistence)

Principle: separate compute from state. The box's *compute* is **cattle (recreatable)**; its *workspace* is the **pet that must persist**.

Data	Store	Why
Boxes, orgs, teams, templates, quotas	Relational (Postgres, HA)	Strong consistency, transactions, relational RBAC/quota
Box runtime state / placement	Postgres + in-mem cache (or etcd)	Fast reconcile loop; source of truth for desired state
Workspace / home dir	Network block volume (EBS-like) per box	Durable, detach/reattach across hosts, snapshot-able
Base images & prebuilds	OCI image registry + content-addressed layer cache on nodes	Fast pulls, dedupe layers, cache near compute
Snapshots (disk ± memory)	Object store (S3/GCS)	Cheap, durable, for pause/resume & fast clone
Secrets	Secrets manager + KMS envelope encryption	Never in images or the DB in plaintext; short-lived injection
Audit logs	Append-only log → immutable store / SIEM	Tamper-evident, queryable, retention
Metrics/Logs/Traces	TSDB (Prometheus) + log store (Loki/ELK) + tracing	Observability + cost metering

Box record (metadata)

```
box {
  id, org_id, team_id, owner_id,
```

```

template_id, state, // CREATING/RUNNING/STOPPED/PAUSED/...
resources { vcpu, mem_gb, disk_gb, gpu },
host_id (nullable when not running),
volume_id, snapshot_id,
repo { url, ref, checkout_path },
created_at, last_active_at, auto_stop_after, cost_to_date
}

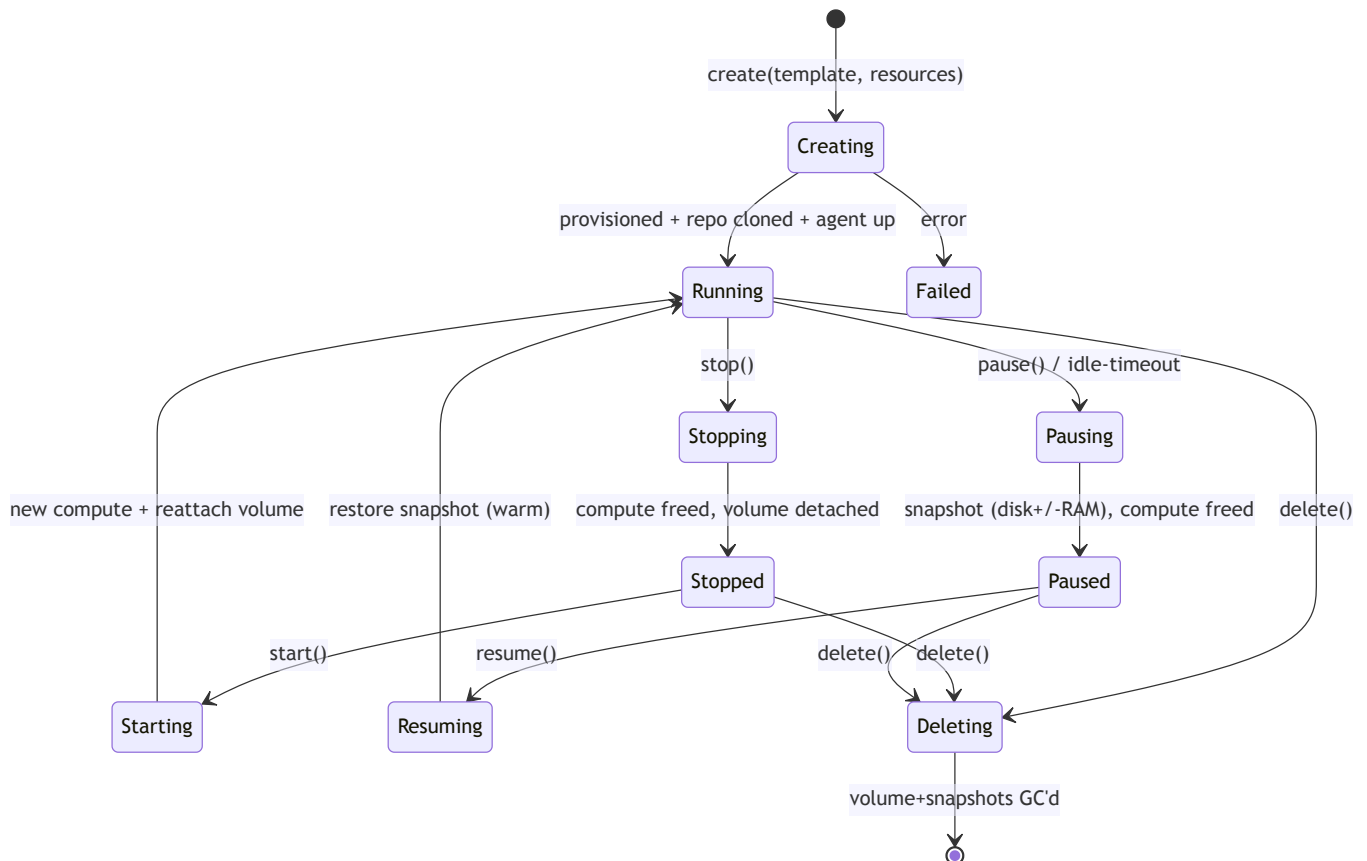
```

Workspace volume layout (what persists)

/home/<user>	← user home: dotfiles, shell history, configs	(PERSIST)
/workspace	← repo checkout + uncommitted work	(PERSIST)
/var, /usr, OS	← from base image, reconstructable	(EPHEMERAL)
/tmp, build cache	← scratch	(EPHEMERAL, maybe cached)
secrets	← injected at runtime via KMS, not on disk	(NEVER persisted in clear)

The durable volume holds **/home + /workspace**. The OS/toolchain come from the (immutable, cached) base image. This separation is what makes **stop = detach volume + kill compute** and **start = new compute + reattach the same volume** safe and fast.

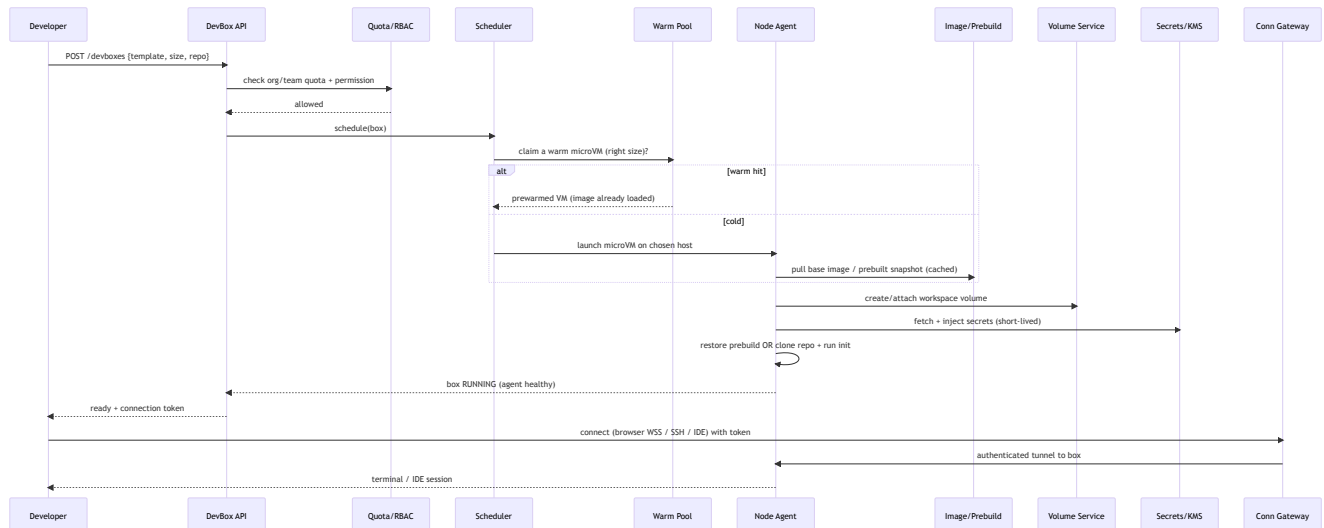
6. Lifecycle — the box state machine



Action	Compute	Workspace volume	RAM state	Cost	Resume speed
Stop	Freed	Detached, kept	Lost	Storage only	Seconds (boot + reattach)
Pause	Freed	Kept (or snapshotted)	Snapshotted	Storage only	Sub-second–seconds (restore)
Resume	New host	Reattach	Restored	Resumes compute	Fast
Delete	Freed	GC'd	—	Stops	n/a

- **Stop** = graceful: flush disk, **detach** the network volume, release the host. Cheap to keep around.
- **Pause/hibernate** = snapshot the running state (disk and optionally **memory**), so resume drops you back exactly where you left off (open files, running servers). Memory snapshots make resume feel instant but cost more storage.
- **Idle auto-pause** is driven by the node agent watching CPU/tty/IDE activity → the biggest cost saver.

7. Data flow: create → provision → connect



The provisioning critical path (and how we shrink each step):

1. **Schedule placement** — pick a host (bin-pack, zone/GPU affinity) → ms.
2. **Get compute** — **warm pool** hit = already-booted microVM → ~0; cold = launch microVM (Firecracker boots in ~125 ms).
3. **Get the filesystem** — **prebuilt snapshot** of the template (deps already installed) instead of `apt/npm install` at runtime → seconds vs minutes.
4. **Get the code** — repo already in the prebuild, or a fast shallow clone; cache the git mirror near the region.
5. **Inject secrets** — pulled from KMS at boot, mounted in-memory (tmpfs), never baked into the image.
6. **Signal ready + connect** — hand back a connection token; traffic flows through the gateway.

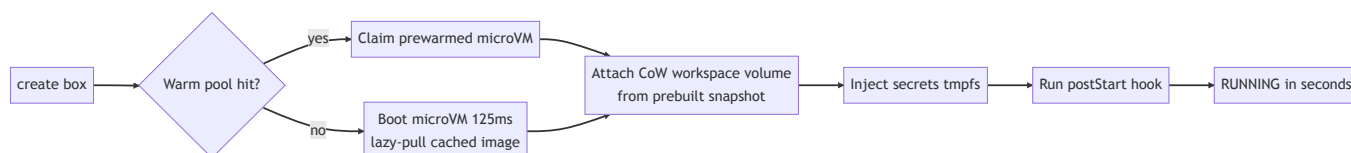
8. Fast provisioning (deep dive — a stated goal)

Cold "boot OS → install toolchain → clone repo → npm install" = minutes. Users expect **seconds**. Layered techniques:

1. **Prebuilds (the big one)**. Build the template *ahead of time* — OS + toolchain + dependencies + repo + `postCreate` hooks — and store the result as a **snapshot/image**. Provisioning becomes *restore a snapshot*, not *run a build*. Rebuild prebuilds on template/branch changes (CI-style), so the common case is always warm.
2. **Warm pools**. Keep a buffer of **already-booted** generic microVMs per popular size/template. A create *claims* one and personalizes it (attach volume, inject secrets) instead of booting cold. The orchestrator autoscaler keeps the pool sized

to demand (predictive: weekday-morning surge).

3. **microVM fast boot.** **Firecracker** microVMs boot in ~125 ms with a minimal kernel; pair with **snapshot/restore** to resume a pre-initialized VM state directly.
4. **Image/layer caching on nodes.** Content-addressed OCI layers cached on each host (and a regional pull-through cache) → near-zero pull for shared bases. **Lazy-pull** image layers (stream blocks on first access) so the box starts before the whole image is local.
5. **Copy-on-write storage.** Clone a box/template volume via **CoW snapshots** (overlay/ZFS/btrfs/EBS fast-snapshot) — instant "copy" of a 20 GB workspace without copying bytes.
6. **Local git mirror/cache.** Maintain a regional mirror of popular repos; shallow/partial clone (`--depth=1`, sparse checkout) to minimize fetch.



One-liner: "I make the common path a **restore, not a build** — prebuilt template snapshots + a warm pool of Firecracker microVMs + copy-on-write volumes turn minutes of `npm install` into a couple of seconds."

9. Strong isolation (deep dive — a stated goal)

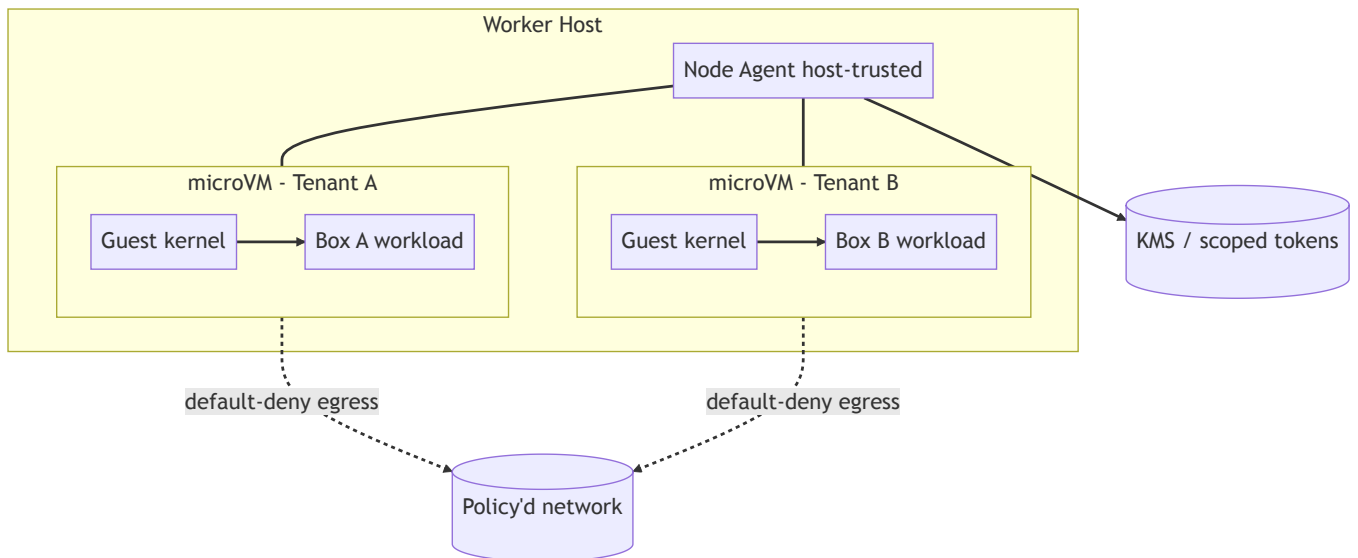
We run **untrusted code from many tenants on shared hosts**. Containers alone share the host kernel → a kernel exploit escapes to other tenants. Options:

Isolation	Boundary	Startup	Density	Security	Use
Plain containers (namespaces/cgroups)	Shared kernel	~ms	Highest	Weak for untrusted multi-tenant	Trusted workloads only
gVisor (userspace kernel)	Syscall interception	~ms	High	Medium-strong	Good middle ground
Kata / microVM (Firecracker) ✓	Own guest kernel + VM	~125 ms	High	Strong (VM-grade)	Multi-tenant DevBoxes
Full VM	Hypervisor	~10s+	Low	Strong	GPU/heavy, dedicated

Recommendation: microVMs (Firecracker/Kata). Each box gets its **own kernel** inside a lightweight VM — VM-grade isolation with near-container speed and density. This is what Codespaces/Fly/AWS Lambda use for exactly this reason.

Defense in depth around the microVM:

- **Per-box network isolation** — its own veth/VLAN/overlay; **default-deny egress** with allowlists (block metadata endpoints, SSRF to cloud IAM, lateral movement). Boxes can't see each other.
- **Resource caps** — cgroups/VM limits on CPU, memory, **disk + network IO** to stop noisy-neighbor and fork-bomb DoS.
- **No host credentials in the guest** — the node agent (host side) handles cloud APIs; the box only gets **scoped, short-lived** tokens.
- **Seccomp/least-privilege** on the agent; read-only base image; secrets in tmpfs, wiped on stop.
- **Per-tenant encryption** — volumes and snapshots encrypted with tenant-scoped KMS keys.

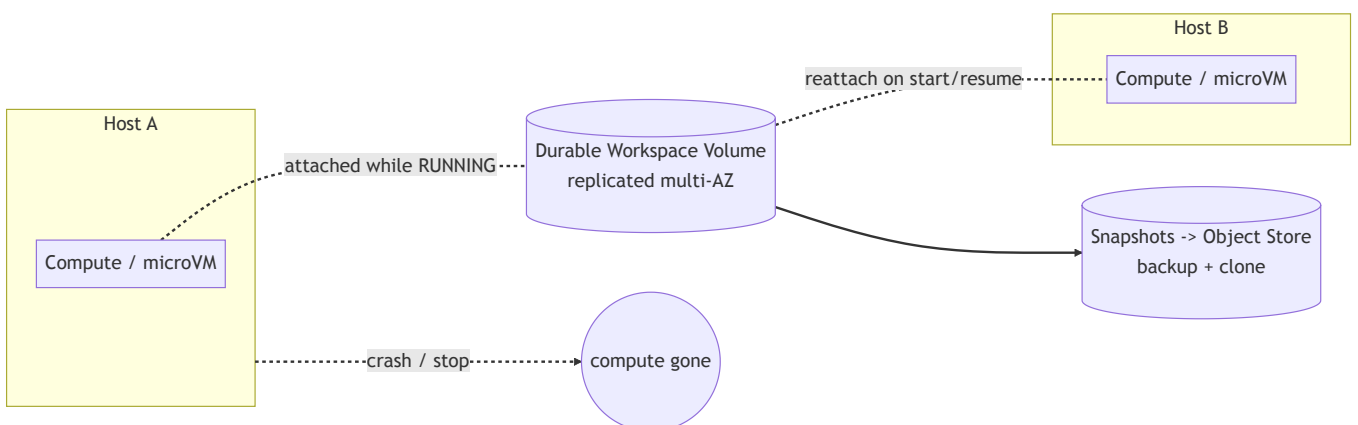


One-liner: "Each box is a Firecracker microVM with its **own kernel**, default-deny egress, cgroup IO caps, and only short-lived scoped tokens — so a `sudo rm -rf` or a kernel exploit stays inside one disposable VM."

10. Reliable persistence (deep dive — a stated goal)

A developer's uncommitted work **must not vanish** on stop, pause, host crash, or rescheduling.

- **Decouple storage from compute.** The workspace lives on a **network block volume** (EBS-like), independent of the host. Compute is disposable; the volume **detaches on stop and reattaches on start**, possibly on a *different* host. Host dies → reattach the volume elsewhere; no data loss.
- **Persist the right paths** — `/home` + `/workspace` on the durable volume; OS/toolchain come from the immutable image (reconstructable).
- **Snapshots** for pause/hibernate, fast cloning, and point-in-time backup → object store. Memory snapshot (optional) for instant resume.
- **Durability of the volume itself** — replicated block storage (multi-AZ) so a single disk/AZ failure doesn't lose data; periodic snapshot backups for DR.
- **Crash consistency** — `fsync`/journaling + filesystem-consistent snapshots; flush on graceful stop. Warn users uncommitted work isn't a substitute for `git push`.
- **Garbage collection** — on delete, GC volume + snapshots after a grace/retention window (undo-delete window). Reap orphaned volumes from failed creates.

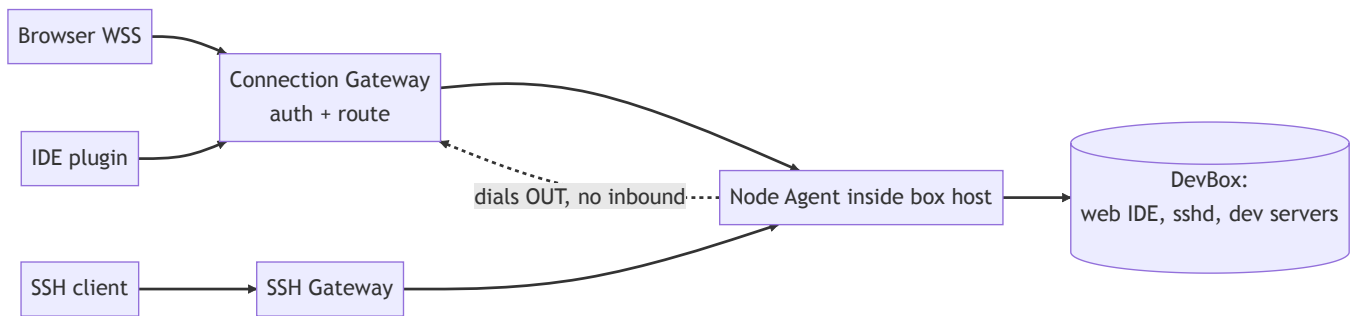


One-liner: "Compute is cattle, the workspace is a pet — the home dir and repo live on a replicated network volume that detaches and reattaches across hosts, so a host crash costs you a reschedule, not your code."

11. Connectivity — browser, SSH, IDE (the data plane)

How does a user reach a box that's on some random host, possibly behind NAT, securely?

- **Connection Gateway / Relay** — a globally reachable, authenticated entry point. The **node agent dials out** to the gateway and holds a persistent tunnel (WebSocket/QUIC), so boxes need **no public IP / inbound ports** (NAT-friendly, smaller attack surface).
- **Browser (web IDE)** — HTTPS/WSS to the gateway → routed to the box's agent → web IDE server (e.g., code-server) and terminal over a multiplexed channel.
- **SSH** — an **SSH gateway/proxy** terminates auth (short-lived certs / keys mapped to identity) and proxies to the box. Users get `ssh box-name@platform`.
- **IDE plugins** — VS Code Remote / JetBrains Gateway run a **remote agent inside the box** and tunnel through the same relay; the local IDE is just a thin client.
- **Port forwarding** — expose a dev server port via a per-box subdomain `https://3000-box-id.platform.dev`, authenticated by RBAC.
- **Identity-aware** — every connection is authenticated and authorized (Zero-Trust); the gateway checks RBAC before bridging, and all sessions are audited.



12. Multi-tenancy: orgs, RBAC, quota, audit, cost

Org / team / RBAC

- Hierarchy: **Org** → **Teams** → **Users**, with **roles** (Owner, Admin, Member, Viewer) and **permissions** (create box, use template, manage quota, view others' boxes).
- Enforce at the API gateway on **every** lifecycle call. RBAC also gates **connection** (can this identity attach to this box / forwarded port?).

Quota enforcement

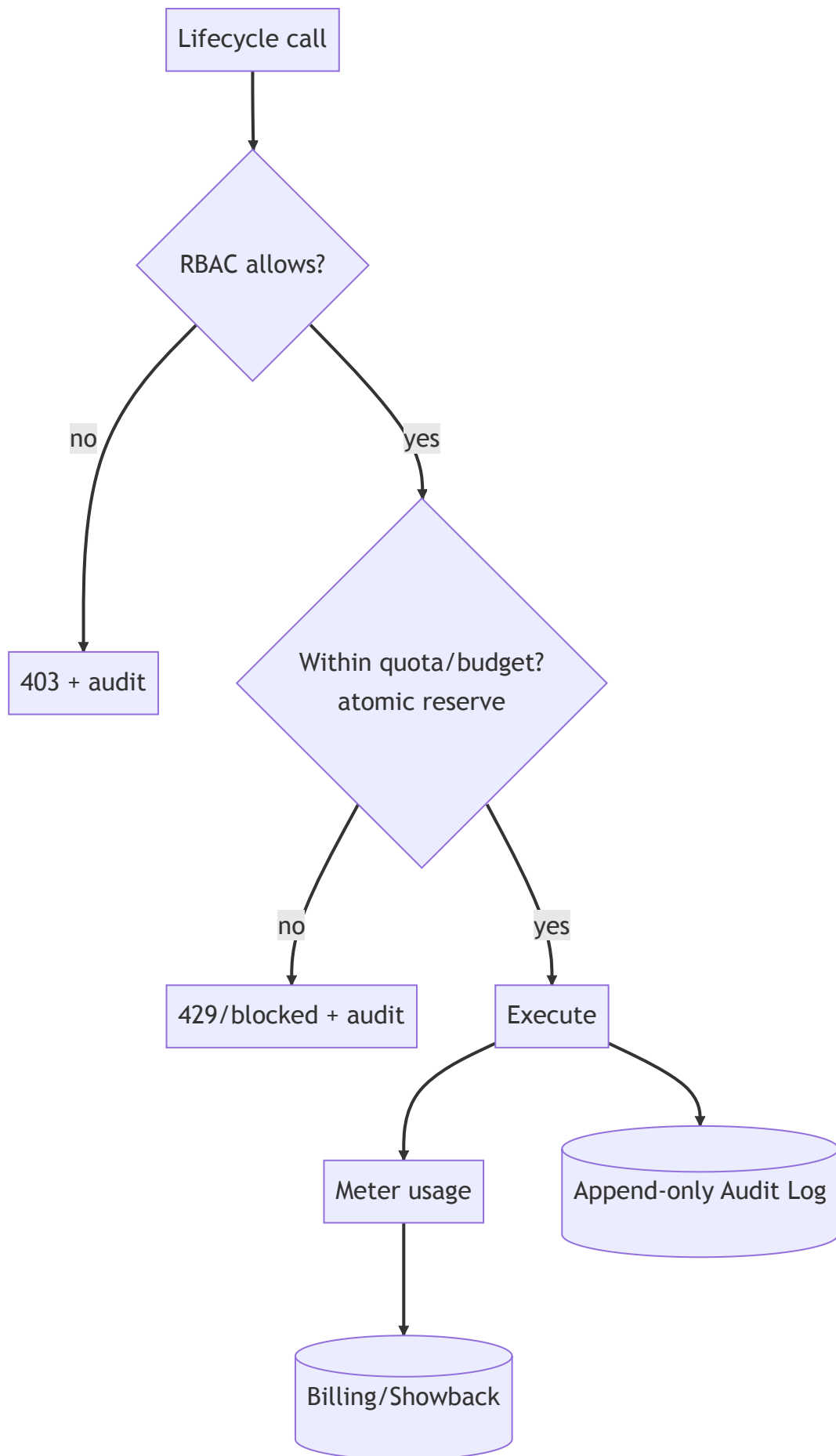
- Limits per org/team/user: **max concurrent boxes, total vCPU/RAM/GPU, storage GB, monthly spend**.
- Checked **transactionally at create/start/resize** (reserve → commit) so two concurrent creates can't both pass the last slot (the read-modify-write race — use atomic reserve).
- Soft (warn) vs hard (block) limits; templates can pin allowed sizes per tier.

Audit logs

- **Append-only, immutable** record of who did what: create/start/stop/delete, **connect/SSH sessions**, secret access, role changes, quota overrides. Tamper-evident, exported to SIEM, with retention. Essential for security and compliance.

Cost controls (huge given idle ratios)

- **Auto-stop/auto-pause** idle boxes after N minutes (agent-detected inactivity) — the #1 lever.
- **Auto-delete** disposable boxes after TTL; expire stopped boxes after a retention window.
- **Budgets & alerts** per org/team; **metering** every box's vCPU·hr, GB·hr, GPU·hr → showback/chargeback and billing.
- **Right-sizing** suggestions; **bin-packing** + node autoscaler scales the fleet down off-hours; **spot/preemptible** hosts for disposable boxes (with checkpointing).



13. Observability (a stated goal)

- **Metrics** — control-plane (provision latency p50/p99, success rate, warm-pool hit rate, scheduler queue) and per-box (CPU/mem/disk/net, idle state) in a TSDB; SLOs + alerting.
- **Logs** — structured agent/box/control-plane logs centralized; per-tenant scoping.
- **Tracing** — distributed trace of the **provisioning path** (API → scheduler → agent → ready) to find the slow step.
- **Health & self-healing** — agent heartbeats; the lifecycle controller reconciles (restart crashed boxes, reschedule off dead hosts, reap orphans).
- **Cost observability** — usage metering doubles as the billing/showback signal.
- **Golden signal**: "time-to-ready" (create→connected) is the product's headline SLO.

14. Failure scenarios — "what if X fails?"

Failure	Impact	Mitigation
Worker host crashes	Running boxes on it die	Volume is network-attached → reschedule box on a healthy host + reattach; RAM-only state lost (recover from last snapshot)
Control plane down	Can't create/manage boxes	CP is stateless+HA (multi-replica, multi-AZ); running boxes keep running (data-plane independent)
Connection gateway down	Can't reach boxes	Multiple gateways, health-checked, client reconnect; boxes redial
Image registry slow/down	Cold provisions stall	Node-local + regional cache; warm pool absorbs; lazy-pull
Volume/storage AZ failure	Risk to persistence	Multi-AZ replicated volumes + snapshot backups (DR restore)
Quota race (double-spend)	Over-allocation	Atomic reserve (transactional/conditional) at create/start
Noisy neighbor / fork bomb	Host degraded	cgroup CPU/mem/IO caps per microVM; per-box network limits
Tenant escape attempt	Cross-tenant breach	microVM (own kernel) + default-deny egress + no host creds in guest
Provisioning storm (Mon 9am)	Latency spike	Predictive warm-pool scaling + node autoscaler + admission queue
Stuck/orphaned resources	Cost leak	Reconcile loop GCs orphans; idle auto-stop; TTLs

Guiding principle: the **data plane must outlive the control plane**, and **storage must outlive compute**. A control-plane outage should never disconnect a coding developer or lose their work.

15. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Isolation vs density/speed	Full VM (strong, slow, low density)	Container (fast, dense, weak)	microVM (Firecracker) — VM-grade isolation at near-container speed ✅
Provision speed vs cost	Big warm pool (instant)	No pool (cheap, slow)	Pool sized to predicted demand; CoW + prebuilds get speed without huge idle cost
Pause: disk-only vs disk+RAM	Stop (cheap, slower resume)	Hibernate (instant resume, more storage)	Offer both; default stop, hibernate for premium/active boxes
Persistence vs cost	Keep all volumes forever	Aggressive GC	Retention window + snapshot-to-object-store for cold boxes
Multi-tenant hosts vs dedicated	Shared (cheap, dense)	Dedicated per org (isolation, cost)	Shared + microVM by default; dedicated for high-security tiers
Spot vs on-demand hosts	Spot (cheap, preemptible)	On-demand (stable)	Spot for disposable boxes w/ checkpoint; on-demand for persistent
Consistency vs availability	Strong CP everywhere	Eventual reconcile	Strong for quota/RBAC/billing; eventual reconcile for fleet state

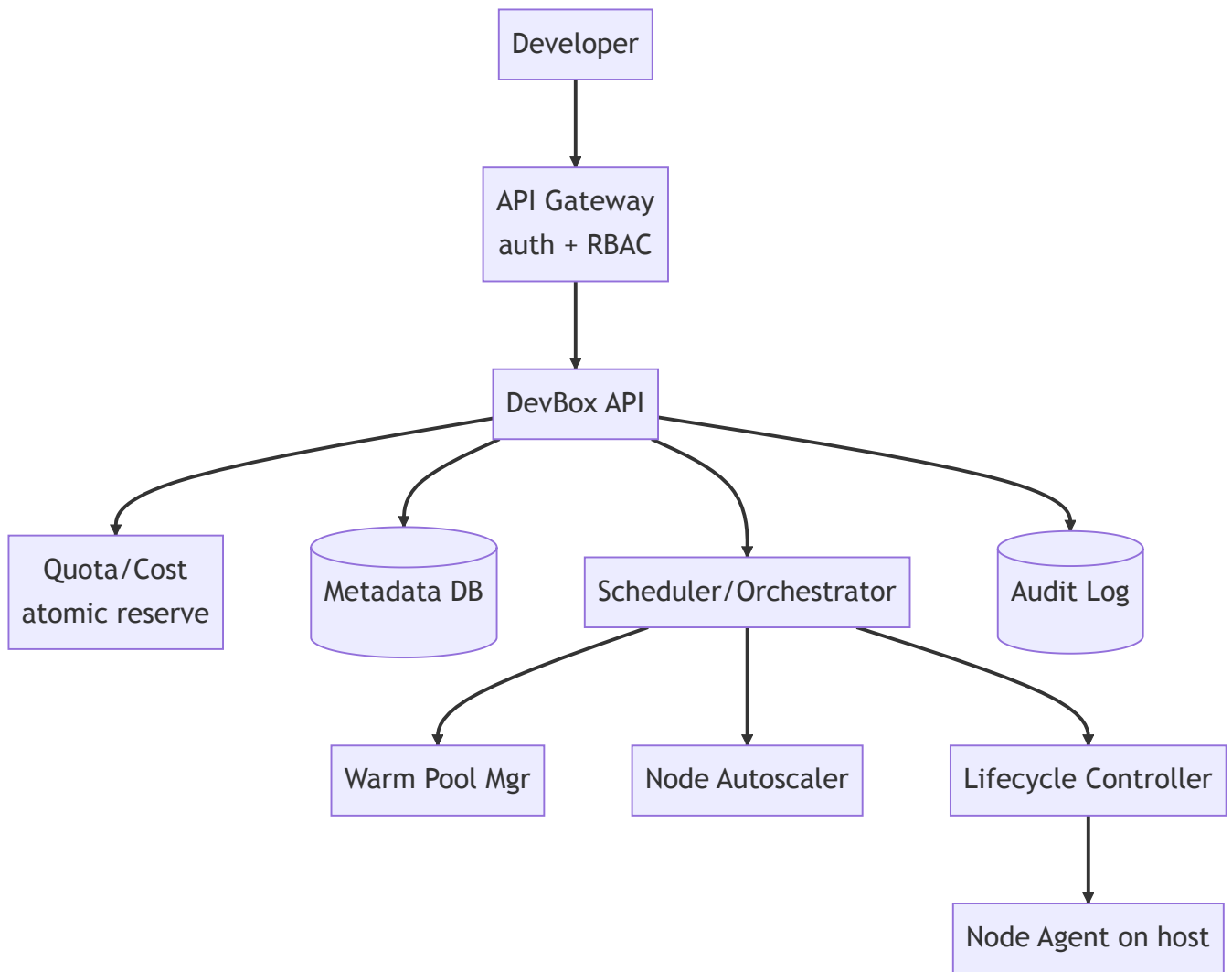
CAP/PACELC framing: the **metadata/quota/RBAC** store is **CP** (must be correct — no double-spend, no privilege bypass). The **fleet reconciliation** is **eventually consistent** (controllers converge desired→actual). The **data plane** is designed to **stay available** even when the control plane is partitioned away.

One-liner to say out loud: "I'd split a **stateless HA control plane** (API, scheduler, lifecycle controller, quota/RBAC/audit on Postgres) from a **regional data plane** of worker hosts running each box as a **Firecracker microVM** for strong isolation. Fast provisioning comes from **prebuilt template snapshots + warm pools + copy-on-write volumes**, turning minutes into seconds. The **workspace lives on a replicated network volume** decoupled from compute, so stop/pause/host-failure never lose work — compute is cattle, the volume is the pet. Users connect through an **outbound-dialed connection gateway** (browser/SSH/IDE) so boxes need no inbound ports. Quotas are enforced with **atomic reserves**, everything is **audited append-only**, and **idle auto-pause + bin-packing** controls cost."

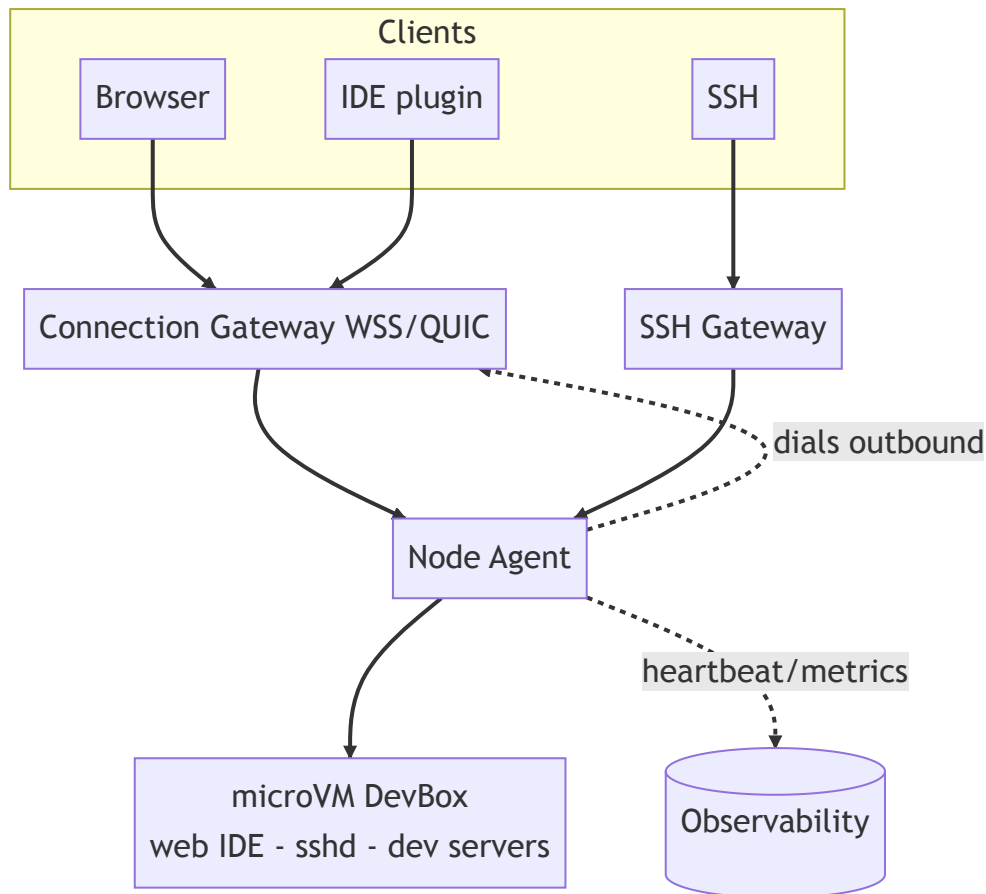
16. Full system design (detailed)

End-to-end, split into **(A)** the provisioning/control path, **(B)** the running/connect data path, and **(C)** the persistence & lifecycle path.

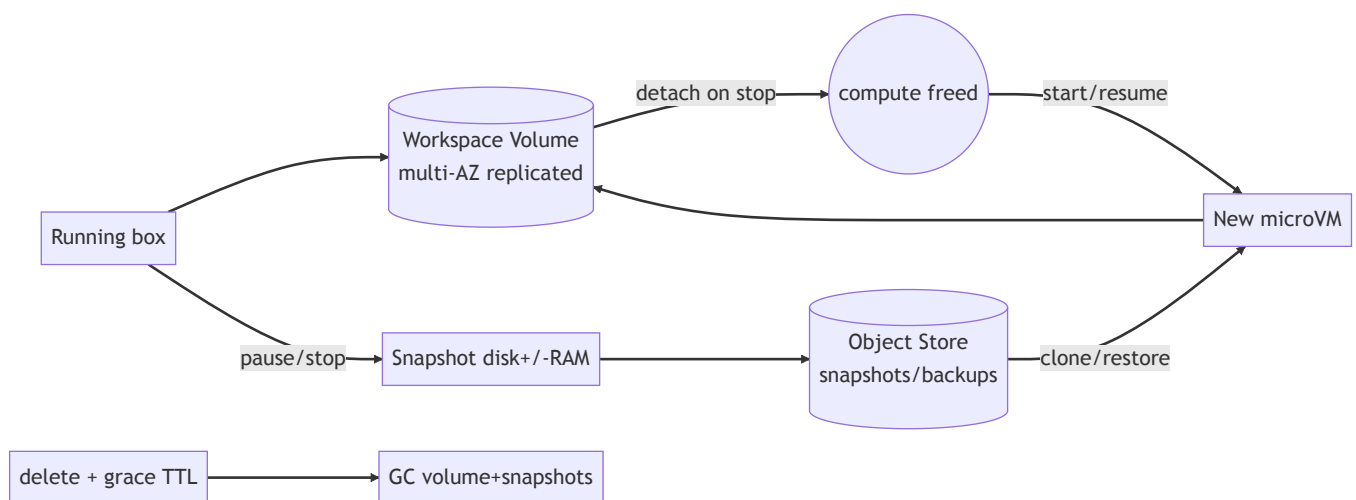
16A. Control path — create & schedule



16B. Data path — running box & connectivity



16C. Persistence & lifecycle path



17. Networking, security & performance best practices

Networking

- **Outbound-dialed connection gateway** — boxes hold a tunnel out; **no inbound ports** (NAT-friendly, smaller attack surface).
- **WSS/QUIC** for browser/IDE traffic; **SSH via a proxy** with short-lived certs mapped to identity.

- **Per-box network namespace/overlay**; **default-deny egress** + allowlist; **block the cloud metadata endpoint** (SSRF/IAM-credential theft).
- **Co-locate** compute with the workspace volume's AZ to minimize attach latency; per-box auth-gated subdomain port-forwarding.

Security

- **microVM per box** (own kernel); **no host/cloud creds in the guest** — the agent holds them and hands out short-lived scoped tokens.
- **Secrets from KMS** injected to tmpfs, wiped on stop; **per-tenant volume/snapshot encryption**.
- **Identity-aware / Zero-Trust gateway** — RBAC checked before bridging; **all sessions audited**.
- **Read-only base image, signed images** (cosign/provenance), seccomp + least-privilege agent.
- **cgroup CPU/mem/IO caps** to contain noisy neighbors and abuse (crypto-mining).

Performance

- **Prebuilt snapshots + warm pools + CoW volumes + lazy image pull** → seconds to ready ("restore, not build").
- **Firecracker snapshot/restore** for instant resume; node-local + regional **image layer cache**.
- **Idle auto-pause + bin-packing** for cost/perf; right-size suggestions.

18. Staying current — modern & emerging approaches

- **Isolation: Firecracker / Kata Containers** / Cloud Hypervisor microVMs; **gVisor** for syscall-level isolation.
- **Reference systems:** GitHub **Codespaces** (devcontainers), **Gitpod**, **Coder**, AWS (Lambda/Fargate on Firecracker).
- **Env as code:** `devcontainer.json` (Dev Container spec), **Nix/Nixpkgs** for reproducible toolchains.
- **Networking:** WireGuard/**Tailscale**-style mesh, **Cloudflare Tunnel**, relay-based reverse tunnels.
- **Storage:** CSI drivers, EBS multi-attach / fast snapshots, **ZFS/btrfs** copy-on-write.
- **IDE protocols:** VS Code Remote, JetBrains Gateway, SSH; browser IDE via code-server.
- **How I stay current:** Firecracker/Kata release notes, KubeCon talks, Codespaces/Gitpod engineering blogs.

19. Likely follow-up questions (rehearse these)

- microVM vs container vs gVisor — why microVM for untrusted code? (*own kernel = VM-grade isolation*)
- Host dies mid-session — does the user lose work? (*no — workspace is a network volume, reattach elsewhere*)
- How do you get sub-second provisioning? (*prebuilt snapshot + warm pool + CoW volume*)
- Pause vs stop — difference? (*pause snapshots RAM for instant resume; stop is disk-only*)
- How does a browser reach a box behind NAT? (*box dials out to the gateway; no inbound*)
- Stop a tenant stealing node IAM creds? (*block metadata endpoint; no host creds in guest*)
- Quota double-spend on concurrent creates? (*atomic/transactional reserve*)
- Cost control for idle boxes? (*idle auto-pause, TTL auto-delete, spot for disposable*)

20. Summary checklist (whiteboard recap)

- **Control plane vs data plane** — stateless HA orchestrator + regional worker fleet; data plane outlives the control plane.
- **microVM isolation** (Firecracker) — own kernel per box, default-deny egress, no host creds in guest, IO caps.
- **Fast provisioning** — prebuilt snapshots + warm pools + CoW volumes + lazy image pull = seconds, not minutes ("restore, not build").
- **Reliable persistence** — workspace on a **replicated network volume** decoupled from compute; snapshots for pause/clone/DR; compute=cattle, volume=pet.

- **Lifecycle state machine** — stop (detach volume), pause (snapshot RAM), resume (restore), delete (GC).
- **Connectivity** — outbound-dialed gateway for browser/SSH/IDE; no inbound ports; identity-aware + audited.
- **Multi-tenancy** — Org/Team/RBAC, **atomic quota reserves**, append-only **audit**, metering → billing.
- **Cost controls** — idle auto-pause, TTL auto-delete, bin-packing + autoscaling, spot for disposable.
- **Observability** — provision-latency/time-to-ready SLO, per-box metrics, tracing the provisioning path, self-healing reconcile.
- **CAP** — CP for quota/RBAC/billing; eventual reconcile for fleet; available data plane under CP partition.

Designing an Instagram-like Photo/Video Sharing App (Feed-Focused)

A complete, interview-ready walkthrough: requirements → estimates → APIs → data model → data flow → **feed generation (push/pull/hybrid)** → celebrities → read scaling → failure handling → trade-offs. Use the headings as your whiteboard agenda. The **feed** is the heart of this problem — spend most of your time there.

0. How to drive the interview (talk track)

1. **Clarify** functional + non-functional requirements (don't assume).
2. **Estimate** scale (DAU, posts/sec, feed reads/sec, fan-out, storage) to justify choices.
3. **Define the APIs** and core components.
4. **Walk the data flow**: post creation → media pipeline → storage → feed build → delivery.
5. **Pick storage** per data type (posts, media, follow graph, feeds) and justify.
6. **Feed strategy**: push (fan-out-on-write) vs pull (fan-out-on-read) vs **hybrid** — trade-offs.
7. **Handle celebrities** (millions of followers) — the fan-out bomb.
8. **Scale reads**: caching, replicas, precomputation; avoid feed-read bottlenecks.
9. **Handle failure** and summarize **trade-offs**.

Keep saying *"here's the trade-off..."* — that's what's being graded.

1. Problem & motivation

Build a photo/short-video sharing service where users **post media**, **follow** other users, and consume a **home feed** of recent + relevant posts from accounts they follow.

What makes it hard:

- **Read-heavy**: feed reads dominate writes by ~100:1. Feeds must load in tens of ms.
- **Write fan-out**: one post by a popular account must reach millions of followers' feeds.
- **Media-heavy**: photos/videos need transcoding, multi-resolution storage, and global CDN delivery.
- **Skewed graph**: follower counts span from 10 to 100M+ (celebrities) — no single strategy fits all.
- **Relevance**: not just chronological — ranking drives engagement.

The central tension: **do work at write time (precompute feeds) or read time (assemble on demand)**? The answer is *both*, chosen per-account — that's the hybrid design.

2. Requirements

Functional

- Users **create posts** (1+ photos or a short video, caption, location, tags).
- Users **follow / unfollow** other users (asymmetric — like Twitter, unlike Facebook).
- **Home feed**: recent and relevant posts from followed accounts, paginated, infinite scroll.
- Users can **like / comment** (engagement signals feed ranking).

- View a **user profile** (their own posts, reverse-chronological).

Out of scope (state it, then park it)

Stories, DMs, Reels recommendation/discovery (non-followed content), search, ads. Mention them so the interviewer knows you know, then focus on the feed.

Non-functional

- **Low feed latency**: p99 feed load < ~200 ms.
- **High availability**: feed must always render (stale-but-available > unavailable).
- **Scalable**: 500M DAU, billions of feed reads/day, 100M+ posts/day.
- **Eventual consistency is OK**: a new post appearing in followers' feeds a few seconds late is fine. (No need for read-after-write on *others'* feeds.)
- **Durability**: never lose a user's uploaded media or post.

Clarifying questions to ask the interviewer

- **Chronological or ranked** feed? (Assume ranked, with recency as a strong signal.)
- **Follower distribution** — do we need to support celebrity accounts (100M+)? (Yes → hybrid.)
- Read-after-write for the **author's own** feed/profile? (Yes — they must see their post instantly.)
- **Media limits** — max video length, max photo size, supported formats?
- **Global** or single-region? (Global → CDN + regional replicas.)
- Do we need **edit/delete** of posts? (Yes — affects cache invalidation.)

3. Back-of-the-envelope estimation

Assume Instagram-ish scale:

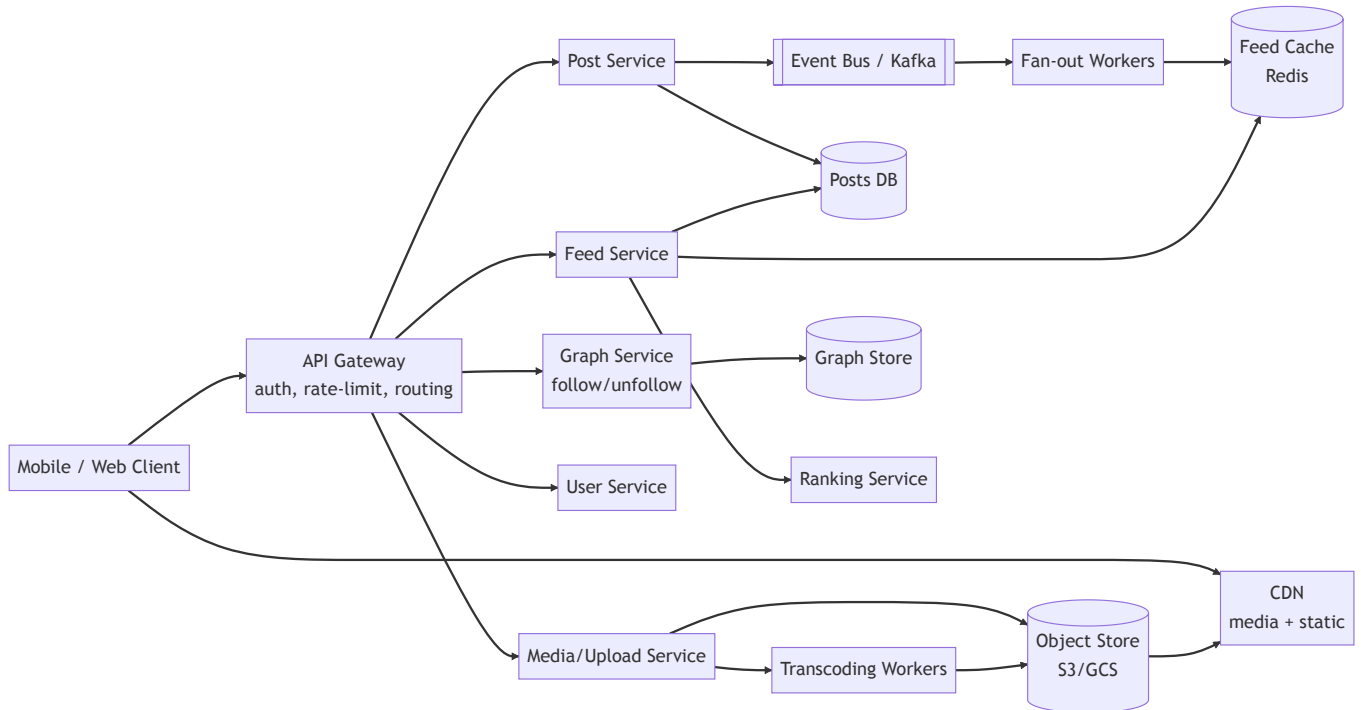
Quantity	Assumption	Result
DAU	—	500M
Posts/day	~20% of DAU post, ~1 each	100M/day ≈ ~1.2K/sec avg, ~5–10K/sec peak
Feed reads/day	each DAU opens feed ~10–20x	5–10B/day ≈ **100K/sec** avg, ~500K–1M/sec peak
Read : Write ratio	—	~100 : 1 (read-optimize everything)
Avg followers	—	~200 (median far lower; mean skewed by celebs)
Fan-out writes	100M posts × 200 followers	~20B feed-inserts/day ≈ ~230K/sec avg, ~1M/sec peak
Media size	photo ~2 MB × multi-res ≈ 3 MB; video larger	100M × 3 MB ≈ **300 TB/day** ingested → ~100 PB/year
Follow graph	1B users × ~200 edges	~200B edges (~3–5 TB raw, more with indexes)
Precomputed feed cache	500M active × ~500 post-IDs × ~16 B	~4 TB in RAM (sharded Redis)

Takeaways that drive the design:

1. **100:1 read:write** → precompute and cache feeds aggressively.

2. **Fan-out peak ~1M/sec** with celebrity spikes of **100M inserts for a single post** → can't naively fan-out everyone → **hybrid**.
3. **300 TB/day media** → object storage + CDN, never in the DB.

4. Core components & APIs



Component responsibilities

- **API Gateway** — auth (OAuth/JWT), TLS, rate limiting, routing.
- **User Service** — profiles, auth, settings.
- **Graph Service** — follow/unfollow, "who follows X", "who does X follow"; serves the fan-out and pull paths.
- **Media/Upload Service** — issues pre-signed upload URLs, validates, triggers transcoding.
- **Transcoding Workers** — generate multiple resolutions/formats (thumbnail → 4K, HLS/DASH for video).
- **Post Service** — creates the post record, emits a `post.created` event.
- **Fan-out Workers** — consume `post.created`, write post-IDs into followers' feed caches (push path).
- **Feed Service** — assembles a user's home feed (merge precomputed + pulled celebrity posts → rank → paginate).
- **Ranking Service** — orders candidate posts by relevance.
- **Event Bus (Kafka)** — decouples writes from fan-out; absorbs spikes; enables replay.

Key APIs (REST-ish; gRPC internally)

```
# Create a post (media already uploaded → returns media IDs)
POST /v1/posts
{ "media_ids": ["m_123", "m_124"], "caption": "sunset 🌅",
  "location": "SF", "tags": ["#nofilter"] }
→ 201 { "post_id": "p_987", "created_at": 1717612800 }

# Get home feed (cursor pagination — never OFFSET)
GET /v1/feed?limit=20&cursor=eyJ0cyI6MTcx... # opaque cursor
→ 200 { "items": [ {post...}, ... ], "next_cursor": "eyJ0cyI6..." }
```

```
# Follow / unfollow
POST    /v1/users/{id}/follow
DELETE /v1/users/{id}/follow

# Request a pre-signed upload URL (client uploads media directly to object store)
POST /v1/media/upload-url
{ "type": "image/jpeg", "bytes": 2097152 }
→ 200 { "media_id": "m_123", "upload_url": "https://s3...&sig=...", "expires_in": 600 }

# Profile feed (author's own posts, reverse-chronological)
GET /v1/users/{id}/posts?limit=20&cursor=...
```

Why cursor pagination, not page/OFFSET? Feeds are append-mostly and huge. `OFFSET` scans and skips rows (slow, $O(N)$); a **cursor** (e.g., last (ranking_score, post_id) seen) is $O(1)$ to resume and stable as new posts arrive at the top.

5. Data model & storage choices

Principle: polyglot persistence — match each data shape to the right store.

Data	Store	Why
Posts (metadata)	Wide-column (Cassandra) or sharded SQL	Massive write volume, simple key access by post_id / user_id, time-ordered
Media blobs	Object store (S3/GCS) + CDN	Cheap, durable, infinitely scalable; never store blobs in a DB
Follow graph	Sharded KV / wide-column (or graph DB)	200B edges, need fast "followers of X" and "followees of X"
Feeds (precomputed)	Redis (in-memory)	Sub-ms reads, sorted by score, TTL/trim to bound size
Counters (likes/comments)	Redis + async durable rollup	High-frequency increments; eventual durability
Social/derived analytics	Data lake / OLAP (later)	Offline ranking model training

Posts table (Cassandra-style, partitioned by author for profile reads)

```
posts_by_user (
  user_id      partition key,
  created_at   clustering key DESC,  -- newest first
  post_id, media_ids, caption, location, tags, like_count, comment_count
)
posts_by_id (post_id PK → full post)  -- point lookups for feed hydration
```

Follow graph (store BOTH directions — denormalize for read speed)

```
followers_of (user_id → set/rows of follower_ids)  -- drives FAN-OUT (push)
```

```
following_of (user_id → set/rows of followee_ids) -- drives PULL + profile-of-followees
```

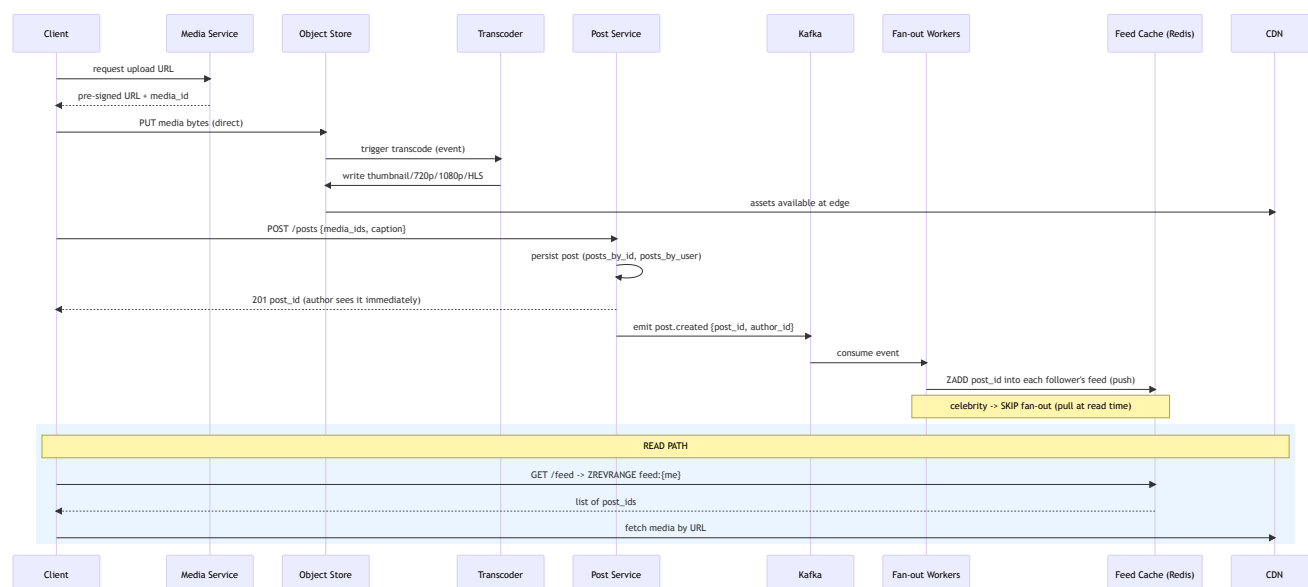
Storing both directions doubles writes on follow/unfollow (rare) to make reads cheap (constant). Classic **read-optimization via denormalization**.

Feed cache (Redis Sorted Set per user)

```
feed:{user_id} → ZSET of member=post_id , score=ranking_score (or timestamp)
ZADD feed:{u} <score> <post_id>          # fan-out insert
ZREVRANGE feed:{u} 0 19                  # top 20 for the home feed
ZREMRANGEBYRANK feed:{u} 0 -1001         # trim to newest/best ~1000
```

We store **only post-IDs** (pointers) in the feed, then **hydrate** full post + media URLs from `posts_by_id` + cache. Keeps the feed cache tiny and avoids duplicating post data per follower.

6. Data flow: post creation → storage → feed → delivery



Step-by-step:

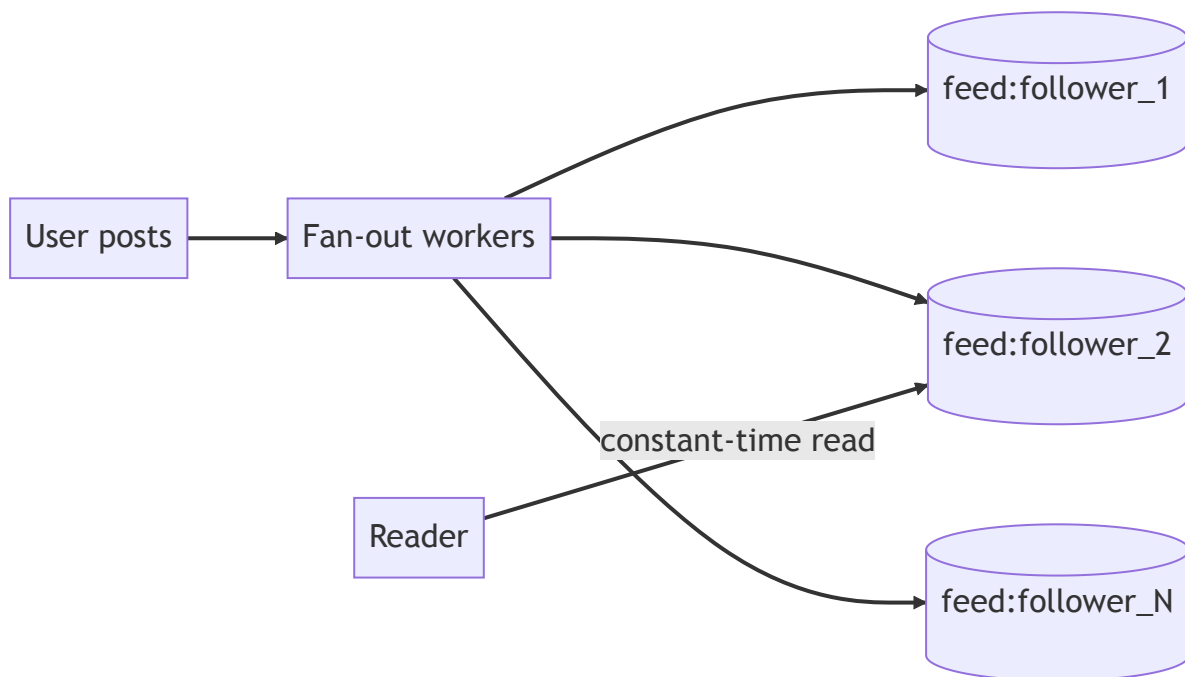
- Upload first, post second.** Client gets a pre-signed URL and uploads media **directly to the object store** (bypasses our app servers — saves bandwidth/compute). Returns `media_id`.
- Transcode asynchronously.** Object-store event triggers workers that produce a thumbnail, multiple resolutions, and HLS/DASH renditions for video. Assets land back in the object store, fronted by the **CDN**.
- Create post.** POST /posts writes metadata to the Posts DB and returns 201 immediately — **the author sees their own post right away** (read-after-write for self).
- Emit event.** Post Service publishes `post.created` to **Kafka** (decouples the slow fan-out from the user-facing request).
- Fan-out (push).** Workers look up the author's followers and ZADD the `post_id` into each follower's `feed:{user_id}` Redis ZSET — **unless** the author is a celebrity (see §8).
- Delivery (read).** On feed open, Feed Service reads the top-N post-IDs from Redis, **merges in** any followed-celebrity posts (pull), **ranks**, hydrates post/media, and returns. Media itself streams from the **CDN**, not our servers.

7. Feed generation strategies — push vs pull vs hybrid (the core)

This is where to spend the most time. Three approaches:

7.1 Push — fan-out-on-write (precompute feeds)

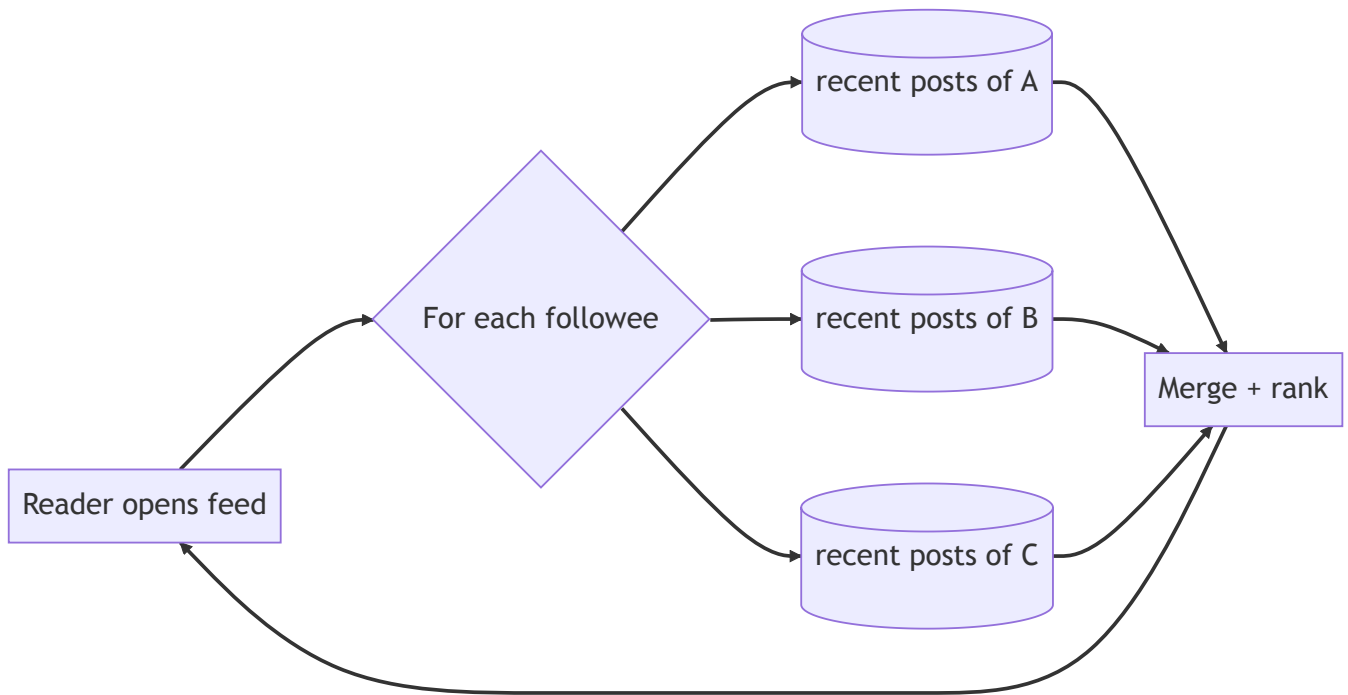
On post creation, **write the post-ID into every follower's feed** immediately.



- **✓ Reads are trivially fast** — feed is already materialized (one `ZREVRANGE`). Perfect for the 100:1 read-heavy reality.
- **✓ Read load is predictable and cache-friendly.**
- **✗ Write amplification:** 1 post → up to *millions* of writes. A celebrity with 100M followers = 100M Redis writes per post (the **fan-out bomb**).
- **✗ Wastes work for inactive followers** (we precompute feeds nobody reads).
- **✗ New follow/unfollow needs backfill/cleanup.**

7.2 Pull — fan-out-on-read (assemble on demand)

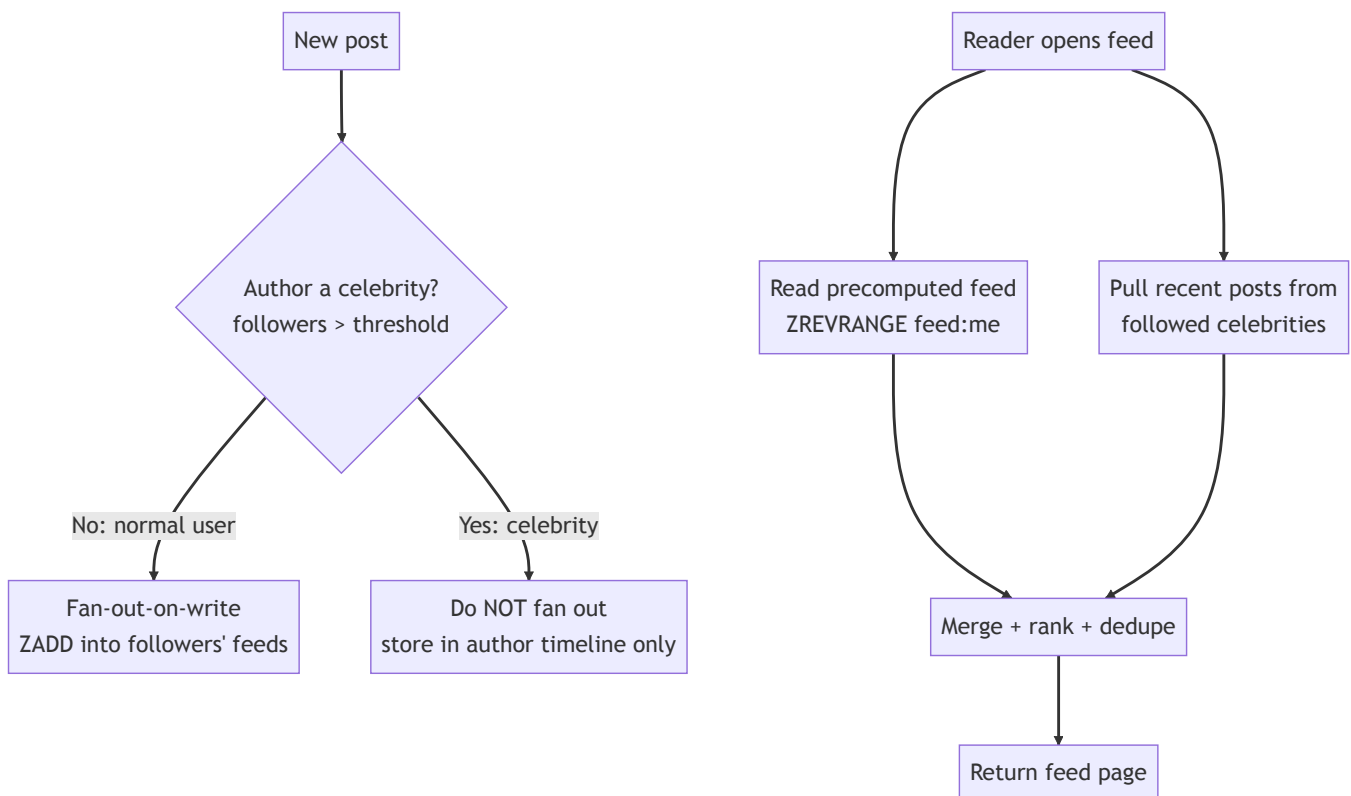
Store nothing per-follower. On feed open, **fetch recent posts from everyone the user follows**, merge, rank.



- **✓ No write amplification** — posting is cheap; great for celebrities.
- **✓ No wasted precomputation** for inactive users.
- **✓ Always fresh**; easy to re-rank.
- **✗ Reads are expensive**: a user following 1,000 accounts triggers many queries + a merge **on every feed open** → high latency, heavy DB load at 100K+ reads/sec.
- **✗ Hard to cache**; "thundering herd" on popular followees.

7.3 Hybrid — the real-world answer **✓**

Push for the masses, pull for celebrities. Split accounts by follower count.



- **Normal authors (followers < ~10K–100K): push** — fan out to followers' feeds. Cheap fan-out, instant reads for the long tail.
- **Celebrity authors (followers > threshold): don't fan out.** Their posts live in their own timeline; followers **pull** them at read time and merge.
- **Read path** = precomputed feed (push) ⊕ small pull from the handful of celebrities each user follows, then rank.

Because a user typically follows only a **few** mega-accounts, the pull side is small and **cacheable** (a celebrity's recent posts are read by millions → cache once, serve everyone).

Strategy	Write cost	Read cost	Best for	Used by (real world)
Push	High (fan-out)	Very low	Most users; read-heavy systems	Instagram/Twitter baseline
Pull	Very low	High	Celebrities; inactive users	Celebrity timelines
Hybrid ✓	Balanced	Low	Everything at scale	Instagram, Twitter, Tumblr

Extra optimizations layered on the hybrid:

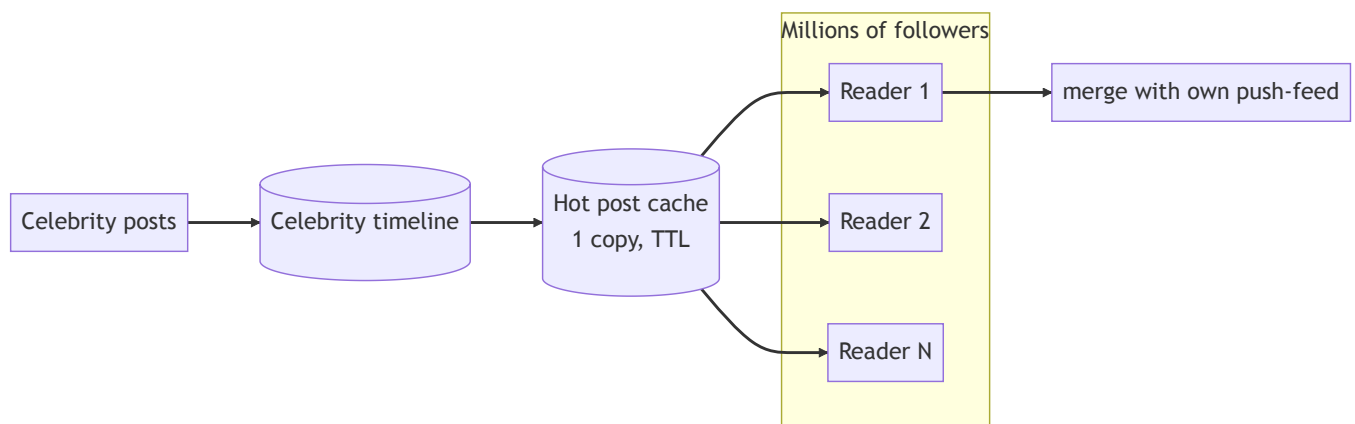
- **Don't fan out to inactive users.** Skip precompute for users who haven't opened the app in N days; lazily build their feed on next login (pull). Saves a large fraction of fan-out writes.
- **Active-follower fan-out.** Prioritize fanning out to recently-active followers first; backfill the rest async.

8. Celebrities / creators with millions of followers

The single hardest part. One post by a 100M-follower account naively = **100M feed writes**, spiking Kafka, workers, and Redis — a self-inflicted DoS.

Mitigations (stack them):

1. **No fan-out for celebrities (pull instead).** Their posts are fetched at read time and merged. This is the primary fix — turns 100M writes into ~0 writes + cached reads.
2. **Cache the celebrity's recent posts hard.** Their latest posts are requested by millions → cache in Redis/CDN with short TTL; **read once, serve all**. Pull cost becomes a single cache hit per reader.
3. **Async, throttled, prioritized fan-out (if any).** For "borderline" accounts, fan out gradually: active followers first, the rest drained from Kafka at a controlled rate so we don't saturate Redis.
4. **Per-key hot-shard handling.** A celebrity's follower list and post cache are hot keys → replicate across many Redis replicas / CDN nodes; split the follower set across shards for parallel fan-out if we *do* push.
5. **Dedupe + merge at read.** Reader's feed = precomputed (push from normal followees) merged with pulled celebrity posts, deduped, then ranked.



Threshold tuning: the push/pull cutoff (e.g., 50K–100K followers) is a knob. Below it, push (cheap fan-out, fast reads). Above it, pull (avoid the write bomb). Some systems also flip an account to pull dynamically when it suddenly goes viral.

One-liner: *"I never fan out a celebrity's post to 100M feeds — I leave it in their timeline and pull it at read time from a single hot cache that millions share."*

9. Media pipeline (upload, transcode, deliver)

- **Direct-to-object-store upload** via pre-signed URLs → app servers never touch raw bytes (saves bandwidth/CPU).
- **Async transcoding** → thumbnail, 720p/1080p/4K, and **HLS/DASH** adaptive bitrate for video. Store all renditions in the object store.
- **CDN in front of everything** → media served from edge POPs near the user; our origin handles only cache misses. This offloads the vast majority of read bytes.
- **Image/video IDs are content-addressable** and immutable → cache forever (`Cache-Control: immutable`); edits create new IDs.
- **Process out of band:** a post can be created with media "processing"; renditions attach when ready.

This keeps the **feed path metadata-only and fast** — media bytes flow client ↔ CDN, never through the feed services.

10. Ranking / relevance (recent and relevant)

A pure reverse-chronological feed is simplest; "relevant" implies **ranking**:

- **Candidate generation:** the merged set (push feed ⊕ pulled celebrity posts).
- **Scoring signals:** recency, affinity (how much you interact with the author), predicted engagement (like/comment/dwell probability), media type, diversity (don't show 5 posts from one account).
- **Two-stage:** cheap heuristic score at read time for the top page; heavier ML model offline/near-line writing a `ranking_score` used as the Redis ZSET score.
- **Keep recency dominant** so the feed feels live; blend relevance on top.

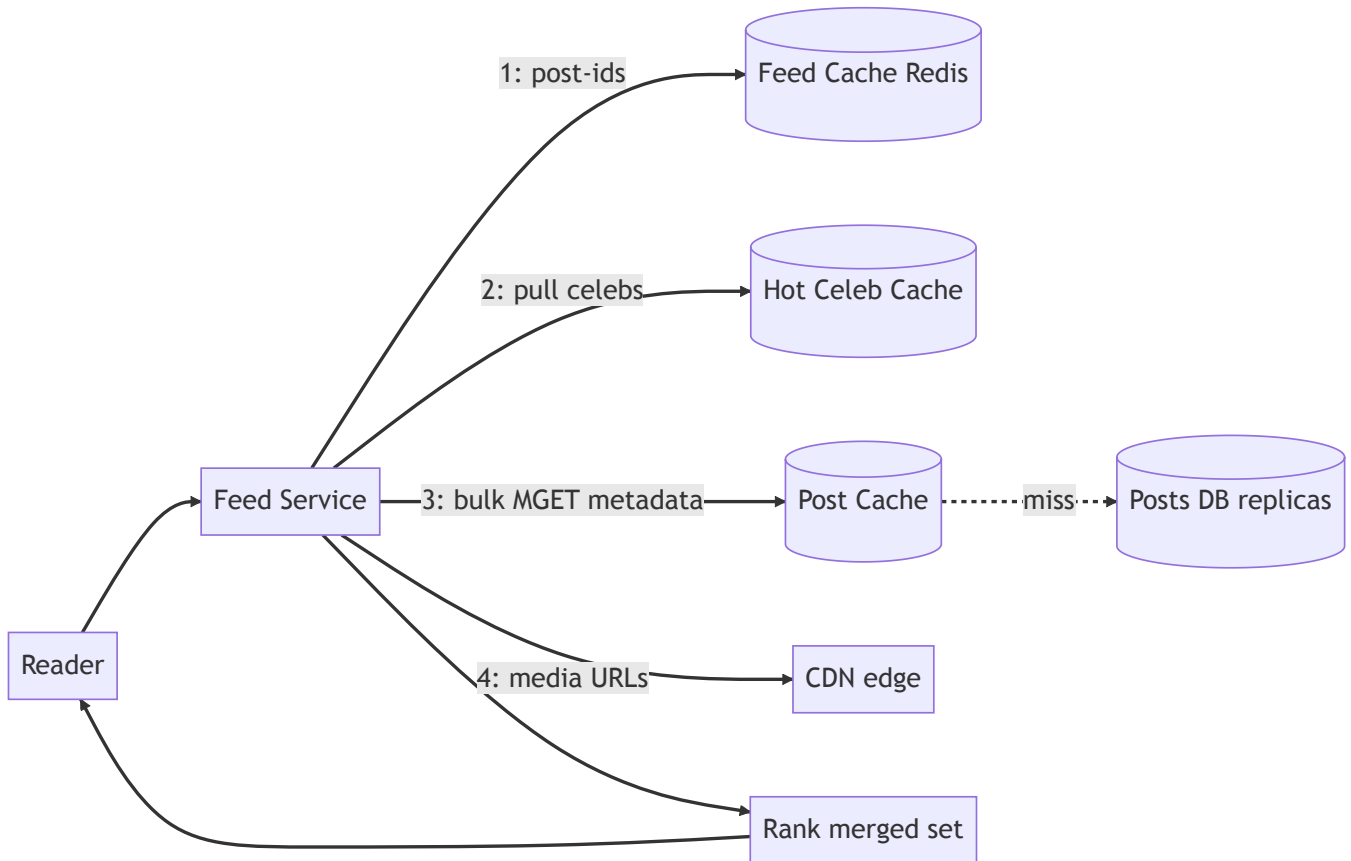
Store the score as the **ZSET score** so `ZREVRANGE` returns ranked order directly. Re-rank the small merged set per request for freshness.

11. Scaling reads & avoiding feed-read bottlenecks

Reads are 100× writes — this section is where the system lives or dies.

1. **Precomputed feeds in RAM (push).** The home feed read is a single `ZREVRANGE` from Redis → sub-ms. This is the #1 defense against read load.
2. **Multi-layer caching:**
 - **CDN** — all media + some static feed fragments.
 - **Feed cache (Redis)** — materialized post-ID lists per user.
 - **Post/object cache (Redis)** — hydrated post metadata by `post_id` (read once, reused across all feeds containing it).
 - **Hot-content cache** — celebrity posts shared by millions.
3. **Read replicas + sharding** for the Posts DB; route reads to replicas, writes to primary. Shard by `user_id/post_id`. Accept **replica lag** (eventual consistency is fine for others' feeds).
4. **Hydrate in bulk.** Feed returns N post-IDs → one **multi-get** (`MGET`) for all post metadata, not N round-trips. Avoids the N+1 query problem.
5. **Cursor pagination** (not `OFFSET`) → $O(1)$ page resumption, stable under inserts.

6. **Trim feeds.** Keep only the newest/best ~500–1,000 IDs per feed (`ZREMRANGEBYRANK`). Bounds memory and read cost; older history comes from the profile/DB on demand.
7. **Avoid hot keys.** Replicate hot Redis keys across replicas; for a celebrity follower-list/post, fan reads across many replicas or the CDN.
8. **Backpressure & async.** Fan-out via Kafka absorbs write spikes so reads aren't starved; consumers scale independently.
9. **Prefetch / pagination on client** — load page 1 fast, prefetch page 2 during scroll.



12. Failure scenarios — "what if X fails?"

Failure	Impact	Mitigation
Feed cache (Redis) node down	Some users' feeds missing	Replicas + failover; on miss, rebuild from DB (pull) — degraded but available
Fan-out worker backlog	New posts slow to appear in feeds	Kafka buffers; autoscale consumers; prioritize active followers; eventual consistency is acceptable
Celebrity post spikes Kafka	Fan-out lag for everyone	Celebrities are pull-only (no fan-out); throttle/borderline-account draining
Object store / CDN miss storm	Slow media	Multi-tier CDN, origin shielding, pre-warm popular assets
Posts DB primary down	Writes blocked	Failover to replica; writes retried via Kafka; reads served from replicas/cache
Hot key (celebrity)	One shard saturated	Replicate hot keys, CDN the post, split follower set across shards
Replica lag	Slightly stale feed	Acceptable for others' posts; route the author's own reads to primary/cache for read-after-write

Guiding principle: the feed must **always render**. Prefer **stale-but-available** over unavailable — fall back from cache → replica → pull-rebuild rather than erroring.

13. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Push vs Pull	Fan-out-on-write (fast reads, costly writes)	Fan-out-on-read (cheap writes, costly reads)	Hybrid: push for the long tail, pull for celebrities
Consistency vs Latency	Strong (read-after-write everywhere)	Eventual (feeds lag a few s)	Eventual for <i>others'</i> feeds; strong only for author's own view
Precompute vs On-demand	Materialize all feeds	Build at read time	Precompute active users; skip inactive users (lazy build)
Storage cost vs Read speed	Store feeds per user (RAM)	Recompute each read	Trade RAM for speed — it's a read-heavy system; trim to bound cost
Freshness vs Cost	Re-rank every read	Cache ranked feed	Cache base, re-rank only the small merged top page
Blob in DB vs Object store	DB (simple)	Object store + CDN	Always object store + CDN for media

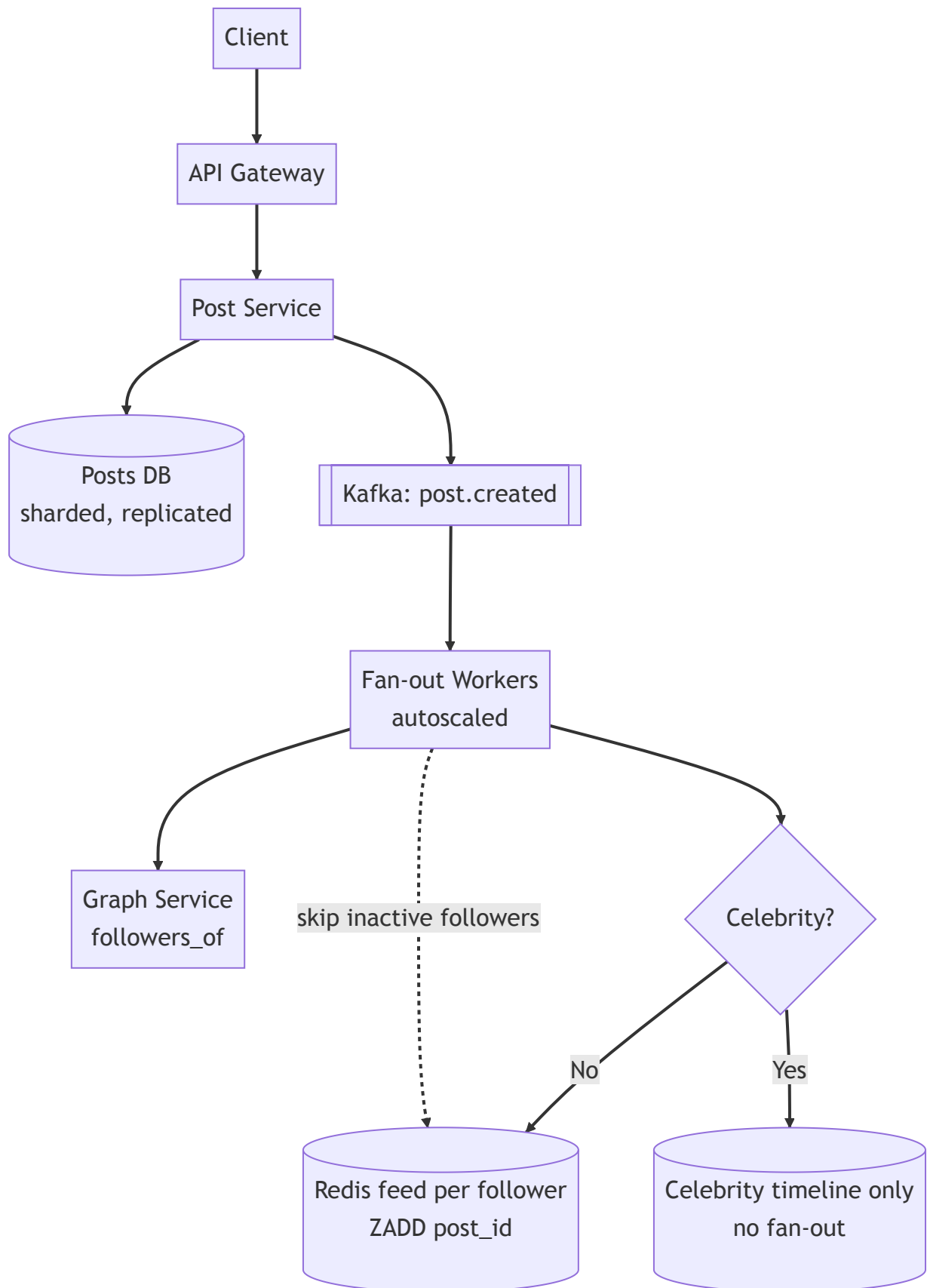
CAP framing: the feed system leans **AP** — under partition we keep serving (possibly stale) feeds rather than blocking. Strong consistency is reserved for the narrow case of the **author seeing their own post** immediately.

One-liner to say out loud: "I'd use a **hybrid fan-out**: push posts from normal accounts into followers' precomputed Redis feeds for sub-ms reads, but **never** fan out celebrities — their posts stay in their timeline and are pulled from a shared hot cache at read time. Media goes to object storage behind a CDN, post metadata to a sharded wide-column store, the follow graph denormalized both directions, and Kafka decouples the write path from fan-out so read traffic is never starved. I skip precomputing feeds for inactive users and trim feeds to the newest ~1,000 to bound memory."

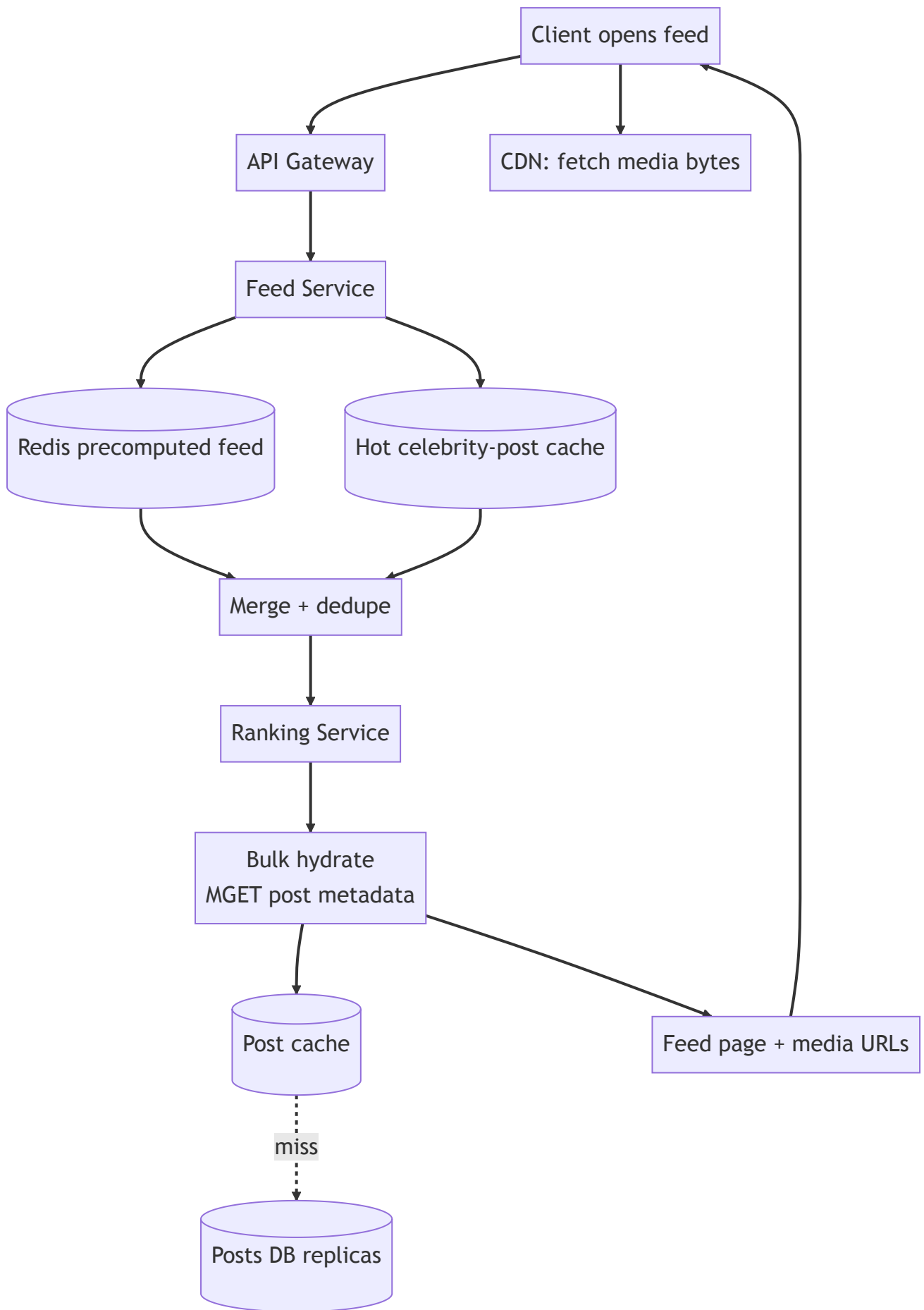
14. Full system design (detailed)

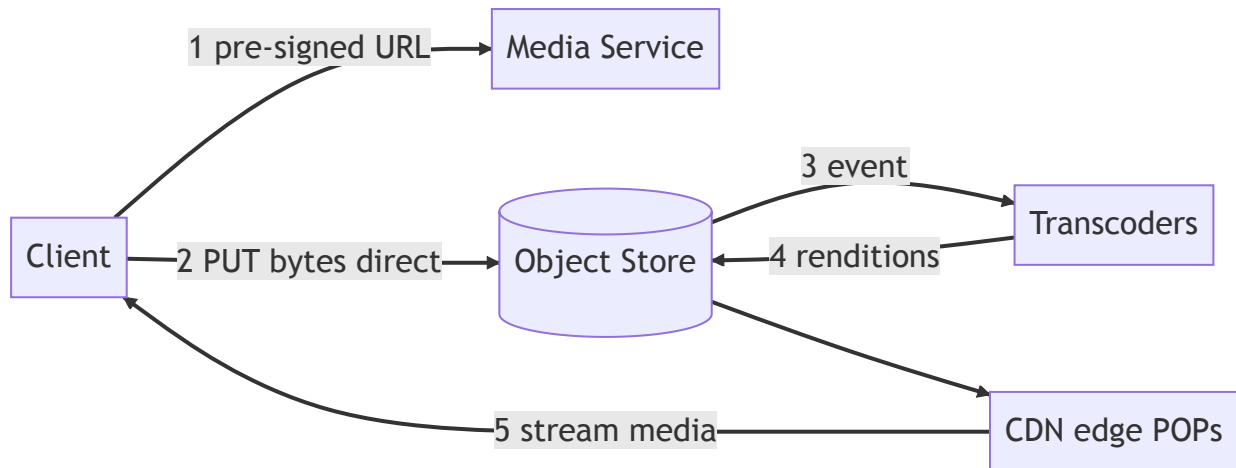
End-to-end, split into **(A)** write path (post creation + fan-out), **(B)** read path (feed delivery), and **(C)** media path.

14A. Write path — post creation & fan-out



14B. Read path — feed delivery





15. Networking, security & performance best practices

Networking

- **CDN for all media** — photos/videos served from edge POPs; origin hit only on cache miss. **Adaptive bitrate (HLS/DASH)** for video, modern codecs (H.265/AV1).
- **TLS at the edge/LB**; **HTTP/3 (QUIC)** for multiplexed, low-latency feed + media fetches on flaky mobile networks; **gRPC** between internal services.
- **Direct-to-object-store uploads** via pre-signed URLs — raw media bypasses app servers (saves bandwidth/CPU).
- **Cursor pagination** on the wire (compact, stable under inserts) — never `OFFSET`.
- **Connection pooling / keep-alive** between services; co-locate feed cache with feed services.

Security

- **AuthZ on every read** — enforce private accounts, blocks, and channel membership before returning posts; never trust the client.
- **Signed, expiring media URLs** so private media isn't hotlinkable; per-object access checks.
- **Sanitize uploads in a sandbox** — strip EXIF/GPS, re-encode to kill polyglot/malware files, virus-scan before publish.
- **Rate-limit writes** (post/like/follow) and run **abuse/bot detection** on the fan-out path to blunt spam.
- **Avoid enumeration leaks** (follower lists, user existence); defend against scraping.

Performance

- **Precomputed feeds + multi-layer cache** (CDN → feed cache → post cache) → sub-ms reads.
- **Bulk hydrate** (`MGET`) to kill $N+1$; store only post-IDs in the feed.
- **Trim feeds** to the newest ~1,000; **skip inactive** users.
- **Thumbnails first**, lazy-load full-res; **client prefetch** of the next page during scroll.

16. Staying current — modern & emerging approaches

- **Stores**: Cassandra/**ScyllaDB** for posts, **Redis** for feeds, **Kafka** for fan-out — the industry-standard stack at Meta/Twitter scale; Facebook's **TAO** for the social graph.
- **Media**: HLS/DASH ABR, **AV1/HEVC** codecs, per-title encoding; CDNs (CloudFront/Fastly/Akamai); on-the-fly image resizing at the edge.
- **Transport**: **HTTP/3/QUIC** for mobile; gRPC internally.

- **Ranking: two-tower retrieval** + GBDT/DNN ranking, **feature stores** (Feast), near-line feature computation; embeddings for relevance.
 - **Reference systems:** Instagram feed/Stories architecture, Twitter Timelines + Manhattan store, LinkedIn feed.
 - **How I stay current:** engineering blogs (Meta, Discord, Twitter/X, Pinterest), papers (TAO, f4), and prototyping + load-testing before adopting.
-

17. Likely follow-up questions (rehearse these)

- Push vs pull fan-out — when each? (*read-heavy* → *push*; *celeb write-bomb* → *pull*)
 - How do you keep a celebrity's post out of 100M feeds? (*no fan-out*; *pull from one shared hot cache*)
 - How does the author see their own post instantly? (*read-after-write to primary/cache for self*)
 - Feed cache (Redis) dies — what now? (*rebuild from DB via pull*; *degrade, don't fail*)
 - "Relevant" ranking without killing latency? (*precompute score as the ZSET score*; *re-rank only the small merged top page*)
 - A normal post suddenly goes viral — how? (*dynamically flip the author to pull*)
 - Storing a 200B-edge follow graph? (*denormalize both directions for $O(1)$ reads*)
 - Hot key on a celebrity's follower list? (*replicate across replicas, shard, CDN the post*)
-

18. Summary checklist (whiteboard recap)

- **Read-heavy (100:1)** → precompute + cache feeds; single ZREVRANGE read.
- **Hybrid fan-out** → push normal accounts, pull celebrities; merge + rank at read.
- **Celebrities** → no fan-out, shared hot cache, pull at read (kills the write bomb).
- **Storage** → posts in wide-column, media in object store + CDN, graph denormalized both ways, feeds in Redis ZSETs.
- **Kafka** decouples write from fan-out; absorbs spikes; lets consumers scale.
- **Skip inactive users, trim feeds, bulk-hydrate, cursor paginate** → bound cost and latency.
- **Eventual consistency** everywhere except the **author's own** read-after-write.
- **Always render** the feed — degrade (cache → replica → pull) rather than fail.

Designing an In-Memory Key–Value Database (Redis-like)

A complete, interview-ready walkthrough: requirements → estimates → APIs → in-memory structures → on-disk schemas → **concurrency, transactions, TTL, eviction** → **durability & crash recovery** → **replication & sharding** → **consistency** → **backpressure** → failure modes → trade-offs. Use the headings as your whiteboard agenda. The hard parts are **the concurrency model, durability vs latency, and consistency under replication/sharding** — spend your time there.

Think Redis / Memcached / Aerospike. RAM-resident data for microsecond reads/writes, $O(1)$ average ops, optional persistence (AOF + snapshots), and horizontal scale via sharding + replication.

0. How to drive the interview (talk track)

1. **Clarify** functional + non-functional requirements and the access pattern.
2. **Estimate** scale (QPS, keys, value sizes, RAM, network).
3. **Define the client API / wire contract**.
4. **Pick in-memory structures** (hash table + expiry index + eviction metadata) → $O(1)$.
5. **Choose the concurrency model** (single-threaded event loop vs locks/sharded threads).
6. **Add durability** (AOF + snapshots) and the **crash-recovery** flow — and its latency trade-off.
7. **Scale out**: sharding + replication; state the **consistency** model.
8. **Handle overload** (backpressure) and **failure modes**; summarize **trade-offs**.

Keep saying *"here's the trade-off..."* — that's what's being graded.

1. Problem & motivation

A KV store that keeps data in **RAM** for high-throughput, low-latency get/set/delete with **$O(1)$** average cost, optional **persistence, TTL, CAS**, atomic multi-key ops within a shard, and horizontal scale.

Why in-memory: RAM access is ~~100 ns vs 100 μ s for SSD vs 10 ms for disk seek~~ ****1000x faster****. For caches, sessions, counters, rate-limit buckets, leaderboards, and queues, microsecond latency at high QPS is the whole point.

What makes it hard:

- **$O(1)$ at p99**, not just average — avoid rehash stalls, GC pauses, lock contention, big-key blowups.
 - **Durability vs latency** — every write to disk is safe but slow; the AOF fsync policy is the central trade-off.
 - **Concurrency** — a hash table mutated by millions of ops/sec; single-thread simplicity vs multi-core throughput.
 - **Memory is finite** — eviction policy + expiry must reclaim space without stalling.
 - **Scale-out consistency** — replication lag and sharding break the "one box, one truth" simplicity.
 - **Crash recovery** — rebuild state fast and correctly from snapshot + log.
-

2. Requirements

Functional

- **Core ops:** `set(key, value[, ttl]), get(key), delete(key)`.
- **CAS** — conditional update by **version** or **expected value** (optimistic concurrency).
- **TTL expiration** — keys auto-expire.
- **Atomic multi-key operations within a shard** (mini-transactions / `MULTI...EXEC`).

Non-functional

- **O(1)** average reads/writes; tight p99.
- **Expiration tracking** (timing wheel or min-heap).
- **Memory management + eviction** (LRU/LFU).
- **Durability** — append-only log (AOF) + periodic **snapshots**.
- **Concurrency control** — single-threaded event loop *or* fine-grained locks/sharded threads.
- **Crash recovery** procedure.
- **Replication + sharding** for scale.
- **Consistency** guarantees (stated explicitly).
- **Backpressure** under overload.
- **Observability** — logs, metrics, tracing.

Clarifying questions to ask the interviewer

- **Durability bar** — can we lose the last ~1 s of writes on crash (cache), or must every ack be durable (system of record)? (*Sets the `fsync` policy.*)
- **Consistency** — strong (sync replication) or eventual (async, like Redis)? Read-your-writes?
- **Value size** — small (counters, sessions) or large blobs? Affects memory + eviction.
- **Eviction** — is it a **cache** (evict under pressure) or a **store** (reject writes when full)?
- **Multi-key transactions** — within a shard only, or cross-shard (distributed txn)?
- **Single region or geo-distributed?**

3. Back-of-the-envelope estimation

Quantity	Assumption	Result
Target QPS	high-throughput cache	~1M ops/sec per node (Redis does ~100k–1M+ on one core)
Keys/node	—	~100M keys
Avg entry	~64 B key + ~256 B value + ~80 B overhead	400 B → 100M × 400 B → **40 GB** data
RAM/node	data + replication buffers + fragmentation (~1.5×)	~64 GB box
Value sizes	small-object dominated	keep big values rare; chunk/externalize blobs
Network	1M ops × ~400 B	400 MB/s → **3.2 Gbps** → 10/25 GbE NIC
AOF write rate	~300k writes/sec × ~50 B record	~15 MB/s → trivial sequential SSD; fsync cadence is the real cost
Cluster	1 TB dataset / 40 GB per shard	~25 shards (× replicas)

Takeaways that drive the design:

1. **~1M ops/sec/core** → a **single-threaded event loop** per shard avoids lock overhead; scale by **adding shards/cores**, not threads on one map.
2. **RAM is the budget** → eviction + expiry must be cheap and incremental (no stop-the-world).
3. **fsync is the latency knob** → batch/group-commit; choose per-write vs per-second durability.
4. **1 TB > one box** → **shard**; replicate each shard for HA.

4. Client API (wire contract)

Simple request/response over TCP (a compact binary protocol like RESP; pipelining for throughput).

```

SET key value [EX ttl_seconds] [NX|XX]      → OK | (nil)
GET key                                     → value | (nil)
DEL key                                     → 1 | 0
TTL key                                     → seconds | -1 (no ttl) | -2 (missing)
EXPIRE key ttl                             → 1 | 0

# CAS — optimistic concurrency
CAS key expected_value new_value           → OK | CAS_MISMATCH
CASV key expected_version new_value        → OK,new_version | CAS_MISMATCH # version-based
INCR key                                   → new_int      # atomic counter

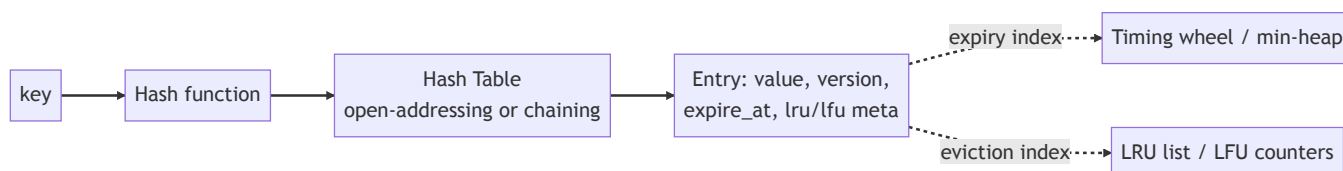
# Atomic multi-key (within a shard)
MULTI                                     # begin
  SET a 1
  INCR b
EXEC                                     → [results] | ABORT # all-or-nothing
# optimistic variant:
WATCH k1 k2 ; MULTI ; ... ; EXEC         → nil if a watched key changed

```

Design notes:

- **CAS** returns a typed mismatch (not a generic error) so clients can retry the read-modify-write loop.
- **Versioned CAS** — each key carries a monotonically increasing **version**; CASV succeeds only if the version matches → lock-free optimistic updates, ABA-safe.
- **Pipelining** — clients batch many commands per round-trip (the main throughput lever at 1M QPS).
- **Idempotency** — SET/DEL are naturally idempotent; client request IDs let proxies dedup retries.

5. In-memory data structures & algorithms (O(1))



Primary index — hash table (O(1) avg)

- A **hash table** maps key → entry; average O(1) get/set/delete. Use a good hash (xxHash/SipHash — SipHash resists hash-flooding DoS).

- **Incremental rehash** (Redis-style two-table) — on growth, migrate buckets a few at a time on each op so resizing never stalls the world (protects p99). Load factor target ~0.7.
- **Entry** holds: `value`, **version** (for CAS), **expire_at** (0 = none), and **eviction metadata** (LRU timestamp / LFU counter).

Expiration tracking

Approach	How	Trade-off
Lazy expiry	Check <code>expire_at</code> on access; if past, delete + return nil	Free, but dead keys linger in RAM until touched
Min-heap by <code>expire_at</code>	Pop due keys	$O(\log n)$ per op; exact next-expiry
Timing wheel 	Buckets by time slot; advance a hand each tick	$O(1)$ insert/expiry; great for many TTLs
Active sampling (Redis)	Periodically sample N keys with TTL, evict expired	Probabilistic, bounded CPU

Recommended: lazy expiry + active background sweep (timing wheel or sampled scan). Lazy handles the hot path for free; the background sweep reclaims cold expired keys so they don't waste RAM — bounded per tick to avoid spikes.

Eviction policy (when memory is full)

- **LRU** (approximate, Redis-style sampling) — evict least-recently-used; cheap, good default. Exact LRU = an intrusive doubly-linked list (move-to-head on access).
- **LFU** — least-frequently-used (with decay) for skewed/hot-key workloads.
- **TTL-aware** (`volatile-lru`) or **random** as policies; **noeviction** = reject writes when full (store, not cache).
- Maintain eviction metadata **inline** in the entry → $O(1)$ update on access.

Values

- Small values inline; **large values chunked**/externalized to avoid copy/allocation spikes and big-key stalls. Specialized structures (lists, sets, sorted sets) as needed, each $O(1)/O(\log n)$.

6. On-disk schemas — AOF & snapshot (durability formats)

Persistence is **optional** but, when on, two complementary mechanisms:

Append-Only File (AOF) — the write log

Every mutating command is appended as a record (the operation, not the whole dataset).

AOF record (binary, length-prefixed):

magic	rec_len	lsn	op (1B)	key_len	key ...	val ...
		(u64)	SET/DEL/			+ttl
			EXPIRE/			+ver
			CAS			

+ CRC32 per record (detect torn/corrupt tail)

- **Append-only, sequential** → fast on SSD; the **LSN** (log sequence number) orders writes and drives replication + recovery.
- **fsync policy** (the durability knob): `always` (every write — safest, slowest), `everysec` (group-commit each second — Redis default, lose ≤1 s), `no` (OS flushes — fastest, least safe).
- **AOF rewrite/compaction** — periodically rewrite the log to the minimal set of commands reproducing current state (drop overwritten/expired keys) so it doesn't grow unbounded.

Snapshot (point-in-time dump)

A compact serialization of the whole keyspace at an instant (Redis RDB-style).

```
Snapshot file:
[ header: magic, version, created_at, last_lsn ]
[ repeated: key_len, key, type, value, expire_at, version ]
[ footer: count, CRC ]
```

- Produced by **fork + copy-on-write** (snapshot the child's frozen memory while the parent keeps serving) or a background thread → no long stall.
- `last_lsn` records where the snapshot ends so recovery knows which AOF tail to replay.

AOF + snapshot together: snapshot = fast bulk restore; AOF = the durable tail since the snapshot. Best of both: quick recovery + minimal data loss.

7. Concurrency, transactions & atomic multi-key (focus area)

The model: single-threaded event loop per shard

- **One thread owns a shard's hash table**, processing commands from an **event loop** (epoll/kqueue) over many connections.
- **No locks, no races, no deadlocks** on the data — each command runs to completion atomically. This is why Redis is fast *and* simple: removing lock contention often beats multi-threading for in-RAM ops.
- **Scale across cores** by running **many shards (one event-loop thread each)**, partitioning the keyspace → linear scale without shared-memory locking. I/O (network read/parse, TLS) can use helper threads; the **data mutation stays single-threaded per shard**.

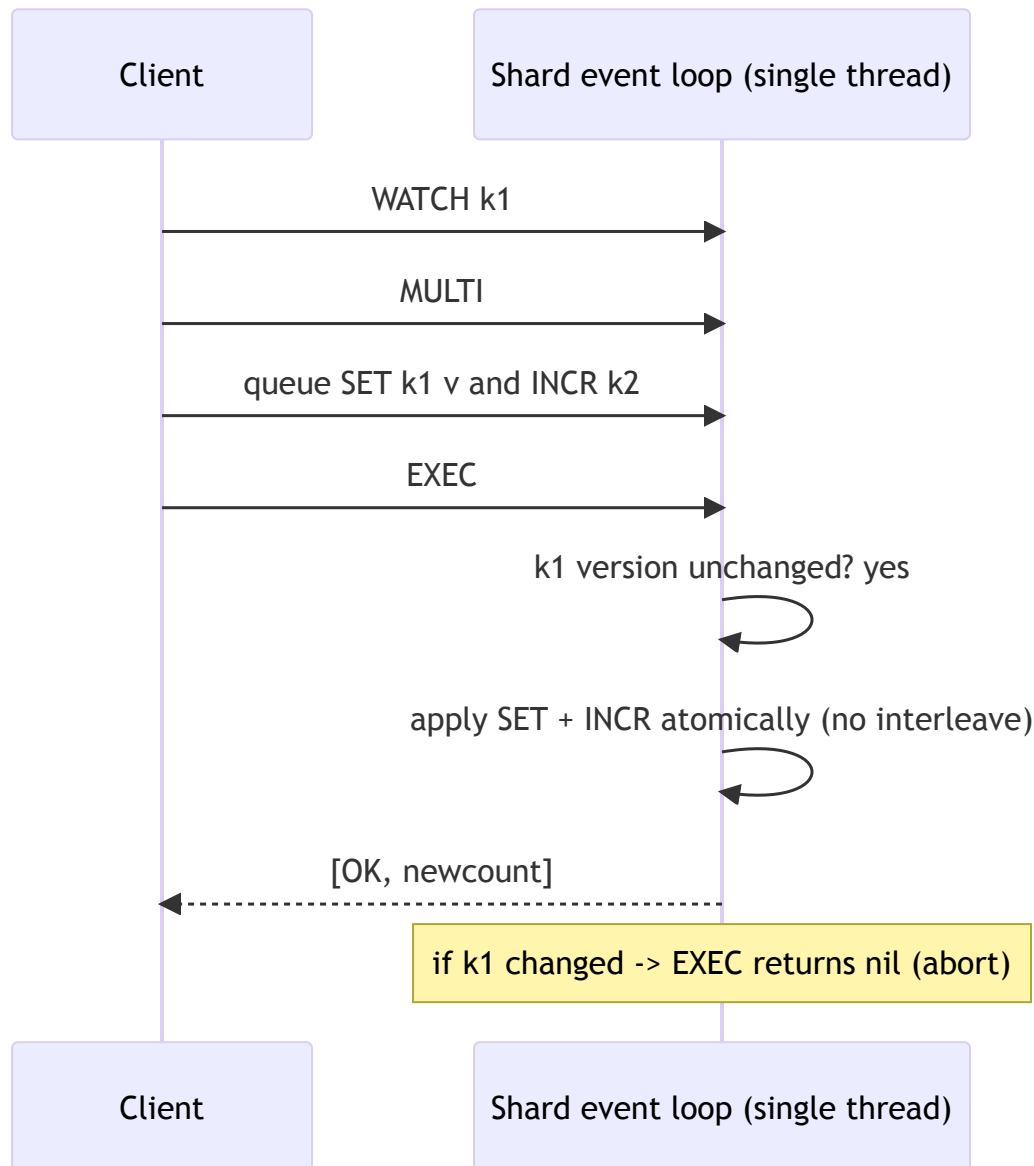
Model	Pros	Cons
Single-threaded event loop 	No locks, simple, atomic ops free, great p99	One core per shard → scale via sharding
Multi-threaded + fine-grained locks	Uses all cores on one keyspace	Lock contention, deadlocks, complex; cache-line bouncing
Sharded threads (lock per shard)	Parallel, bounded contention	Cross-shard ops need coordination

Recommendation: single-threaded per shard, scale by sharding. It makes **atomicity, CAS, and multi-key EXEC** trivially **correct** (they just run start-to-finish on the owning thread).

Atomic multi-key (within a shard)

Because one thread owns the shard, a `MULTI...EXEC` block **queues commands then executes them as one uninterrupted unit** → all-or-nothing, no interleaving.

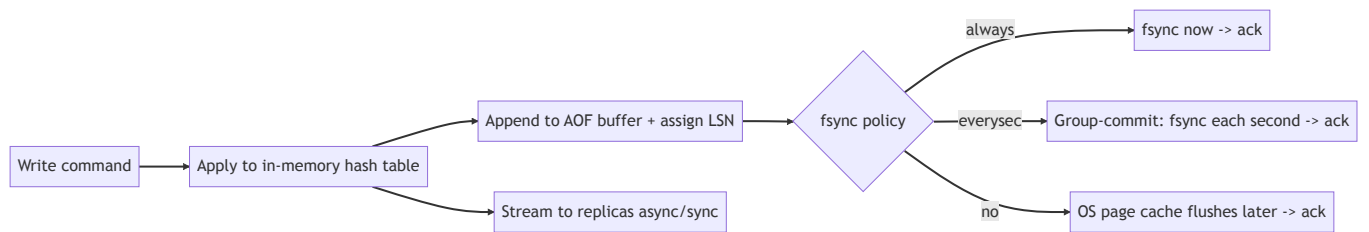
- **Optimistic WATCH** — watch keys, build the transaction, EXEC aborts if any watched key changed since WATCH (compare versions) → lock-free optimistic concurrency for read-modify-write across keys.
- **CAS** is the single-key special case: succeed iff version/value matches, then bump version — all on the owning thread, so no lost updates.
- **Constraint:** atomic multi-key requires the keys be **on the same shard**. Use **hash tags** ({user1}:profile, {user1}:cart) so related keys co-locate → multi-key txns stay shard-local.



Cross-shard transactions (if required) need **2-phase commit / Raft-replicated txn coordinator** — expensive; call out the trade-off and prefer keeping a transaction within one shard via hash tags.

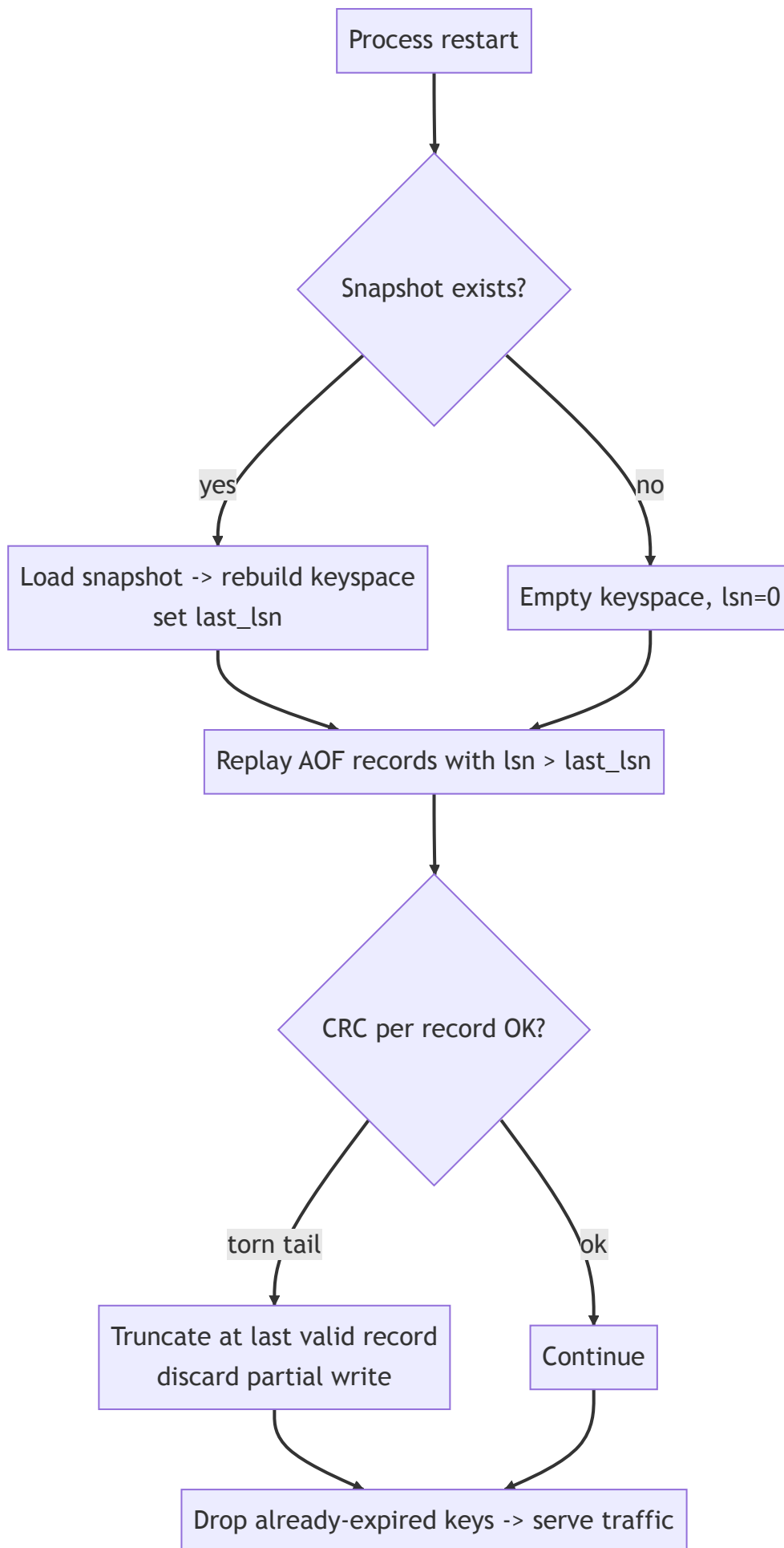
8. Durability & crash recovery (focus area)

Write path with durability



- **Group commit** — batch many writes into one fsync (amortize the ~ms fsync over thousands of ops) → durability with high throughput.
- **Order**: apply in memory → append to AOF → (per policy) fsync → ack. With `always`, ack only after fsync (durable); with `everysec`, ack early (fast, ≤ 1 s loss window).

Crash-recovery flow

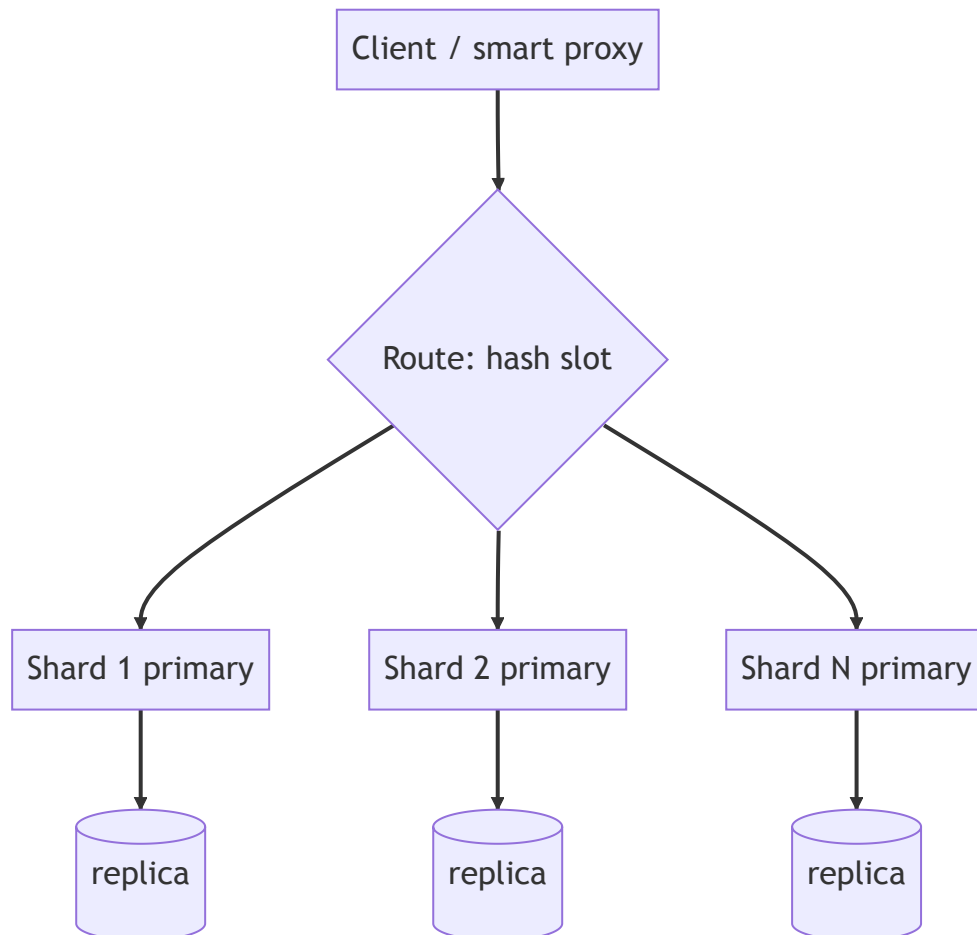


1. **Load the latest snapshot** (fast bulk restore) → `keyspace + last_lsn`.
 2. **Replay the AOF tail** (records with `lsn > last_lsn`) to reapply writes since the snapshot.
 3. **CRC-check each record**; a crash mid-write leaves a **torn tail** → truncate at the last valid record (that write was never acked under `always`, or is within the ≤ 1 s loss window under `everysec`).
 4. **Drop expired keys** (TTL passed during downtime), then accept traffic.
- **Recovery time** $\propto$ snapshot size + AOF tail length → frequent snapshots keep the tail (and recovery) short.

9. Replication & sharding (focus area)

Sharding (horizontal scale, partition the keyspace)

- **Hash partitioning** — `shard = hash(key) mod N`, or better **consistent hashing** / **hash slots** (Redis Cluster = 16,384 slots) so adding/removing nodes moves a **minimal** fraction of keys.
- Each shard is an independent single-threaded keyspace → cores/nodes scale linearly.
- **Hash tags** keep related keys on one shard for atomic multi-key ops.
- **Resharding** — move slots between nodes with live migration (copy + delta + cutover); clients follow **MOVED/ASK** redirects or use a routing proxy.



Replication (HA per shard)

- **Primary–replica per shard.** Primary streams its write log (LSN-ordered) to replicas; replicas apply in order.
- **Async (default, Redis-like)** — ack the client immediately, replicate in background → **low latency**, but a primary crash can lose the un-replicated tail → **eventual consistency**.
- **Semi-sync / sync** — ack only after ≥ 1 replica acks → **stronger durability**, higher latency. Tunable (`WAIT n`).

- **Failover** — a **sentinel/cluster manager** (or Raft) health-checks primaries; on failure, **promote a replica** and repoint clients. Replicas also **serve reads** (read scaling) at the cost of staleness.
- **Quorum option** — for strong consistency, replicate via **Raft/Paxos** (majority ack) — used by strongly-consistent KV stores (etcd, TiKV); higher latency, no lost acked writes.

10. Consistency guarantees & trade-offs (focus area)

- **Within a shard (single thread): linearizable** — ops are totally ordered and atomic; CAS/EXEC are correct. This is the strong island.
- **Across replicas (async): eventual consistency** — a replica may lag; reads from replicas can be stale; a failover can lose the un-replicated tail. Default for cache-like deployments (favor latency/availability).
- **Across shards: no global transaction/order** by default — each shard is its own consistency domain. Cross-shard atomicity needs 2PC/Raft (opt-in, costly).
- **Tunable knobs:**
 - **Read-your-writes** → read from primary (or wait for replica LSN $\geq$ write LSN).
 - **Stronger durability** → `wait`/semi-sync or Raft quorum.
 - **Lower latency** → async replication + `everysec` fsync.

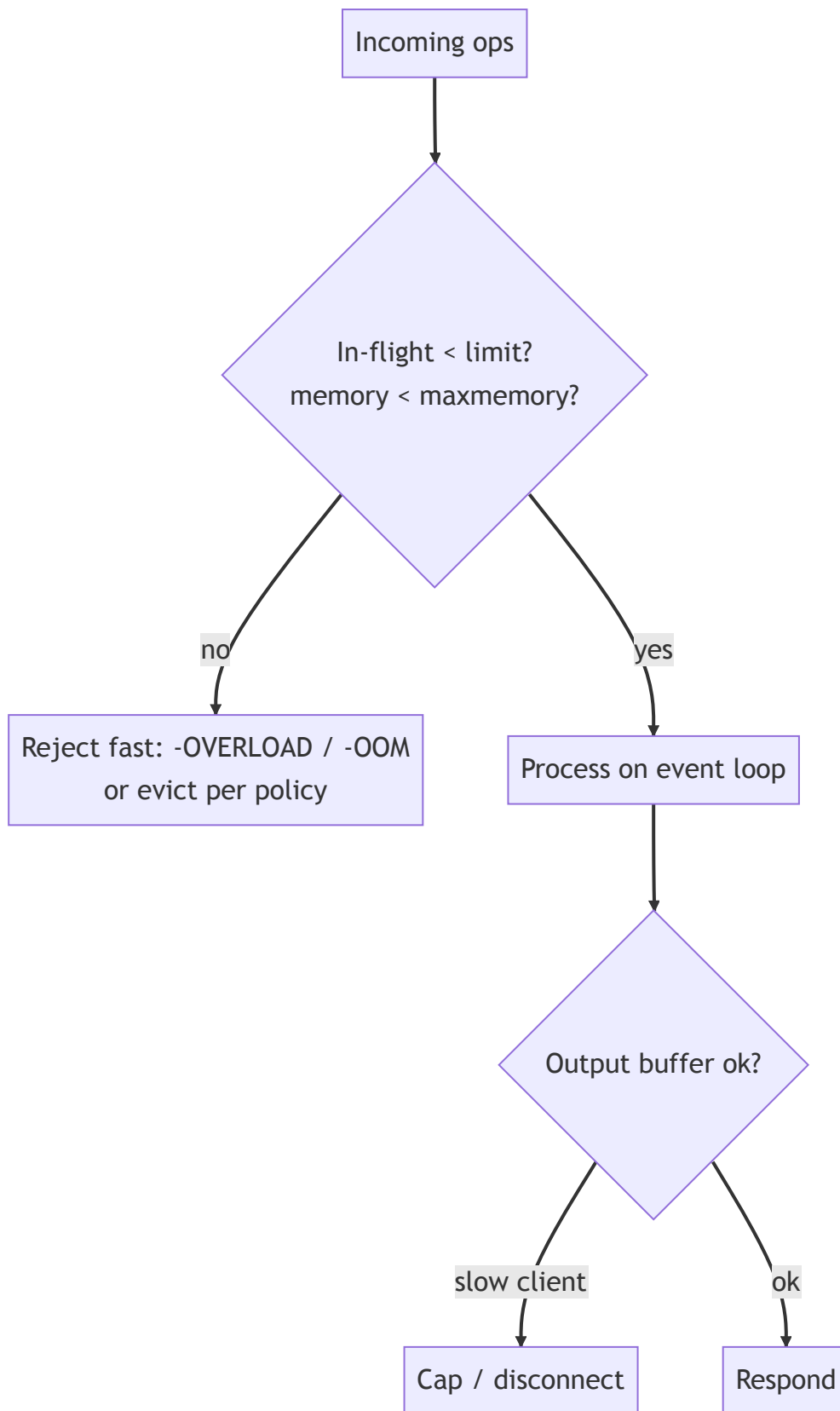
CAP/PACELC framing: a single shard is **CP** (linearizable, but unavailable during failover). The replicated system is typically run **AP/eventual** (async replication: stay available + fast, tolerate staleness) — or **CP** if you choose quorum replication. **PACELC:** *else* (no partition) you still trade **Latency vs Consistency** via the fsync + replication-ack settings.

Want	Setting	Cost
Lowest latency	async replication, <code>fsync no/everysec</code>	may lose recent writes; stale replica reads
No lost acked writes	sync/quorum replication, <code>fsync always</code>	higher write latency
Read-your-writes	read from primary or wait LSN	less read scaling
Strong cross-shard txn	2PC / Raft coordinator	much higher latency/complexity

11. Backpressure under overload (focus area)

At 1M QPS, overload **will** happen — degrade gracefully, never melt down.

- **Bounded queues + admission control** — cap the per-connection command pipeline and global in-flight; when full, **reject fast** (`busy/-OOM/-OVERLOAD`) instead of unbounded buffering.
- **Memory limit + eviction/noeviction** — at `maxmemory`, either evict (cache) or **reject writes** (store) with a clear error; never OOM-kill the process.
- **Client output-buffer limits** — slow consumers (e.g., replicas, pub/sub) get capped buffers; disconnect abusers rather than ballooning RAM.
- **Connection/request rate limits + max-clients** cap; **slowlog** to catch expensive commands (big `KEYS`, large-value ops).
- **Flow control to replicas** — if a replica lags, throttle or signal the primary; don't let replication backlog exhaust memory.
- **Load shedding** — under sustained overload, shed low-priority traffic (e.g., reads) to protect writes/replication; **circuit-break** at the client/proxy.
- **Avoid p99 stalls** — incremental rehash, incremental expiry, fork-based (CoW) snapshots, no synchronous big-key deletes (lazy/async free) so one fat operation can't block the loop.



12. Observability (a stated requirement)

- **Metrics** — ops/sec by type, **p50/p99/p999 latency**, hit/miss ratio, **memory used vs maxmemory + fragmentation**, evictions/sec, expired keys/sec, keyspace size, **replication lag (LSN delta)**, AOF fsync time, connected clients, rejected/over-limit ops → TSDB + SLOs/alerts.

- **Logs** — slowlog (commands over a threshold), rewrites/snapshots, failovers, evictions, errors; structured.
- **Tracing** — propagate a request/trace id through proxy → shard → replica for cross-node latency attribution; sample to bound overhead.
- **Health/introspection** — INFO-style stats, latency histograms, hot-key/big-key detection, per-command stats.
- **Golden signals** — p99 latency, hit ratio, memory headroom, replication lag.

13. Failure modes — "what if X fails?"

Failure	Impact	Mitigation
Process crash	In-RAM data gone	Recover from snapshot + AOF replay ; replica takes over meanwhile
Primary node dies	Shard write-unavailable	Failover : promote replica, repoint clients; async tail may be lost (eventual)
fsync/disk slow or full	Write stalls / AOF can't flush	Group commit; alert on fsync latency; switch to <code>everysec</code> ; cap AOF, alert on disk
Torn AOF write (crash mid-append)	Corrupt tail	CRC per record → truncate at last valid record on recovery
Memory pressure / maxmemory	Risk of OOM	Eviction policy or reject writes; bounded buffers; never OOM the process
Hot key	One shard/core saturated	Client-side cache, key splitting, replica reads, LFU
Big key / big op	Event-loop stall (p99 spike)	Chunk large values; lazy/async free ; incremental ops
Replica lag	Stale reads, weak durability	Monitor LSN delta; read-from-primary for RYW; throttle on backlog
Network partition	Split-brain risk	Quorum/sentinel decides primary; minority steps down (CP)
Hash-flood DoS	Degraded to O(n) buckets	SipHash seeded per-process; key/size limits
Resharding	Migration errors	Live slot migration + MOVED/ASK redirects; idempotent moves

Guiding principle: a single shard is a **strongly-consistent, recoverable island**; the cluster trades some consistency for availability and latency, and **degrades (evict/shed/redirect) rather than crashes**.

14. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Concurrency	Single-thread event loop (simple, no locks)	Multi-thread + locks (all cores)	Single-thread per shard , scale by sharding ✅
Durability	fsync always (safe)	everysec/no (fast)	everysec group-commit default; always for system-of-record
Persistence	AOF (small loss window, slower restart)	Snapshot (fast restore, more loss)	Both — snapshot for bulk + AOF tail for recency
Replication	Async (low latency, may lose tail)	Sync/quorum (no loss, slower)	Async for cache; quorum for strong durability
Consistency	Linearizable (CP, quorum)	Eventual (AP, async)	Per shard CP; cluster usually AP — tunable
Eviction	Evict (cache)	noeviction/reject (store)	Match to whether it's a cache or source of truth
Expiry	Lazy (free, lingers)	Active sweep/wheel (reclaims, CPU)	Both — lazy + bounded background sweep
Memory vs accuracy	Exact LRU (linked list)	Approx LRU (sampling)	Approximate — near-exact, far cheaper

One-liner to say out loud: *"I'd build it as **single-threaded event-loop shards** over a **hash table with incremental rehash** for O(1) ops with tight p99, **lazy + background TTL expiry**, and **approximate-LRU eviction**. Durability is **AOF with group-commit fsync (everysec by default, always for systems of record) plus fork-CoW snapshots**, recovered as *snapshot* → *replay AOF tail* → *truncate torn record*. Scale is **hash-slot sharding** (hash tags keep multi-key txns shard-local, where the single thread makes MULTI/EXEC and CAS trivially atomic) with **primary–replica replication per shard — async for latency, quorum for strong durability**. A single shard is **linearizable (CP)**; the cluster is **tunably AP/eventual** via the fsync + replication-ack knobs, and it **sheds load (bounded queues, eviction, output caps) rather than melting** under overload."*

15. Networking, security & performance best practices

Networking

- **Compact binary protocol** (RESP-like) over TCP; **pipelining** to amortize round-trips at high QPS; optional **RDMA/kernel-bypass (DPDK)** for extreme throughput.
- **TLS** with session resumption; co-locate clients/proxy/nodes in one AZ to keep RTT μ s-low.
- **Smart client / proxy routing** (slot-aware) to hit the owning shard directly; connection pooling + keep-alive.

Security

- **AuthN** (ACL users/passwords, mTLS); **per-key/command ACLs**; disable dangerous commands (`FLUSHALL`, `KEYS`) in prod.
- **SipHash-seeded** hashing to stop **hash-flooding** DoS; key/value **size limits**.
- **Network isolation** (private subnet, no public exposure — the classic unauthenticated-Redis breach); encrypt snapshots/AOF at rest.

Performance

- **O(1) structures**, incremental rehash/expiry, **fork-CoW** snapshots, **lazy/async free** of big keys → no event-loop stalls.
- **Group-commit fsync**; **approximate LRU/LFU**; per-core sharding for linear scale.

- **NUMA-aware** pinning; jemalloc/tcmalloc to curb fragmentation.
-

16. Staying current — modern & emerging approaches

- **Reference systems:** **Redis** (+ Redis Cluster, AOF/RDB), **Memcached** (multi-threaded, slab), **Aerospike** (SSD-optimized), **DragonflyDB** (multi-threaded, shared-nothing), **KeyDB** (multi-threaded Redis fork).
 - **Strongly-consistent KV:** **etcd**, **TiKV**, **FoundationDB** (Raft/Paxos, distributed txns).
 - **Memory:** jemalloc/tcmalloc, **NUMA**, huge pages; persistent memory tiers; **tiered RAM+SSD** (Redis on Flash, Aerospike).
 - **Concurrency:** io_uring, DPDK/kernel-bypass, lock-free/shared-nothing per-core designs (DragonflyDB, Seastar).
 - **Consistency:** Raft/Paxos, CRDTs (Redis active-active / geo), leases.
 - **How I stay current:** Redis/Dragonfly/Aerospike engineering blogs, the Raft & FoundationDB papers, USENIX/VLDB talks, and benchmarking before adopting.
-

17. Likely follow-up questions (rehearse these)

- Single-threaded but fast — how? (*in-RAM $O(1)$ + no lock contention; scale via sharding*)
 - How do you keep p99 low during a resize? (*incremental rehash — migrate a few buckets per op*)
 - `fsync` always vs `everysec`? (*durability vs throughput; ack-after-fsync vs ≤ 1 s loss window*)
 - Crash mid-write to the AOF? (*CRC per record $\rightarrow$ truncate the torn tail on recovery*)
 - Atomic update across two keys? (*same shard via hash tags; MULTI/EXEC on the single owning thread*)
 - Lost-update protection? (*versioned CAS / WATCH optimistic concurrency*)
 - Primary dies with un-replicated writes? (*async = may lose the tail (eventual); use quorum/WAIT for none*)
 - Memory full — then what? (*evict per policy for a cache; reject writes for a store*)
 - Hot key saturating one core? (*replica reads, client cache, key splitting, LFU*)
 - Strong consistency across the cluster? (*Raft/quorum per shard + 2PC across shards — call out the latency cost*)
-

18. Summary checklist (whiteboard recap)

- **$O(1)$ ops** — hash table with **incremental rehash**; entry holds value + version + `expire_at` + eviction meta.
- **TTL** — **lazy expiry + bounded background sweep** (timing wheel / sampling).
- **Eviction** — **approximate LRU/LFU**; `noeviction` for a store, `evict` for a cache.
- **Concurrency** — **single-threaded event loop per shard** $\rightarrow$ atomic ops, CAS, MULTI/EXEC free; scale by sharding.
- **Atomic multi-key** — shard-local via **hash tags**; `WATCH`/version for optimistic txns.
- **Durability** — **AOF (group-commit `fsync`) + fork-CoW snapshots**; LSN orders writes.
- **Recovery** — **snapshot $\rightarrow$ replay AOF tail (lsn>last) $\rightarrow$ CRC-truncate torn record $\rightarrow$ drop expired $\rightarrow$ serve.**
- **Scale** — **hash-slot sharding + primary-replica per shard**; async (latency) vs quorum (durability).
- **Consistency** — per-shard **linearizable (CP)**; cluster **tunably AP/eventual** (PACELC: latency vs consistency).
- **Backpressure** — bounded queues, eviction/reject, output-buffer caps, load shedding $\rightarrow$ degrade, don't melt.
- **Observability** — p99 latency, hit ratio, memory headroom, **replication lag (LSN)**, `fsync` time, evictions.

Designing a Distributed Rate Limiter

A complete, interview-ready walkthrough: requirements → estimates → API → algorithms → architecture → distributed challenges → failure handling → trade-offs. Use the headings as your whiteboard agenda.

0. How to drive the interview (talk track)

1. **Clarify** functional + non-functional requirements (don't assume).
2. **Estimate** scale (QPS, keys, memory) to justify choices.
3. **Define the API / contract** and where the limiter lives.
4. **Pick an algorithm** and explain why (trade-offs vs alternatives).
5. **Go distributed**: shared state, atomicity, race conditions, latency.
6. **Handle failure**: Redis down, hot keys, network partitions.
7. **Summarize trade-offs** (consistency vs availability vs latency).

Keep saying *"here's the trade-off..."* — that's what's being graded.

1. Problem & motivation

A **rate limiter** controls how much traffic a client/service may send within a window. Excess traffic is **throttled** (HTTP 429 Too Many Requests).

Why we need it:

- **Protect** backends from overload / cascading failures.
 - **Security**: blunt brute-force, credential stuffing, scraping, L7 DoS.
 - **Cost control**: cap usage of expensive downstreams (3rd-party APIs, LLM tokens, DB).
 - **Fairness / QoS**: stop one noisy tenant from starving others.
 - **Monetization**: enforce plan tiers (Free = 100/min, Pro = 10k/min).
-

2. Requirements

Functional

- Throttle requests that exceed a configured limit (e.g., 100 req/min per user).
- **Configurable rules** per dimension: user ID, API key, IP, endpoint, tenant, plan tier.
- Support **multiple limits at once** (e.g., 10/sec **and** 1000/hour).
- Return a clear signal when throttled (429 + Retry-After).
- Rules are **hot-reloadable** without redeploy.

Non-functional

- **Low latency**: add only single-digit ms to each request.
- **High availability**: the limiter must not become a single point of failure that takes down the app.
- **Scalable**: millions of req/sec, 100M+ unique keys.
- **Accurate enough**: small over-count at boundaries is usually acceptable.

- **Distributed-correct**: consistent decisions across many limiter instances.

Clarifying questions to ask the interviewer

- Per-user, per-IP, per-API-key, or global limits? (Often **all**, layered.)
- Hard limit (reject) or soft limit (queue/delay)?
- Is **approximate** counting acceptable, or must it be exact?
- Single region or **multi-region**? Global budget or per-region?
- On limiter failure, do we **fail-open** (allow) or **fail-closed** (deny)?

3. Back-of-the-envelope estimation

Assume **1B requests/day**:

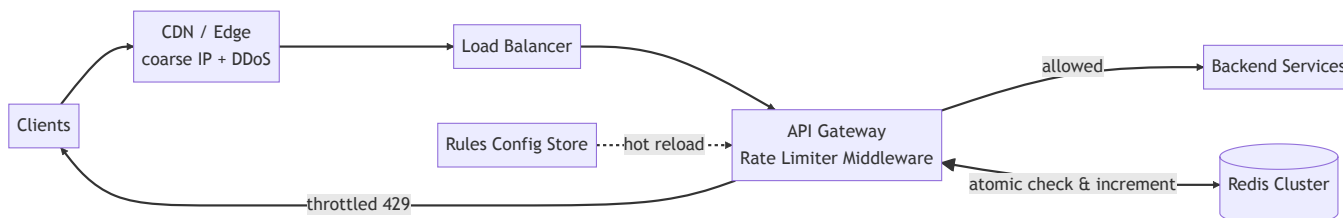
- Average $\approx 1e9 / 86,400 \approx \sim 12k \text{ req/sec}$; peak (5x) $\approx \sim 60k \text{ req/sec}$.
- Each check = 1–2 Redis ops (read + atomic increment).
- **Memory**: ~~100M active keys~~ $\times (\sim 16B \text{ key} + 8B \text{ counter} + \text{overhead} \sim 100B) \sim \sim 10 \text{ GB}^{**}$ → fits in a small Redis cluster, fully in-memory.
- Counters are short-lived (TTL = window), so memory stays bounded.

Takeaway: an in-memory store (Redis) easily handles this; the real challenges are **atomicity, latency, and failure handling**, not raw storage.

4. Where does the rate limiter live?

Placement	Pros	Cons
Client-side	No server cost	Untrusted, easily bypassed/tampered — never the only defense
In-app middleware	Full request context (user, route)	Couples limiter to every service; duplicated logic
API Gateway / dedicated edge	Centralized, language-agnostic, also does auth/TLS/routing	Extra hop; must be HA

Recommendation: a **dedicated middleware at the API Gateway** (Envoy, Kong, NGINX, AWS API Gateway, Cloudflare). Push coarse IP/DDoS limits to the **CDN/edge**; do fine-grained per-user/key limits at the gateway.



5. API & response contract

Config / rules API (control plane):

```
POST /v1/rules
{
  "key": "plan:pro",
  "dimension": "api_key",
  "limit": 10000,
  "window_seconds": 60,
  "algorithm": "token_bucket",
  "burst": 2000
}
```

Enforcement (data plane) is transparent — the limiter inspects each request and either forwards it or returns:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 12
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1717612800
```

Returning `X-RateLimit-*` headers on **successful** responses too lets well-behaved clients self-throttle.

6. Algorithms (know all five + trade-offs)

Algorithm	Memory	Burst handling	Accuracy	Notes / used by
Token Bucket	Low (2 fields/key)	Allows bursts up to bucket size	Good	Smooth, intuitive. <i>Stripe, AWS</i>
Leaky Bucket	Low (queue)	Smooths to constant out-rate	Good	Adds latency; can starve new reqs. <i>Shopify</i>
Fixed Window	Lowest (1 counter)	Bad — 2× burst at edges	Coarse	Simplest; boundary spike problem
Sliding Window Log	High (timestamp/req)	Exact	Best	Stores every timestamp; costly memory
Sliding Window Counter 	Low	Smooths edges	~99.99%	Weighted prev+curr window. <i>Cloudflare</i>

Token Bucket (recommended default)

- Bucket of capacity B , refilled at R tokens/sec. Each request consumes 1 token; empty $\Rightarrow$ reject.
- **Pros:** memory-efficient, allows controlled bursts, easy to reason about.
- **Cons:** two parameters (B , R) to tune.

Fixed Window — the boundary problem

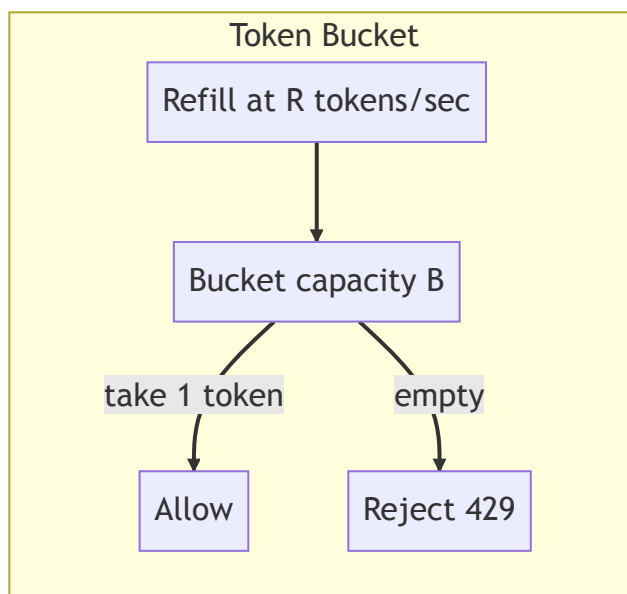
With 100/min, a client can send 100 at 00:00:59 and another 100 at 00:01:00 $\rightarrow$ **200 in ~1s**. This is why we prefer sliding approaches.

Sliding Window Counter (recommended when accuracy matters)

Approximate the rolling window by weighting the previous fixed window:

```
count = curr_window_count
      + prev_window_count * (overlap_fraction_of_prev_window_still_in_view)
allow if count < limit
```

Smooths the edge spike, uses ~2 counters per key, and Cloudflare measured error well under 1%.



7. Distributed design — the hard part

With many limiter instances behind a load balancer, **counter state must be shared and updated atomically**.

7.1 Where to keep counters

- **Centralized Redis (cluster)** — single source of truth; atomic ops; TTL eviction.
- Use **Redis Cluster** sharded by key for horizontal scale; replicas for HA.

7.2 Race conditions (the classic bug)

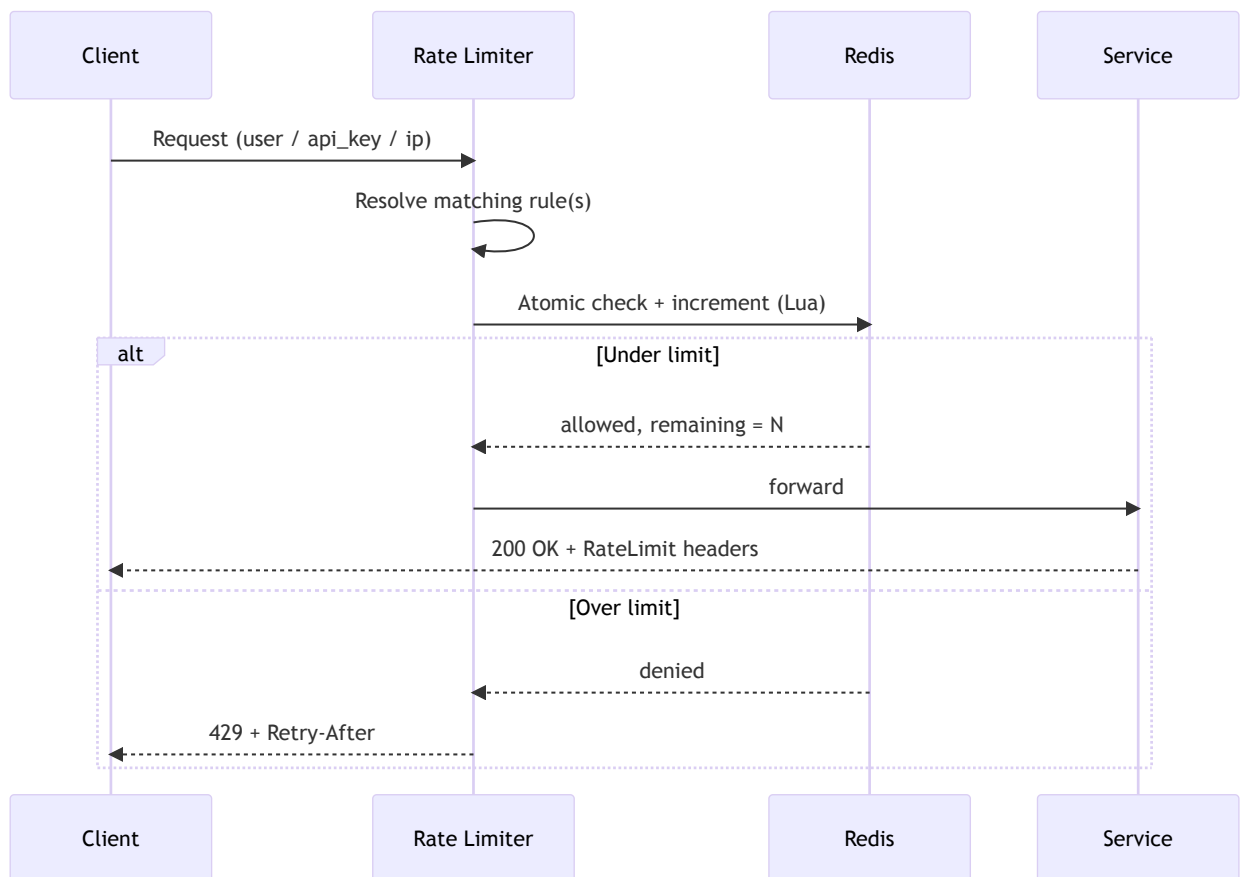
Naïve GET → check → SET is a **read-modify-write race**: two instances both read 99, both allow, count becomes 101.

Fixes (pick atomic):

- **INCR + EXPIRE** (fixed window) — INCR is atomic.
- **Lua script** — runs atomically on the Redis server (best for token/sliding-window logic).
- **Sorted sets** (ZADD/ZREMRANGEBYSCORE/ZCARD) for sliding-window log.
- Avoid distributed **locks** — correct but slow; they kill throughput.

7.3 Latency

- **Co-locate** Redis with limiters (same AZ/region) to keep RTT ~sub-ms.
- For ultra-low latency, allow a **local in-memory cache** with periodic sync (trade accuracy for speed).



8. Reference implementations (Redis)

Fixed window (simple, atomic)

```

key = "rl:{user}:{window_start_epoch}"
count = INCR key
if count == 1:      # first hit in this window
    EXPIRE key window_seconds
if count > limit:
    reject (429)
else:
    allow
  
```

Token bucket (atomic via Lua — avoids races)

```

-- KEYS[1] = bucket key
-- ARGV: capacity, refill_rate, now_ms, requested(=1)
local b = redis.call("HMGET", KEYS[1], "tokens", "ts")
local cap = tonumber(ARGV[1])
local rate = tonumber(ARGV[2]) -- tokens per ms
local now = tonumber(ARGV[3])
local req = tonumber(ARGV[4])

local tokens = tonumber(b[1]) or cap
  
```

```

local ts      = tonumber(b[2]) or now
tokens = math.min(cap, tokens + (now - ts) * rate)  -- refill

local allowed = tokens >= req
if allowed then tokens = tokens - req end

redis.call("HMSET", KEYS[1], "tokens", tokens, "ts", now)
redis.call("PEXPIRE", KEYS[1], math.ceil(cap / rate))
return allowed and 1 or 0

```

Lua guarantees the **refill + check + decrement** happen as one atomic step.

9. Failure scenarios — "what if X fails?"

Failure	Impact	Mitigation
Redis node down	Can't read counters	Replicas + Sentinel/Cluster failover; circuit breaker → local fallback
Redis cluster unreachable	Global enforcement lost	Decide policy: fail-open (availability) vs fail-closed (protection); usually fail-open with a local cap
Hot key (one viral user/IP)	Single shard overloaded	Local pre-aggregation, key splitting/sharding, edge IP limits
Limiter instance dies	Capacity loss	Stateless instances behind LB; just add more
Network partition	Split-brain counts	Accept eventual consistency / over-count; reconcile after heal
Clock skew	Wrong windows	Use Redis server time (<code>TIME</code>) as source of truth

Fail-open vs fail-closed: for most user-facing APIs, **fail-open** (allow traffic if the limiter is down) preserves availability — losing rate limiting briefly is better than an outage. For abuse-sensitive endpoints (login, payments), **fail-closed**.

10. Scaling the system

- **Limiter tier:** stateless → scale horizontally behind the LB.
- **Redis: Cluster** sharded by key; add shards as keys grow; replicas for reads/HA.
- **Hot keys:** two-tier counting — buffer locally per node, flush deltas to Redis every few ms (trades a little accuracy for huge throughput).
- **Multi-region:**
 - *Independent regional limits* (simple, low latency, but global budget can be exceeded) — usually fine.
 - *Global budget* (strong, but needs cross-region coordination → latency) — only if strictly required.
- **Layered limits:** global → per-service → per-tenant → per-user → per-IP, evaluated cheapest-first.

11. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Consistency vs Latency	Centralized Redis (accurate)	Local cache + async sync (fast, approximate)	Most APIs accept slight over-count → favor latency
Availability vs Protection	Fail-open	Fail-closed	Fail-open for general APIs; fail-closed for auth/payments
Accuracy vs Memory	Sliding-window log	Sliding-window counter	Counter $\approx$ same accuracy, far less memory → preferred
Burst tolerance vs Smoothness	Token bucket (bursts ok)	Leaky bucket (constant rate)	Token bucket for APIs; leaky for steady downstreams
Simplicity vs Correctness	Fixed window	Sliding window	Sliding avoids 2x boundary burst

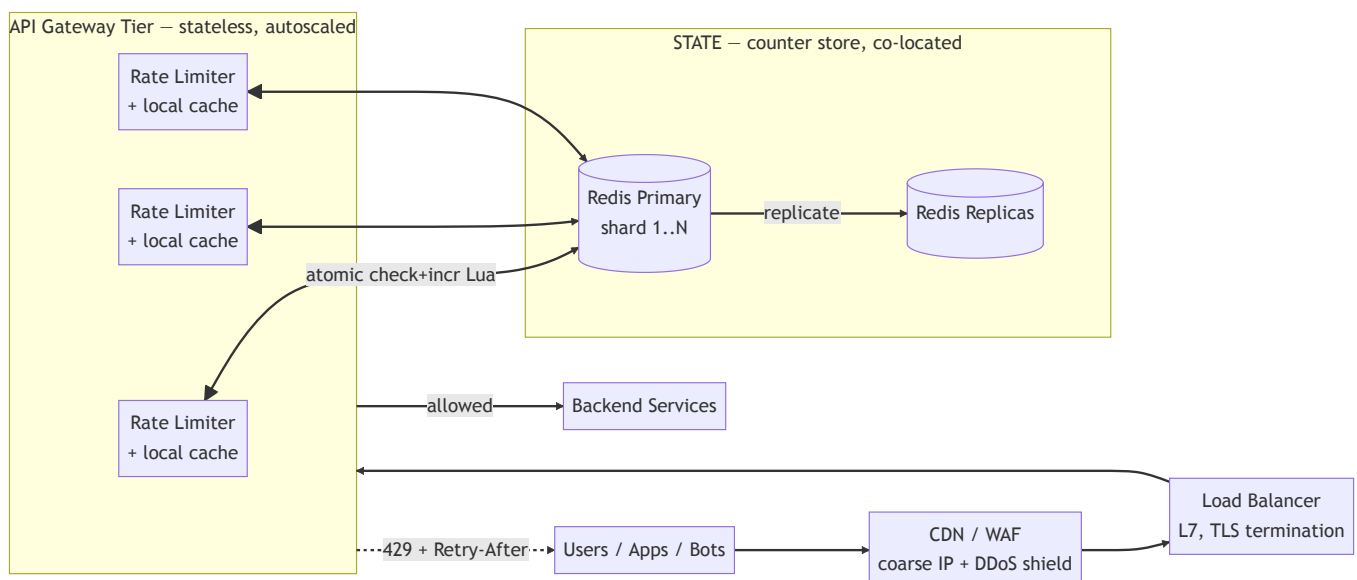
One-liner to say out loud: "I'd start with a token-bucket limiter at the API gateway, counters in a co-located Redis Cluster updated atomically via Lua, fail-open on Redis outage with a conservative local cap, and move to local pre-aggregation only if hot keys or latency force it."

CAP framing: the counter store forces a choice under partition — **fail-open = AP** (stay available, risk over-admitting) vs **fail-closed = CP** (stay correct, risk rejecting valid traffic). Most rate limiters lean **AP**: availability of the protected service usually matters more than perfectly exact counts.

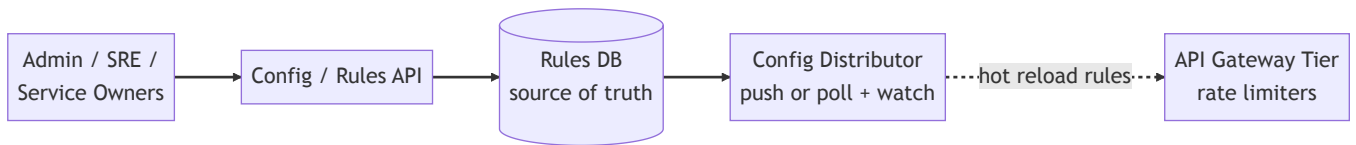
12. Full system design (detailed)

End-to-end view, split into three diagrams for clarity: **(A)** the hot request path (data plane + state), **(B)** the control plane (rule management), and **(C)** observability.

12A. Data plane — the hot request path

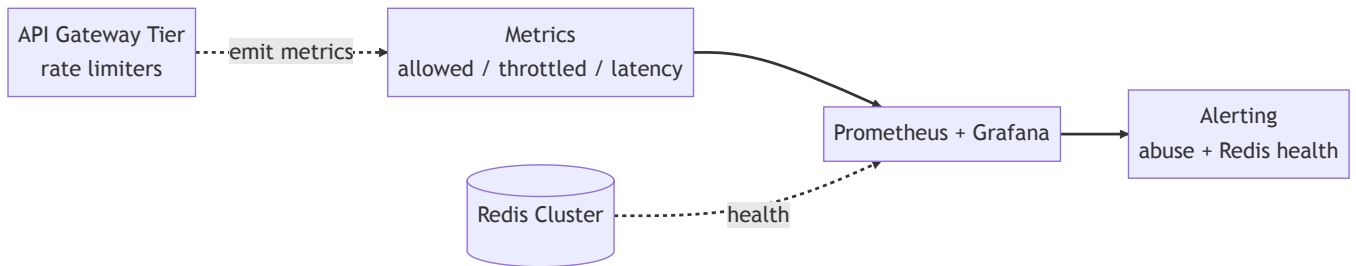


12B. Control plane — rule management (cold path)



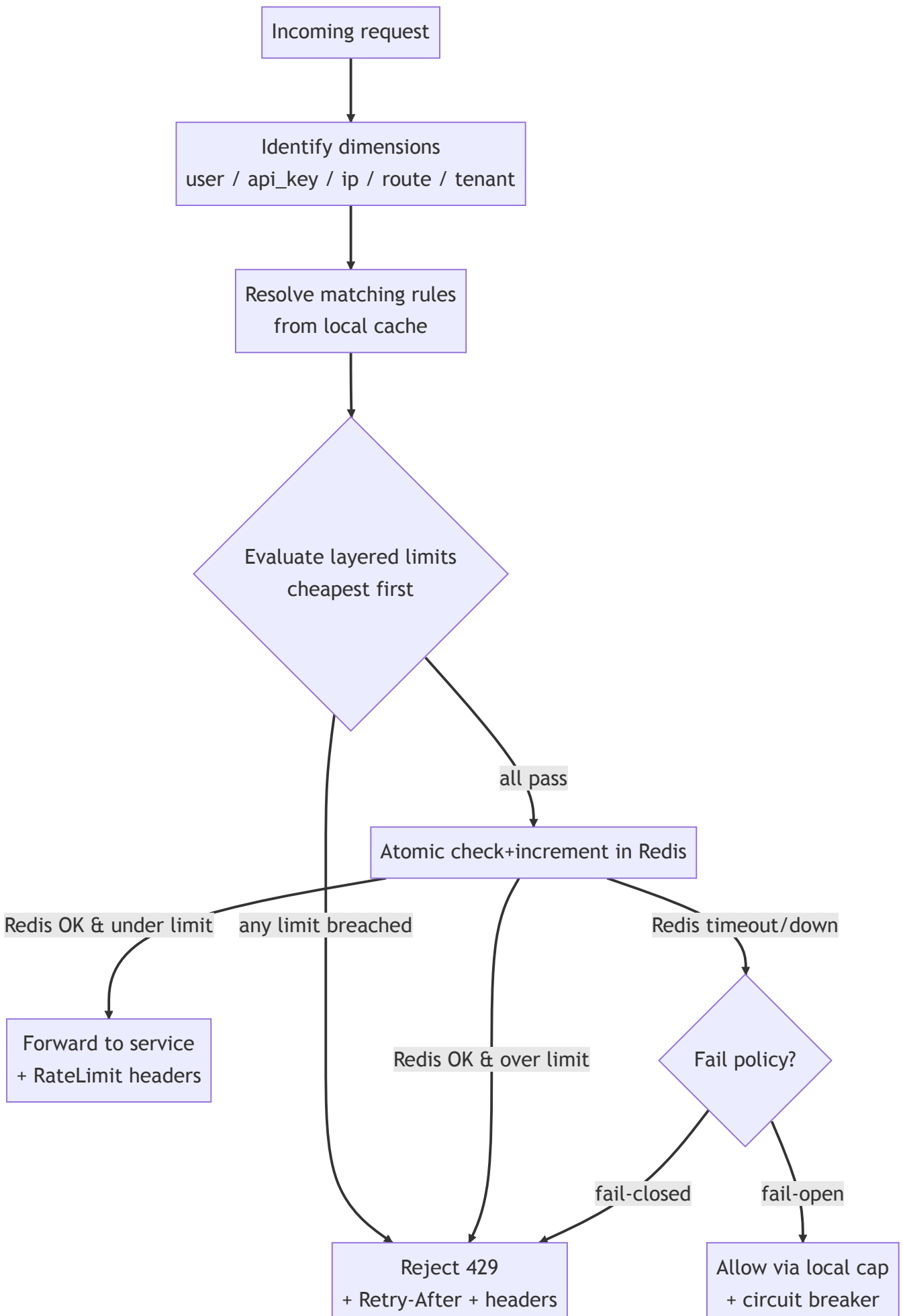
The Rules API + DB are the source of truth; the distributor pushes (or limiters poll + watch) so rules hot-reload into every limiter with **no redeploy**.

12C. Observability



Per-decision metrics (allowed / throttled / latency) drive dashboards and abuse alerts; Redis health feeds the same monitoring so on-call sees failover early.

Request decision logic (inside each limiter)



Components recap

- **CDN / WAF (edge)**: coarse IP-based + DDoS protection before traffic hits us.
 - **Load Balancer**: L7 routing + TLS, spreads load across limiter instances.
 - **Rate Limiter middleware**: stateless, evaluates layered rules, allow/deny, emits metrics.
 - **Redis Cluster**: atomic counters, TTL eviction, sharded by key + replicated for HA.
 - **Control plane**: Rules API + DB are the source of truth; a distributor hot-reloads rules into limiters (push or poll-and-watch), so no redeploy is needed.
 - **Observability**: per-decision metrics drive dashboards, abuse alerts, and Redis health checks.
-

13. Networking, security & performance best practices

Networking

- **Terminate TLS at the edge/LB**; keep limiter→Redis traffic on a private network in the same AZ to minimize RTT.
- **Connection pooling + keep-alive** to Redis — reuse connections instead of reconnecting per request.
- **Pipeline / `MULTI` / single Lua call** to batch ops and cut round-trips (one atomic call beats several).
- **Tight timeouts** on the Redis call (e.g., 5–10 ms) so a slow store can't stall the request path; then apply the fail-open/closed policy.
- **Retries with capped exponential backoff + jitter** — never unbounded; retry storms amplify outages.
- **Anycast/CDN** to absorb volumetric attacks close to the client.

Security

- **Don't trust client-supplied identity**. Derive the IP from the real connection or a *validated* `X-Forwarded-For` — the header is spoofable, so attackers rotate it to dodge per-IP limits.
- **Secure the control plane**. AuthN + RBAC + audit on the rules API so limits can't be tampered with or silently disabled.
- **Layered defense**. Coarse IP/DDoS at the edge/WAF + fine-grained per-identity limits at the gateway; stricter **fail-closed** limits on auth/payment endpoints (brute force, credential stuffing).
- **The limiter must be cheap**. Reject over-limit traffic *before* expensive work (auth, DB) so the limiter isn't itself a DoS amplifier.
- **Avoid info leaks**. Don't let `RateLimit-*` headers or error messages reveal whether an account/key exists (enumeration).

Performance

- **O(1) atomic ops**, no distributed locks (see §7.2).
 - **L1 local cache** for hot keys + async flush (see §10) to shed Redis load.
 - **Pre-resolve rules** from a local cache; never hit the control plane on the hot path.
 - **TTL every key** so memory stays bounded; **sample** metrics to keep observability overhead low.
-

14. Staying current — modern & emerging approaches

- **Managed / cloud-native**: AWS API Gateway usage plans & throttling, Cloudflare Rate Limiting, Azure API Management, GCP Apigee, Kong/Tyk — offload limiting to the platform when it fits.
- **Service mesh**: Envoy's **global rate-limit service (RLS)** backed by Redis, deployed via Istio — limiting as a sidecar concern with no app changes.
- **Battle-tested algorithms/libraries**: **GCRA** (Generic Cell Rate Algorithm) via the `redis-cell` module; app-side libs like `bucket4j`, `resilience4j`, Guava `RateLimiter` (Java), `golang.org/x/time/rate` (Go).
- **Emerging**: **eBPF/XDP** for L3/L4 limiting at kernel/line speed; **WASM** filters in Envoy; **CRDT-based counters** for multi-region eventual consistency without coordination.

- **Standards:** the IETF **RateLimit header fields** draft is standardizing `RateLimit-Limit/Remaining/Reset` — prefer it over ad-hoc `X-RateLimit-*`.
 - **How I stay current:** engineering blogs (Cloudflare, Stripe, AWS Builders'), conference talks, and prototyping + benchmarking before adopting.
-

15. Likely follow-up questions (rehearse these)

- Token bucket vs sliding window — when each? (*burst tolerance vs strict accuracy*)
 - How do you make the counter update atomic? (*INCR/Lua, not GET+SET*)
 - Redis goes down — what now? (*fail-open + circuit breaker + local cap*)
 - One user sends 1M req/sec to one key — hot key mitigation? (*edge limit + local pre-agg + key split*)
 - Multi-region global limit — how? (*coordination cost vs per-region budgets*)
 - How do you limit on multiple dimensions at once? (*evaluate layered rules, reject on first breach*)
 - Exactly-once vs approximate counting trade-off?
-

Distributed Rate Limiter — One-Page Cheat Sheet

Print: open in VS Code → Markdown preview → ⌘P / Print → Save as PDF. Dense by design.

Talk track (order on whiteboard)

Clarify → Estimate → Placement → Algorithm → Distributed/atomicity → Failure → Trade-offs

Requirements

- **Functional:** throttle over-limit traffic; configurable rules per user/IP/API-key/tenant/route; multiple simultaneous limits (10/s **and** 1k/h); return 429; hot-reloadable rules.
- **Non-functional:** single-digit-ms latency; HA (not a SPOF); millions req/s, 100M+ keys; approximate counting OK.
- **Ask:** per-user vs IP vs global? hard vs soft? exact vs approx? multi-region? **fail-open vs fail-closed?**

Estimation (1B req/day)

- ~12k req/s avg, ~60k peak (5x). Check = 1–2 Redis ops.
- 100M keys × ~~100B~~ ~**10 GB** → small Redis cluster, in-memory, TTL-bounded.
- *Conclusion:* storage is easy; **atomicity + latency + failure** are the hard parts.

Placement

Client (untrusted) ✗ · In-app middleware (coupled) ~ · **API Gateway (dedicated)** ✓ · Edge/CDN for coarse IP+DDoS.

Algorithms

Algo	Mem	Burst	Accuracy	Use
Token bucket ✓	Low	Allows bursts	Good	Default for APIs (Stripe/AWS)
Leaky bucket	Low	Constant out-rate	Good	Smooth downstreams; adds latency
Fixed window	Lowest	2x edge spike	Coarse	Simplest only
Sliding log	High	Exact	Best	Costly (timestamp/req)
Sliding counter ✓	Low	Smoothed	~99.99%	Accuracy w/o memory (Cloudflare)

- **Token bucket:** capacity B , refill R/s ; take 1/req; empty $\Rightarrow$ 429. Memory-light, controlled bursts.
- **Fixed-window bug:** 100 at 00:59 + 100 at 01:00 = 200 in ~1s → use sliding.

Distributed correctness (the hard part)

- **Shared state:** centralized **Redis Cluster** (sharded by key, replicas, TTL eviction).
- **Race:** GET→check→SET lets two nodes both read 99 → over-count. **Fix = atomic:**
 - INCR+EXPIRE (fixed window) · **Lua** (token/sliding, atomic refill+check+decrement) · ZADD/ZCARD (sliding log).

- **Avoid distributed locks** (kill throughput).
- **Latency**: co-locate Redis with limiters (same AZ); optional local cache + async sync (trade accuracy for speed).

Atomic fixed-window (Redis)

```
key = rl:{user}:{window_start}
c = INCR key
if c == 1: EXPIRE key window
allow if c <= limit else 429
```

Failure — "what if X fails?"

Failure	Mitigation
Redis node down	Replicas + Sentinel/Cluster failover; circuit breaker → local fallback
Cluster unreachable	Fail-open (avail) vs fail-closed (protect); usually fail-open + local cap
Hot key	Edge IP limit + local pre-aggregation + key splitting
Instance dies	Stateless behind LB → add more
Partition	Accept eventual over-count; reconcile after heal
Clock skew	Use Redis <code>TIME</code> as source of truth

Fail-open general APIs (availability > brief loss of limiting) · **Fail-closed** login/payments.

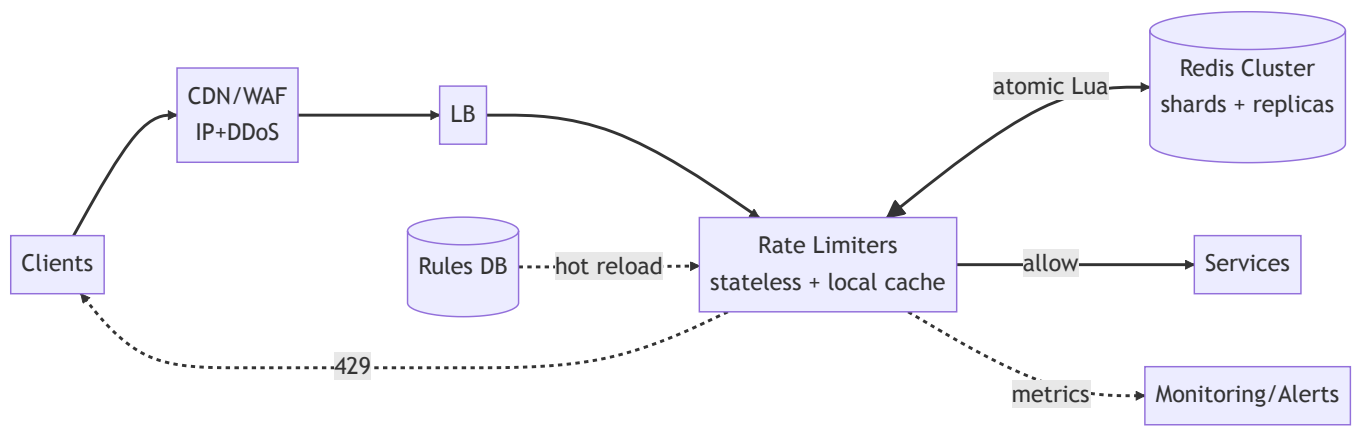
Scaling

- Limiter tier **stateless** → scale horizontally. Redis **Cluster** sharded; add shards as keys grow.
- **Hot keys**: two-tier — buffer locally, flush deltas every few ms.
- **Multi-region**: per-region budgets (simple, fast) vs global budget (coordination → latency).
- **Layered limits**: global → service → tenant → user → IP, cheapest-first, reject on first breach.

Trade-offs (say "here's the trade-off...")

Axis	A	B	Pick
Consistency↔Latency	Central Redis	Local+async	Latency (slight over-count OK)
Availability↔Protection	Fail-open	Fail-closed	Open for APIs / closed for auth
Accuracy↔Memory	Sliding log	Sliding counter	Counter
Burst↔Smooth	Token	Leaky	Token for APIs
Simple↔Correct	Fixed	Sliding	Sliding

Architecture (one glance)



Closing one-liner

"Token-bucket limiter at the API gateway, counters in co-located Redis Cluster updated atomically via Lua, fail-open with a conservative local cap on Redis outage, and local pre-aggregation only if hot keys or latency demand it."

Designing a Slack-like Team Messaging System (Real-Time Focus)

A complete, interview-ready walkthrough: requirements → estimates → entities → APIs → architecture → **real-time delivery, missed-message recovery, ordering, fan-out** → notifications → cold start → storage/scaling → failure handling → trade-offs. Use the headings as your whiteboard agenda. The hard parts are **real-time delivery, ordering, and recovering missed messages across reconnects** — spend your time there.

Think Slack / Microsoft Teams / Discord. Persistent connections deliver messages to channels and DMs in real time; clients must never lose or reorder messages, even across disconnects and offline periods.

0. How to drive the interview (talk track)

1. **Clarify** functional + non-functional requirements.
2. **Estimate** scale (users, concurrent connections, messages/sec, fan-out).
3. **Model entities** (users, workspaces, channels, DMs, messages) + their keys.
4. **Define the APIs** (send/receive) and the **real-time transport** (WebSocket).
5. **Walk the send path**: client → gateway → persist → fan-out → deliver.
6. **Nail ordering** (per-channel sequence) and **missed-message recovery** (cursor + backfill).
7. **Notifications** + the latency/correctness/UX trade-offs.
8. **Cold start** after offline.
9. **Storage, queues, fan-out, scale**; then **failure + trade-offs**.

Keep saying *"here's the trade-off..."* — that's what's being graded.

1. Problem & motivation

Real-time team chat: users in **workspaces** exchange messages in **channels** (group) and **DMs** (1:1 / small group), delivered to all members' devices **instantly**, durably, and **in order** — surviving disconnects, offline periods, and multi-device usage.

What makes it hard:

- **Persistent connections at scale** — millions of always-on WebSockets, not request/response.
- **Fan-out** — one message to a 50k-member channel must reach every online member's device(s) fast.
- **Ordering** — everyone in a channel must see messages in the **same order**; no gaps, no duplicates.
- **Reliability across disconnects** — flaky networks are the norm; a client must **recover exactly the messages it missed**.
- **Multi-device** — the same user on phone + laptop must stay in sync (read state, delivery).
- **Cold start** — opening the app after hours offline must sync efficiently without downloading everything.

The central tension: a message must be **durably stored, globally ordered within its channel, and pushed in real time** — then **reconcilable** when a client was away.

2. Requirements

Functional

- **Send/receive messages** in channels and DMs; edits, deletions, threads, reactions, attachments.
- **Real-time delivery** to all online members' devices (persistent connection).
- **Missed-message recovery** — fetch everything missed while disconnected.
- **Ordering** — consistent per-channel order across all clients.
- **Presence & typing indicators**; read receipts / unread counts.
- **History** — scrollback / search of past messages.
- **Notifications** — push to offline/background devices (mobile/desktop/email).

Non-functional

- **Low latency** — p99 end-to-end delivery < ~200 ms for online users.
- **High availability** — chat must stay up; degrade gracefully.
- **Durability** — an acked message is **never lost**.
- **Scalable** — 10M+ concurrent connections, millions of msgs/sec at peak fan-out.
- **Ordered & exactly-once to the UI** — no gaps, no dupes after dedup.
- **Eventual consistency** acceptable for presence/read-state; **strong ordering** within a channel.

Clarifying questions to ask the interviewer

- **Channel size** — small teams or **giant** channels (50k+)? (Changes fan-out strategy.)
- **Ordering guarantee** — strict per-channel total order, or causal? (Assume **per-channel total order**.)
- **Delivery semantics** — at-least-once + client dedup (→ exactly-once UI), acceptable? (Yes.)
- **Retention** — forever, or N days? Affects storage + search.
- **Multi-device** sync required? (Yes.)
- **E2E encryption**? (Assume server-side encryption at rest, not E2E, like Slack — say so.)
- **Global** users → multi-region? (Yes → regional gateways, home-region per workspace.)

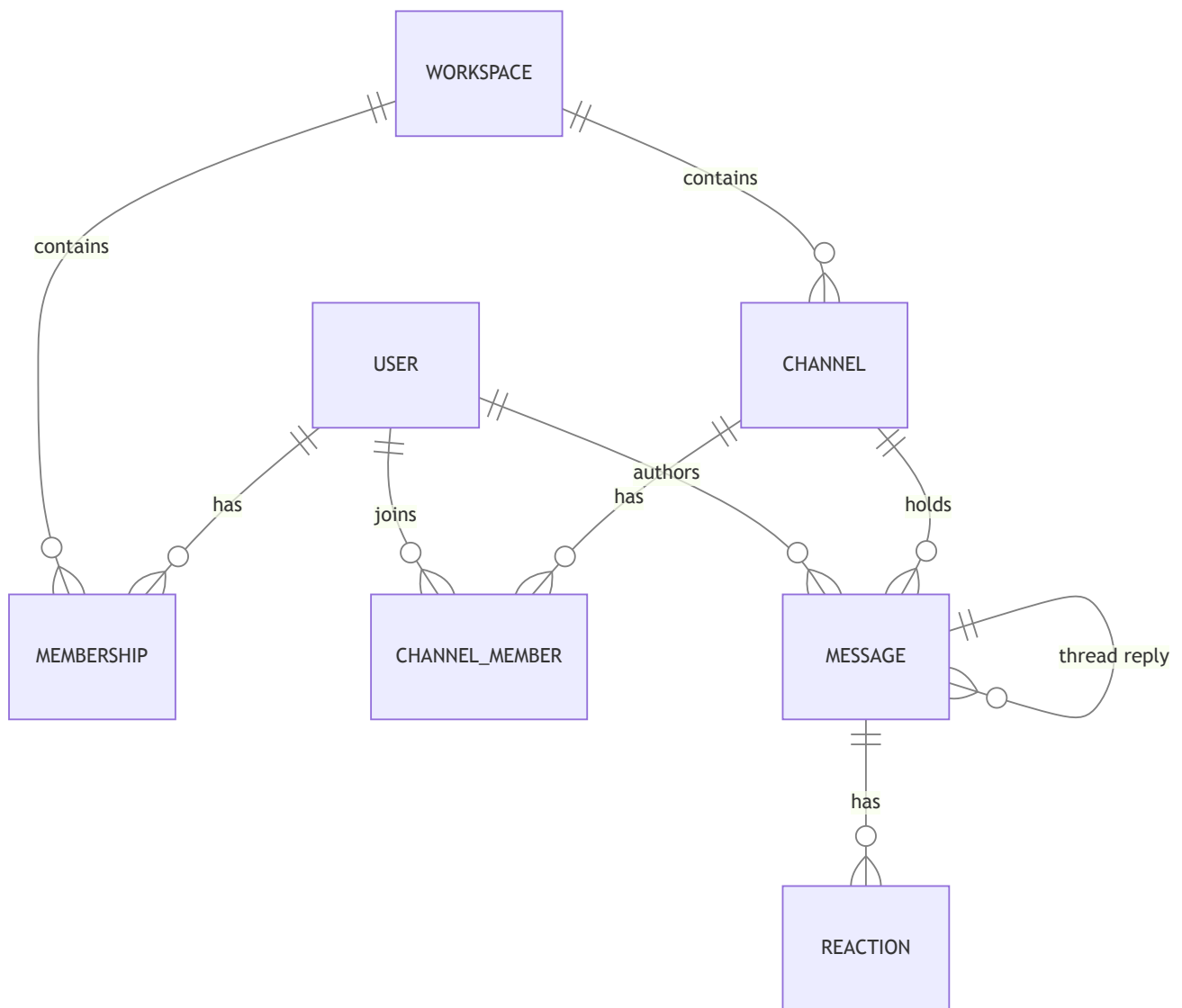
3. Back-of-the-envelope estimation

Quantity	Assumption	Result
DAU	—	50M
Concurrent connections	~40% online, ~1.5 devices	~30M live WebSockets
Messages sent/day	~20 msgs/active user	1B/day ~**12K/sec avg, ~50–100K/sec peak**
Fan-out factor	avg ~50 recipients/msg (channels)	~50B deliveries/day ≈ ~600K/sec avg, several M/sec peak
Connections per gateway	~100K WS/node	~300 gateway nodes for 30M conns
Message size	~1 KB metadata + text	~1 TB/day text; attachments to object store
Storage	1B msgs/day × 1 KB × retention	~PB-scale message store (sharded, tiered)
Read:write	history reads + delivery >> sends	delivery (fan-out) dominates → optimize the push path

Takeaways that drive the design:

1. **30M persistent connections** → a dedicated, horizontally-scaled **connection/gateway tier** decoupled from business logic.
 2. **Fan-out is the bottleneck** (millions of deliveries/sec) → a **message queue + fan-out service**, not synchronous loops.
 3. **Per-channel ordering** at this rate → assign a **monotonic sequence per channel** at persist time.
 4. **PB-scale history** → wide-column store partitioned by channel + time; attachments in object store + CDN.
-

4. Core entities & data model



Entity	Key fields	Notes
User	user_id, name, avatar, status	Global identity
Workspace	workspace_id, name, plan	Tenant boundary; users belong via membership
Membership	(workspace_id, user_id), role	Workspace-level RBAC
Channel	channel_id, workspace_id, type (public/private), name	DMs modeled as a channel with type=dm
ChannelMember	(channel_id, user_id), last_read_seq	last_read_seq drives unread + recovery
Message	message_id, channel_id, seq , sender_id, ts, body, type, thread_id, edited, deleted	seq = per-channel monotonic order
Reaction	(message_id, user_id, emoji)	

DMs = channels. A 1:1 DM is just a channel with two members (type dm); group DM = small private channel. This **unifies the send/deliver/order/recover paths** — you implement them once.

Message identity & ordering keys

```
message_id : globally unique, client-generated UUID (idempotency/dedup)
seq        : per-channel monotonically increasing integer (ORDERING)
ts         : server timestamp (display + tie-break, NOT ordering)
```

- **message_id** (client UUID) → **idempotency**: retries don't duplicate; clients dedup on it.
- **seq** (server-assigned, per channel) → **the ordering and recovery cursor**. "Give me everything in channel C with seq > my_last_seq."

5. APIs

Real-time transport (WebSocket — the primary path)

Client holds a persistent **WebSocket** to a gateway; messages flow both ways as framed events.

```
// client → server: send a message
{ "type":"send", "channel_id":"C1", "client_msg_id":"u-9f3a", "body":"deploying now 🚀" }

// server → client: ack with assigned order
{ "type":"ack", "client_msg_id":"u-9f3a", "message_id":"m_77", "channel_id":"C1", "seq":3412, "ts":...

// server → client: a new message in a subscribed channel
{ "type":"message", "channel_id":"C1", "message_id":"m_78", "seq":3413, "sender":"U2", "body":"..." }

// client → server: heartbeat / cursor checkpoint
{ "type":"ping", "cursors":{"C1":3413, "C2":901 } }

// server → client: presence / typing
```

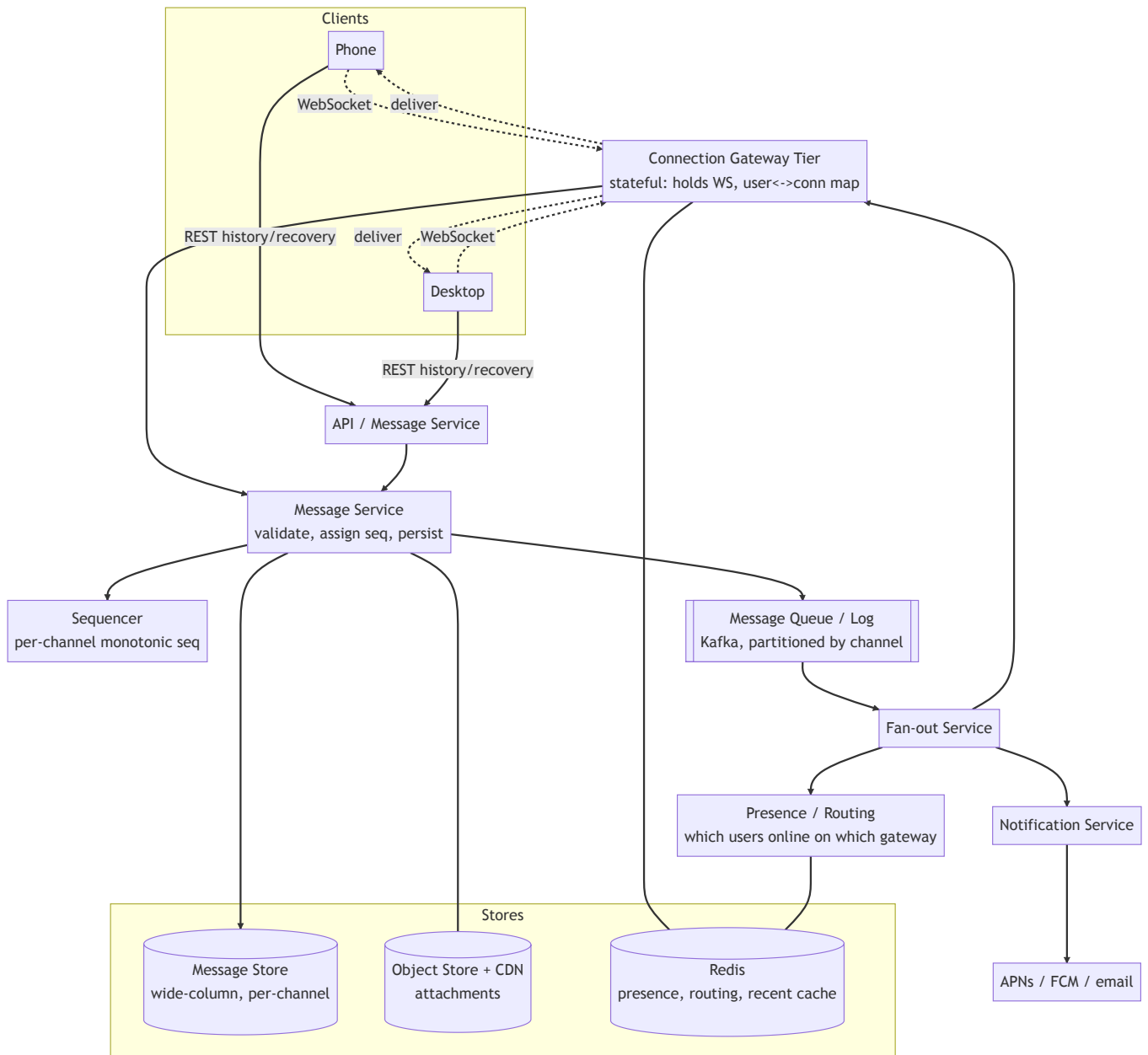
```
{ "type": "presence", "user": "U2", "state": "online" }
{ "type": "typing", "channel_id": "C1", "user": "U2" }
```

REST/HTTP (history, recovery, management — not the hot push path)

```
POST /v1/channels/{id}/messages      {client_msg_id, body}      # send (fallback if WS down)
GET  /v1/channels/{id}/messages?after_seq=3413&limit=50      # backfill missed / scrollbar
GET  /v1/channels/{id}/messages?before_seq=3000&limit=50     # history (older)
POST /v1/channels/{id}/read          {up_to_seq:3413}          # advance read cursor
GET  /v1/sync?since=<cursor_token>    # cold-start multi-channel delta
POST /v1/channels {workspace_id, type, members}              # create channel / DM
```

Why both WS and REST? WebSocket carries the **live** push (low latency); REST carries **bulk history & recovery** (cursor-paginated, cacheable, retry-friendly). Sends can go over either — WS when connected, REST as a fallback — both idempotent via `client_msg_id`.

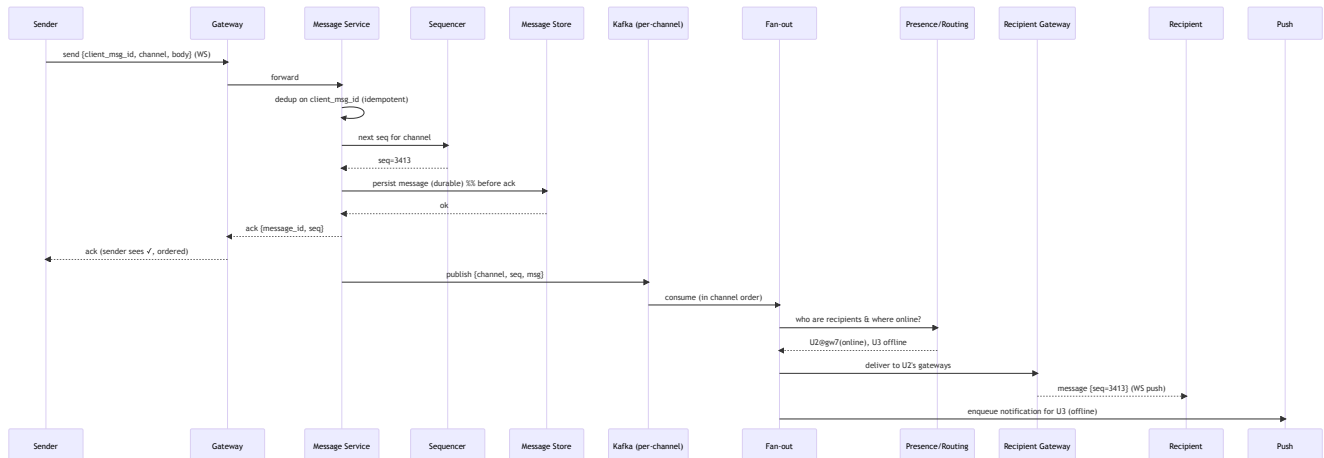
6. Architecture overview



Component responsibilities

- **Connection Gateway** — terminates WebSockets; maintains the **user** → **connection(s)** → **gateway** routing map (in Redis); pushes events to connected devices. *Stateful but disposable* — on disconnect the client reconnects (possibly to another gateway) and recovers.
- **Message Service** — validates, **assigns per-channel seq**, persists the message (durability **before** ack), publishes to the queue. The point of truth for "the message exists."
- **Sequencer** — hands out monotonic per-channel sequence numbers (see §8).
- **Message Queue/Log (Kafka)** — durable, **partitioned by channel_id** → preserves per-channel order and decouples persist from fan-out; absorbs spikes; replayable.
- **Fan-out Service** — for each message, find recipients, look up who's online & on which gateway (presence/routing), and deliver; enqueue notifications for offline users.
- **Presence/Routing** — tracks online users and their gateway(s) in Redis (TTL heartbeats).
- **Notification Service** — push (APNs/FCM), desktop, email for offline/background users.
- **Stores** — message store (durable history), Redis (presence/routing/recent-message cache), object store + CDN (attachments).

7. Data flow: send → persist → fan-out → deliver



Key points:

1. **Persist before ack.** The sender isn't told "delivered/ordered" until the message is **durable** and has a **seq**. Guarantees an acked message is never lost.
2. **Idempotent send.** `client_msg_id` dedup means a retried send (flaky network) doesn't create duplicates.
3. **Kafka partitioned by channel** → fan-out consumes a channel's messages **in seq order**, so delivery preserves order.
4. **Fan-out is async + decoupled** → the sender's ack isn't blocked on delivering to 50k recipients.
5. **Online** → push via gateway; **offline** → notification queue.

8. Message ordering (focus area)

Goal: every client in a channel sees the same order, with no gaps or duplicates. Wall-clock timestamps are *not* reliable ordering (clock skew, concurrent sends) — use a **per-channel monotonic sequence**.

Assign a per-channel `seq` at persist time

- The **Message Service** (via the Sequencer) assigns `seq = last_seq(channel) + 1` atomically when persisting.
- **Total order per channel**; `seq` is the contract for both **display order** and **recovery cursor**.
- Cross-channel order is **not** guaranteed (and not needed) — ordering is *per channel*, which is the natural unit.

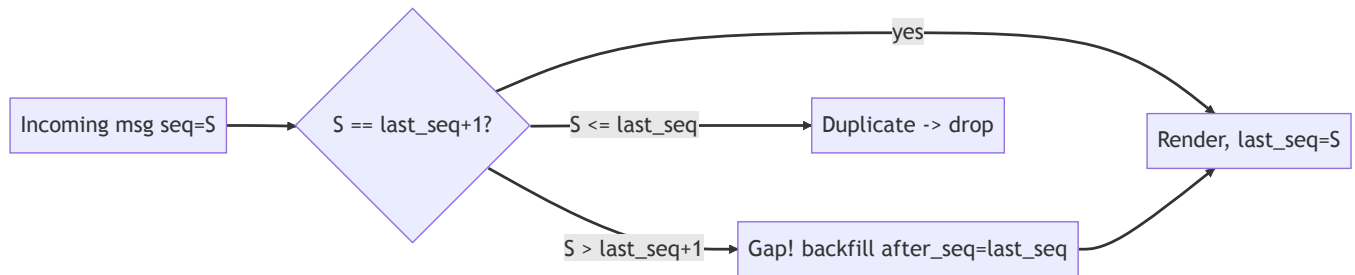
How to implement the sequencer (options)

Approach	How	Trade-off
DB atomic counter / conditional write	UPDATE channels SET last_seq=last_seq+1 RETURNING OR Cassandra LWT	Simple, strongly consistent; the channel row is a hot spot for very busy channels
Single-partition Kafka per channel	Kafka offset = order	Order = partition offset for free; ties ordering to the log
Per-channel owner (sharded actor)	One service instance owns a channel, assigns seq in memory, persists	Fast, no DB contention; needs ownership/failover (consistent hashing)
Logical clock (Lamport/snowflake)	Monotonic ID	Globally unique + roughly ordered; for strict per-channel, still reconcile via seq

Recommended: a **per-channel sequencer** (sharded by `channel_id` via consistent hashing so one owner serializes a channel) writing to a **Kafka partition keyed by `channel_id`**. Order is then preserved end-to-end: assign → log → fan-out → deliver, all per-channel ordered.

Client-side ordering & gap detection

- Client tracks `last_seq` per channel. Incoming `seq` should be `last_seq + 1`.
- **Gap** (`seq > last_seq + 1`) → client missed messages → **trigger backfill** `GET ?after_seq=last_seq (§9)`.
- **Duplicate/old** (`seq <= last_seq`) → dedup/ignore (at-least-once delivery → **exactly-once at the UI**).



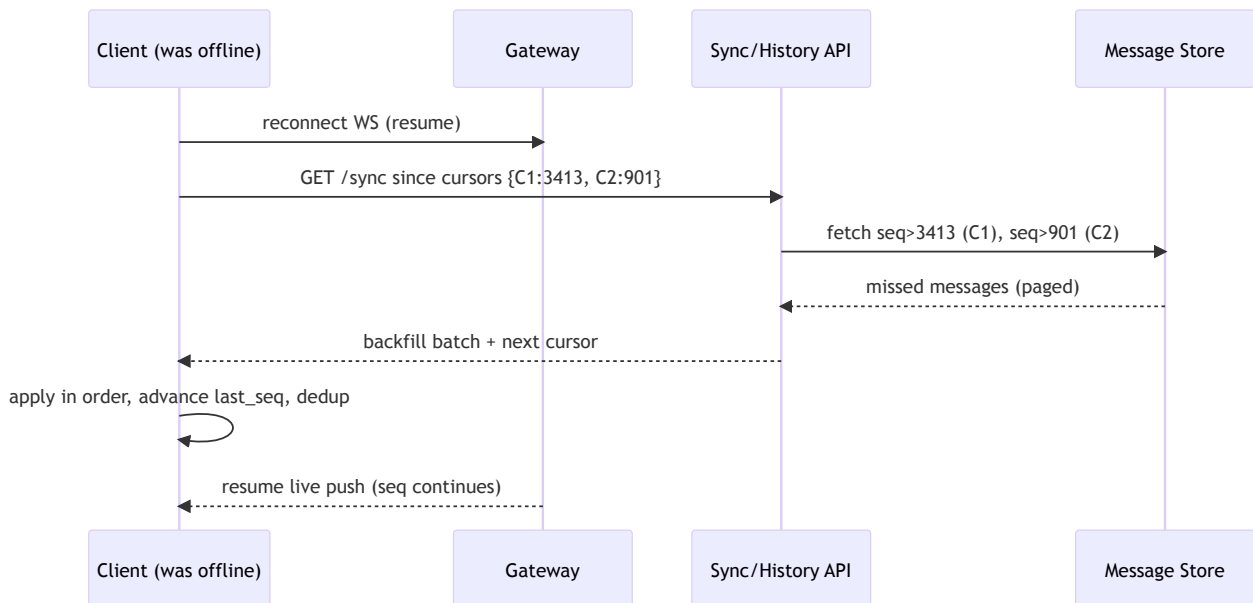
One-liner: "Order is a **per-channel monotonic seq** assigned at persist time by a channel-sharded sequencer over a channel-keyed Kafka partition — timestamps are display-only. Clients detect gaps when `seq` skips and backfill; duplicates are dropped, giving exactly-once at the UI."

9. Missed-message recovery (focus area)

Disconnects are constant (subway, sleep, app background). A reconnecting client must get **exactly what it missed** — efficiently.

The mechanism: per-channel cursors + backfill by seq.

1. **Client persists `last_seq` per channel** (the highest contiguous `seq` it has rendered).
2. **On reconnect**, the client sends its cursors (per channel, or a compact bundle) — or calls `GET /v1/sync?since=cursor`.
3. **Server computes the delta:** for each channel, return messages with `seq > last_seq` (paginated).
4. **Client applies** them in order, advances `last_seq`, then resumes live push.
5. **Live + backfill reconciliation** — messages arriving live during backfill are buffered and **deduped by `seq/message_id`** so nothing is lost or doubled.



Design choices & guarantees:

- **At-least-once delivery + idempotent client** (dedup on `message_id/seq`) → **no loss, no duplicates** at the UI.
- **Server doesn't need per-client outboxes** for history — the **durable message store + seq cursor** is the recovery log. (For brief blips, a short per-connection replay buffer in the gateway avoids a DB hit.)
- **Bounded backfill** — page the delta; if a client is *way* behind (offline for days), fall back to **cold start** (§11) rather than streaming millions of messages.
- **Acks both ways** — server acks sends (durable); clients ack/advance read cursors so the server knows what's delivered/read (drives unread counts + notification suppression).

One-liner: "The durable per-channel message log *is* the recovery mechanism — clients keep a `last_seq` cursor and on reconnect pull `seq > last_seq`, dedup by id, then resume live. No message is lost because we persist before ack; none is duplicated because the client dedups."

10. Fan-out & delivery at scale (focus area)

One message → potentially tens of thousands of recipient **devices**. Naive "loop and send synchronously" doesn't scale.

Fan-out strategies (mirror the social-feed push/pull trade-off)

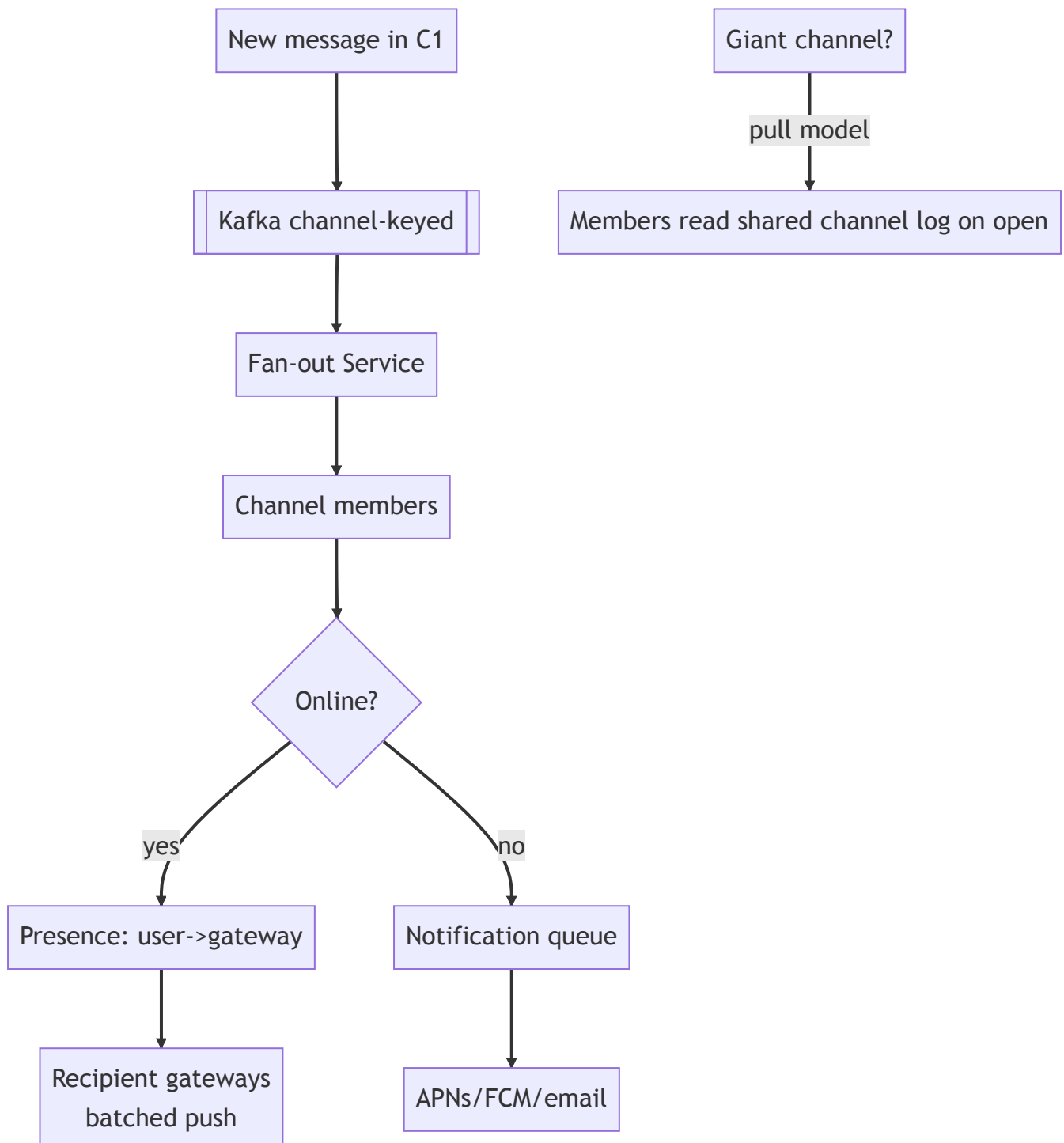
Strategy	How	Best for
Fan-out on write (push)	On send, deliver to every online member's gateway now	Normal channels; real-time feel ✅
Fan-out on read (pull)	Store once; members read the channel log on open	Huge channels (50k+), mostly-offline members
Hybrid ✅	Push to online members; offline → notification + pull on cold start ; giant channels → pull	Real-world default

- **Online members** → push immediately via their gateway(s) (look up presence/routing in Redis).
- **Offline members** → **don't push**; enqueue a **notification** and let them **pull** (recover) on return. Saves the vast majority of wasted deliveries.

- **Giant channels** → avoid fan-out write amplification: members **read the shared channel log** (pull) rather than copying the message into 50k inboxes. The channel's recent messages are cached once (Redis) and served to many — the **celebrity/hot-channel** pattern.

Delivery path mechanics

- **Presence/routing table** (Redis): `user → {gateway, conn_ids}` with heartbeat TTL. Fan-out asks "who's online and where," then sends each gateway a batched delivery for its connected recipients.
- **Per-gateway batching** — one network call carries many recipients on the same gateway.
- **Backpressure** — if a client is slow, buffer briefly then drop to "reconnect & backfill" rather than blocking fan-out.
- **Multi-region** — workspace has a **home region**; gateways are regional; cross-region delivery goes via replicated Kafka.

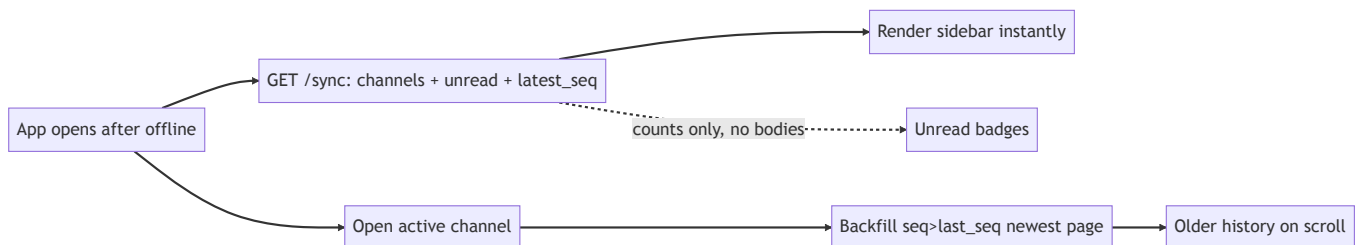


11. Cold start (focus area)

User opens the app after being offline for hours/days. Goal: **fast, bounded sync** — usable UI in a moment, not a multi-GB download.

Strategy: sync state, lazy-load history.

1. **Bootstrap** — fetch the **workspace/channel list + per-channel unread counts and latest seq** in one `GET /v1/sync` (compact: "C1 has 14 unread, latest seq 3500; your last_seq 3486"). Cheap; renders the sidebar immediately.
2. **Prioritize visible/active channels** — backfill **only the channel the user opens first** (and maybe DMs / @-mentions), `seq > last_seq`, newest page first. Other channels load on demand (when opened) or lazily in the background.
3. **Bounded recovery** — if a channel is thousands behind, **don't stream all of it**; show "you're caught up to here," load the most recent page, and page older on scroll. Unread **counts** come from `latest_seq - last_read_seq` without fetching bodies.
4. **Notifications already summarized** the important stuff (mentions/DMs) → cold start confirms & clears them.
5. **Snapshot/caching** — recent messages per channel cached in Redis/CDN so cold-start reads are fast and don't hammer the message store.



Trade-off: unread **counts** (cheap, from seq math) vs full message **bodies** (lazy). Compute badges from `latest_seq - last_read_seq`; fetch bodies only when a channel is opened. Avoids the "download everything" cold-start tax.

One-liner: "Cold start syncs **metadata + unread counts from seq cursors** instantly, then **lazily backfills** only the channel the user actually opens — counts are `latest_seq - last_read_seq`, so badges are free and history is paid for on demand."

12. Notifications & the latency/correctness/UX trade-offs (focus area)

Notifications target **offline/background** devices (the online path is the WebSocket push). The interesting part is the **trade-offs**.

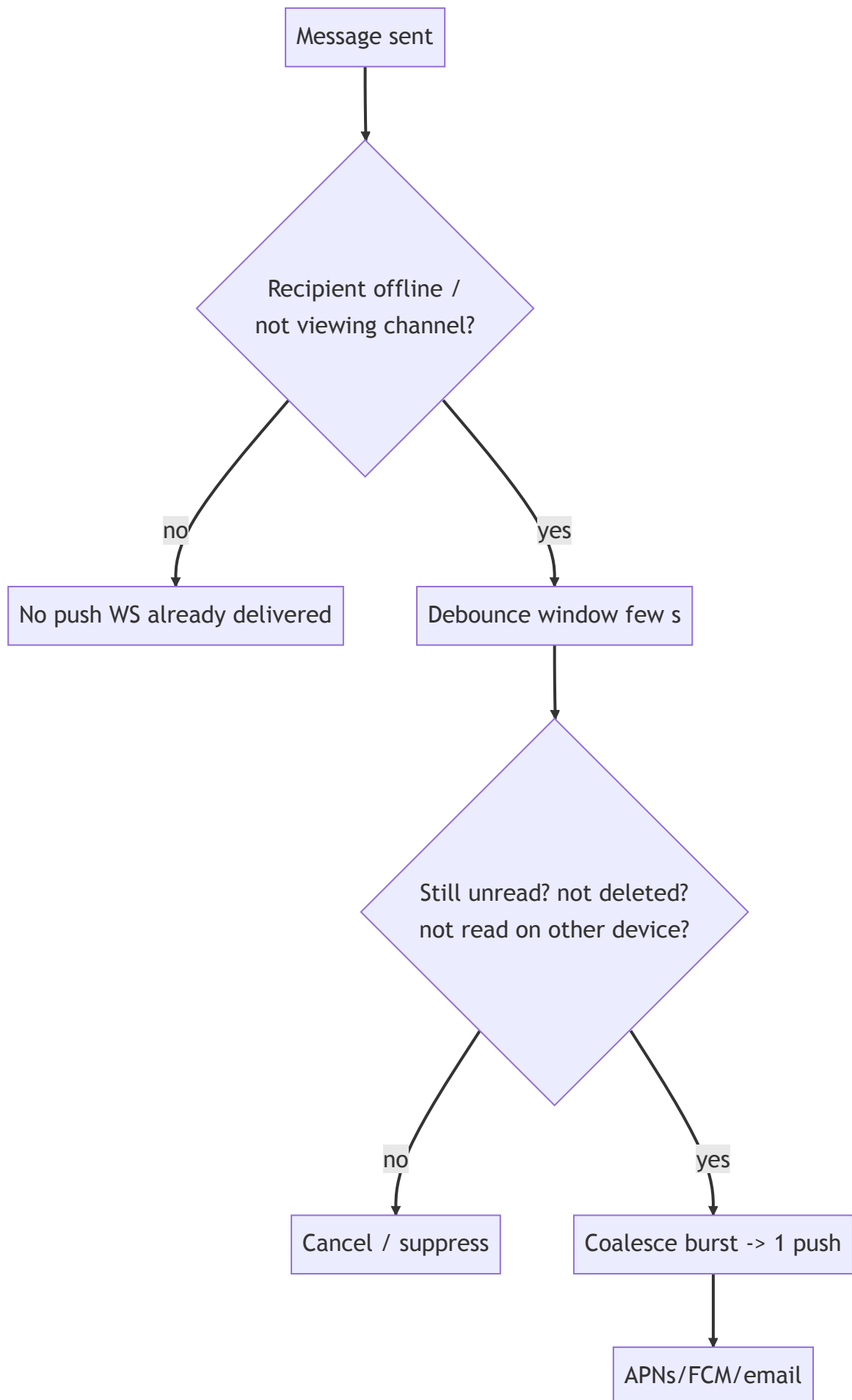
- **Who gets notified** — only if the user is **not actively viewing** that channel on an online device (suppress if they're already reading it). Respect **mentions-only / muted / DND** preferences.
- **Multi-device coordination** — if delivered+read on the laptop, **suppress/clear** the phone push. Requires read-state sync (read cursors broadcast to the user's devices).
- **Channels** — APNs/FCM (mobile), desktop notifications, email digests for long-offline.

The three-way tension

Want	Pull	Tension
Low latency	Fire push immediately on send	May notify for a message the user already saw elsewhere, or that later gets deleted/edited → incorrect/annoying
Correctness	Wait to confirm still-unread + not-viewed-elsewhere + not-deleted	Adds delay (debounce window) → higher latency
UX	Batch/debounce ("3 new messages"), respect DND, dedup across devices	Coalescing adds delay; too much → feels laggy; too little → notification spam

Resolution — a short debounce + suppression window:

- Hold a brief window (e.g., a few seconds) before pushing: lets **read-state from another device** suppress it, **edits/deletes** cancel it, and **bursts coalesce** ("Alice sent 3 messages"). Costs a little latency for much better correctness/UX.
- **DMs / @mentions** → **near-immediate** (latency matters most); **noisy channels** → **batched/digested** (UX matters most). Tune per type.
- **Idempotent notifications** (dedup key per message+device) so retries don't double-buzz.



One-liner: "Online delivery goes over the socket; **notifications are for offline/background only**, gated by a short debounce-and-suppress window so reads on another device, edits, and deletes can cancel them and bursts coalesce — DMs/mentions fire fast (latency), noisy channels batch (UX), all idempotent (correctness)."

13. Storage choices

Data	Store	Why
Messages (history)	Wide-column (Cassandra/HBase/Bigtable), partition <code>channel_id</code> , cluster by <code>seq</code>	Massive write volume, ordered range scans per channel (<code>seq > x</code>), horizontal scale
Channel/user/membership metadata	Relational (Postgres) or wide-column	Relational RBAC, transactions for seq counter
Recent messages / hot channels	Redis cache	Sub-ms reads for live + cold-start + giant-channel pull
Presence / routing	Redis (TTL heartbeats)	Ephemeral, fast <code>user→gateway</code> lookups
Attachments	Object store (S3) + CDN	Blobs never in the message DB; edge delivery
Message log / pipeline	Kafka (partition by channel)	Durable ordered log; decouples persist↔fan-out; replay
Search index	Elasticsearch (async from Kafka)	Full-text history search
Notifications outbox	Queue + store	Reliable async push

Message store schema (wide-column):

```
messages_by_channel (  
  channel_id  partition key,  
  seq         clustering key ASC,      -- ordered scans: seq > last_seq  
  message_id, sender_id, ts, body, type, thread_id, edited, deleted  
)
```

Partitioning by `channel_id` + clustering by `seq` makes **both** hot paths cheap: deliver/recover = a forward range scan `seq > cursor`; history = a backward scan. Very large/old partitions are bucketed by time and tiered to cold storage.

14. Scalability considerations

- **Connection tier scales independently** — ~100k WS/node; add nodes behind an L4/L7 LB; sticky enough that a user's devices land predictably, but **any gateway can serve any user** (routing via Redis) so failover is cheap.
- **Stateless business services** — Message/Fan-out/Notification scale horizontally; state lives in Kafka/DB/Redis.
- **Kafka partitioning by channel** — parallelism across channels while preserving per-channel order; rebalance partitions as load grows.
- **Hot/giant channels** — switch to **pull** (shared cached log) to avoid fan-out write amplification; replicate the hot channel cache across Redis replicas.
- **Sharding** — message store sharded by `channel_id`; presence sharded by `user_id`.

- **Multi-region** — regional gateways for low RTT; **workspace home region** owns ordering; cross-region via replicated Kafka; presence is regional with a global directory.
- **Backpressure & graceful degradation** — queues absorb spikes; under extreme load, shed non-critical (typing/presence) before messages; slow clients fall back to reconnect+backfill.
- **Read scaling** — recent-message cache + read replicas; cold start uses counts (seq math) not bulk bodies.

15. Failure scenarios — "what if X fails?"

Failure	Impact	Mitigation
Client disconnects	Misses live messages	Reconnect + backfill by seq cursor ; brief gateway replay buffer for blips
Gateway node dies	Its connections drop	Clients reconnect to another gateway (routing via Redis) + recover; stateless-ish tier
Message Service down	Sends fail	Client retries (idempotent <code>client_msg_id</code>); HA replicas; persist-before-ack means no half-sent state
Kafka lag/backlog	Delivery delayed	Consumers autoscale; ordering preserved (log); eventual delivery; live feel degrades gracefully
Sequencer/owner failover	Risk of seq gap/dup	Persist seq with the message + conditional writes; new owner resumes from <code>max(seq)</code> in store
Duplicate delivery (at-least-once)	Client sees dupes	Dedup on message_id/seq → exactly-once at UI
Presence stale	Push to a dead conn	TTL heartbeats; failed push → mark offline → notification path
Hot channel storm	Fan-out overload	Pull model + cached shared log + replicas
Region outage	Workspace unreachable	Replicated Kafka + store; failover home region; clients reconnect & recover
Notification storm	Spam / double-buzz	Debounce/coalesce + idempotent dedup key + DND

Guiding principle: persist before ack (never lose an acked message) + **at-least-once + client dedup** (exactly-once UI) + **cursor recovery** (heal any gap). The system favors **durability and ordering**; presence/typing are best-effort.

16. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Delivery semantics	Exactly-once on the wire (hard, slow)	At-least-once + client dedup	At-least-once + dedup → exactly-once at the UI ✓
Ordering	Global total order (bottleneck)	Per-channel order	Per-channel <code>seq</code> — the natural unit, scalable ✓
Fan-out	Push to all (fast, costly)	Pull on read (cheap, slower)	Hybrid : push online, pull for giant/offline
Recovery	Per-client server outbox	Durable log + seq cursor	Cursor on the shared log — no per-client state ✓
Notification	Instant (low latency)	Debounce (correct/UX)	Debounce + suppress; DMs fast, channels batched
Consistency	Strong everywhere	Eventual for presence/read-state	Strong ordering ; eventual presence/typing
Connection	Long-poll (simple)	WebSocket (live, bidirectional)	WebSocket for push; REST for history/recovery
Timestamp vs seq	Order by <code>ts</code> (skew bugs)	Order by <code>seq</code>	<code>seq</code> orders; <code>ts</code> is display-only

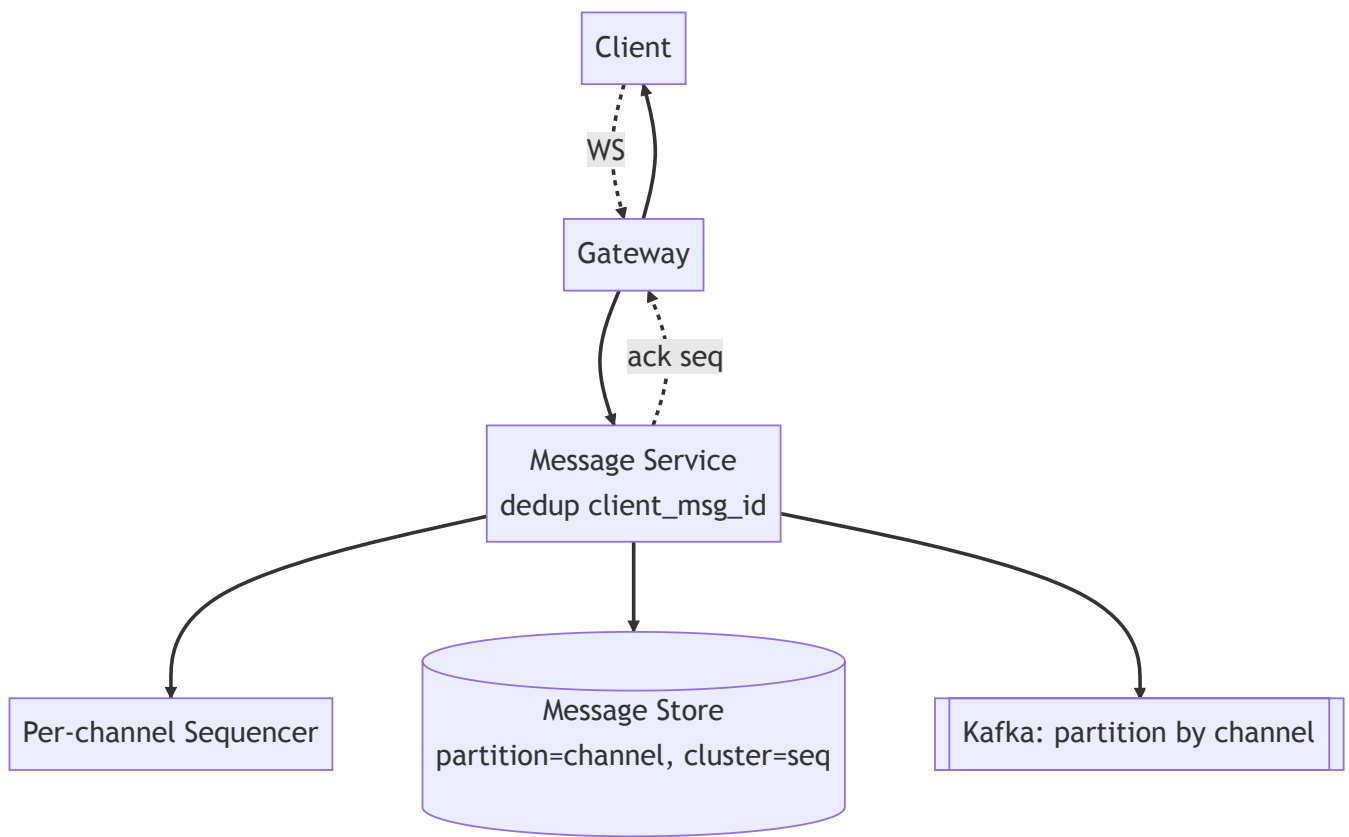
CAP framing: chat leans **AP for delivery/presence** (stay available; reconcile via backfill) but needs **CP-ish ordering within a channel** (the per-channel sequencer is the consistency anchor). Acked-message **durability is non-negotiable**.

One-liner to say out loud: "Clients hold **WebSockets** to a scalable gateway tier; a send is **persisted and assigned a per-channel seq before it's acked**, published to a **channel-partitioned Kafka log**, and **fanned out** to online members' gateways (offline → notifications). **Ordering** is the per-channel `seq`; **recovery** is a `seq` cursor — on reconnect a client pulls `seq > last_seq` from the durable log and dedups by `message_id`, giving no loss and exactly-once at the UI. Giant channels switch to a **pull** model, cold start syncs **unread counts from seq math** and lazy-loads bodies, and notifications use a **debounce-and-suppress** window to trade a little latency for correctness and UX."

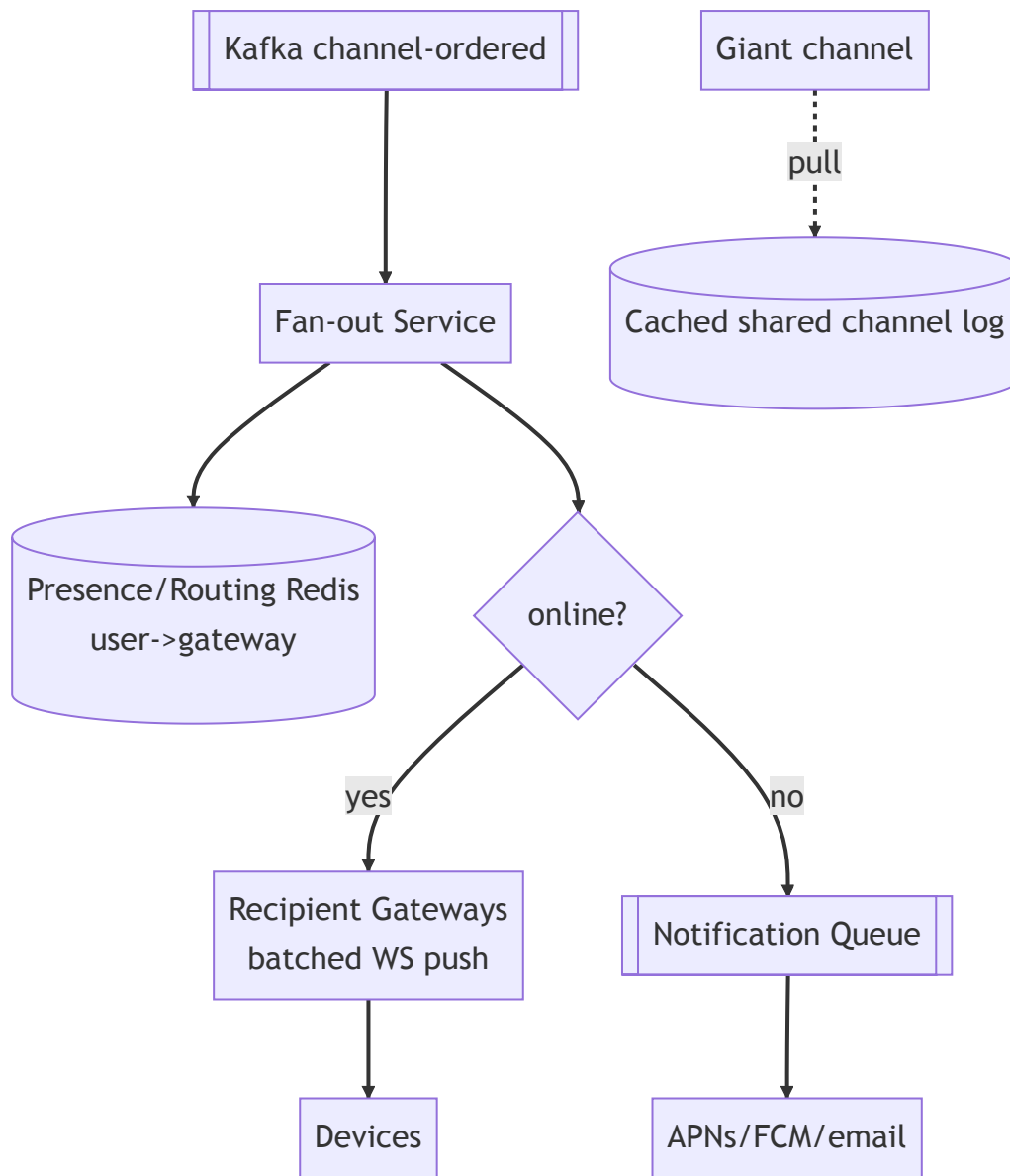
17. Full system design (detailed)

End-to-end, split into **(A)** the send/persist/order path, **(B)** the fan-out/delivery path, and **(C)** reconnect/recovery + notifications.

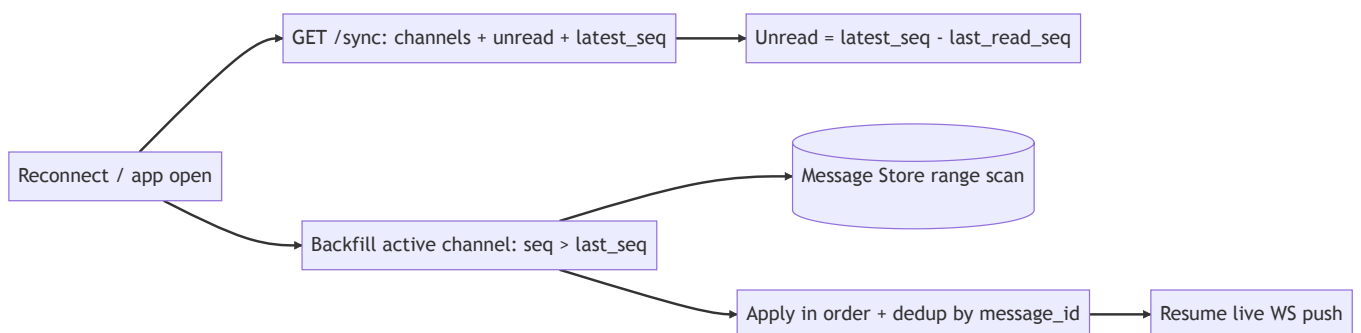
17A. Send → persist → order



17B. Fan-out → delivery



17C. Reconnect, recovery & cold start



18. Networking, security & performance best practices

Networking

- **Persistent WebSocket per device** with heartbeats + idle timeout; **auto-reconnect** with **backoff** + **jitter**.

- **Independent gateway tier** (~100k conns/node) behind an L4/L7 LB; **routing via Redis** so any gateway can serve any user (cheap failover).
- **TLS at the edge**; **HTTP/3 (QUIC)** for mobile resilience; **gRPC** internal; **per-gateway batched fan-out** (one call, many recipients).
- **Multi-region** — regional gateways for low RTT; workspace **home region owns ordering**; **replicated Kafka** cross-region.

Security

- **AuthN/Z on connect and per-channel** (RBAC: can this user read/post this channel?).
- **Server-side encryption at rest**; **per-workspace data isolation** (tenant boundary).
- **DLP / content + attachment scanning**; **signed expiring URLs** for files; malware scan on upload.
- **Rate-limit sends** + abuse/spam detection; avoid presence/user enumeration leaks.
- **Audit** security-relevant events (channel access changes, admin actions).

Performance

- **Per-channel seq + channel-keyed Kafka** → ordered, parallel fan-out.
- **Hybrid fan-out**: push to online members, **pull a cached shared log** for giant channels.
- **Recent-message cache** (Redis); **cold start = unread counts from seq math**, lazy-load bodies.
- **Client dedup** on `message_id/seq` → exactly-once at the UI; **bounded** cursor backfill.

19. Staying current — modern & emerging approaches

- **Reference systems**: **Slack** (channel servers + Flannel edge cache), **Discord** (Elixir/Erlang gateways, **ScyllaDB**), **WhatsApp/Signal** (Erlang/BEAM).
- **Stores**: **Cassandra/ScyllaDB/Bigtable** for messages, **Redis** for presence, **Kafka/Pulsar** for the ordered log.
- **Transport**: **WebSocket**, **HTTP/3/QUIC**, **MQTT** for mobile push, **gRPC** streaming.
- **Push**: **APNs/FCM**, **Web Push**; silent/background push for sync.
- **Patterns**: **CRDTs** for offline edits, **event sourcing**, the **transactional outbox** for reliable fan-out.
- **How I stay current**: **Discord/Slack/Signal** engineering blogs, **Erlang/Elixir** at-scale talks, distributed-systems papers.

20. Likely follow-up questions (rehearse these)

- How do you guarantee per-channel ordering? (*monotonic seq at persist time, not wall-clock timestamps*)
- Client offline for a day — recover missed messages? (*seq cursor backfill; fall back to cold start if far behind*)
- At-least-once delivery — how do you avoid duplicates? (*dedup on message_id/seq → exactly-once UI*)
- A 50k-member channel — fan-out? (*switch to pull: members read one cached shared log*)
- Notification correctness vs latency? (*short debounce + suppress window; DMs fast, channels batched*)
- A gateway node dies — do users drop messages? (*reconnect to another gateway via Redis routing + backfill*)
- Multi-device read-state sync? (*broadcast read cursors to the user's devices*)
- Sequencer hot spot on a busy channel? (*channel-sharded owner / Kafka partition offset as order*)

21. Summary checklist (whiteboard recap)

- **Entities** — users, workspaces, channels, DMs(=channels), messages; **DMs unified as channels**.
- **Transport** — **WebSocket** for live push; **REST** for history/recovery; sends idempotent via `client_msg_id`.
- **Ordering** — **per-channel monotonic seq** assigned at persist time (channel-sharded sequencer + channel-keyed Kafka); timestamps display-only.

- **Recovery** — durable log + **seq cursor**; reconnect pulls `seq > last_seq`, dedups by id → no loss, exactly-once UI.
- **Persist before ack** — an acked message is never lost; **at-least-once + client dedup**.
- **Fan-out** — hybrid: push to online via presence/routing, offline → notifications, **giant channels** → **pull** a cached shared log.
- **Cold start** — sync metadata + **unread counts from seq math**, lazy-load bodies on open.
- **Notifications** — offline/background only; **debounce + suppress** (read-elsewhere/edit/delete cancel; bursts coalesce); DMs fast, channels batched.
- **Storage** — wide-column messages (partition=channel, cluster=seq), Redis presence/cache, Kafka log, object store + CDN attachments, ES search.
- **Scale** — independent connection tier (~100k WS/node), stateless services, channel-partitioned Kafka, multi-region home-region ordering.
- **CAP** — AP for delivery/presence; per-channel sequencer is the consistency anchor; durability non-negotiable.

Designing an Online Chess Platform (Real-Time, Timed Games)

A complete, interview-ready walkthrough: requirements → estimates → entities → APIs → architecture → **matchmaking, server-authoritative move validation, the game clock, real-time delivery, game end & ratings, persistence/replay** → scaling → failure handling → trade-offs. Use the headings as your whiteboard agenda. The hard parts are the **server-authoritative clock (who flags first under network lag), move validation, and rating-based matchmaking** — spend your time there.

Think Lichess / Chess.com. Two human players are matched by strength, join a real-time session, exchange moves over a persistent connection; the **server is the sole authority** on legality, game state, and the clock. Finished games are persisted as PGN for later viewing.

0. How to drive the interview (talk track)

1. **Clarify** functional + non-functional requirements (timed 1v1, rating-based matchmaking, server-authoritative).
2. **Estimate** scale (concurrent players, concurrent games, moves/sec, storage).
3. **Model entities** (players, games, moves, clocks, ratings) + their keys.
4. **Define the APIs** (matchmaking REST + the **real-time move transport** over WebSocket).
5. **Walk the game lifecycle**: queue → match → session → move → validate → clock → broadcast → end → persist.
6. **Nail the clock** (server-authoritative time, flagging, lag compensation) and **move validation** (server-side rules engine).
7. **Matchmaking** (rating windows) + **reconnection** + rating updates.
8. **Persistence & replay** (PGN), then **storage, scaling, failure + trade-offs**.

Keep saying "*here's the trade-off...*" — that's what's being graded. The signature line: "**the server is authoritative for legality, state, and time; clients are just views.**"

1. Problem & motivation

A real-time online chess service: players request a **timed** game, get **matched by rating**, join a **live session**, and alternate **moves**. The server **validates every move**, maintains the **authoritative board and clocks**, decides the **end** (checkmate / resign / draw / timeout), and **persists** the finished game for replay.

What makes it hard:

- **Server-authoritative clock** — deciding *who ran out of time first* is subtle under network latency. A move sent "on the buzzer" may arrive after the server thinks time expired. Flag the wrong player and the game is ruined.
- **Server-authoritative move validation** — never trust the client. Full chess rules (castling, en passant, promotion, check, checkmate, stalemate, threefold repetition, 50-move rule, insufficient material) must run on the server to prevent illegal-move cheating.
- **Rating-based matchmaking** — pair players of similar strength quickly, per time control, widening the search as they wait.
- **Real-time, consistent state** — both players (and spectators) must see the **same** board and clocks with low latency, surviving reconnects.
- **Fairness & anti-cheat** — legality is enforced structurally; engine-assistance cheating needs statistical detection.

The central tension: the game must feel **instant** to both players, yet the **server alone** decides legality, state, and time — and it must resolve the **flag race** fairly when the network is slow.

2. Requirements

Functional

- **Start a timed game** — player picks a time control (e.g., 1+0 bullet, 3+2 blitz, 10+0 rapid) and joins a queue.
- **Rating-based matchmaking** — pair with an opponent of similar rating in that time-control pool.
- **Real-time game session** — both players connected; moves delivered live to the opponent.
- **Server-side move validation** — only legal moves accepted; illegal moves rejected.
- **Authoritative clocks** — per-player countdown; increment support; **timeout $\Rightarrow$ loss** (unless already decided).
- **Game end** — checkmate, resignation, draw (stalemate / threefold / 50-move / insufficient material / agreement), timeout, abort.
- **Rating update** — recompute both players' ratings on completion.
- **Persistence & replay** — store the finished game (moves + clocks + result) as **PGN** for later viewing.
- **(Extensions)** spectating, draw offers / takebacks, reconnection, rematch, anti-cheat.

Non-functional

- **Low latency** — p99 move round-trip $< \sim 100$ ms for co-located players; the clock must feel exact.
- **Correctness** — the authoritative game state and clock are **never wrong**; a legal game is never corrupted by a bad move or a mis-flag.
- **High availability** — an in-progress game must survive a server restart (don't lose a game people are playing).
- **Durability** — a finished, rated game is **never lost**; ratings update atomically.
- **Scalable** — millions of concurrent players, hundreds of thousands of concurrent games.
- **Fairness** — clock decisions are lag-compensated and reproducible from server timestamps.

Clarifying questions to ask the interviewer

- **Only human vs human**, or also vs bots / puzzles? (Assume **human 1v1**.)
- **Time controls** — bullet/blitz/rapid only, or classical/correspondence (days per move)? (Assume real-time bullet $\rightarrow$ rapid; note correspondence differs.)
- **Rating system** — Elo or **Glicko-2**? (Assume Glicko-2 like Lichess; Elo is the simple fallback.)
- **Spectators** — required? (Yes, read-only; affects fan-out.)
- **Reconnection policy** — does the clock keep running while a player is disconnected? (Yes, with a small grace window.)
- **Anti-cheat depth** — structural legality only, or engine-detection too? (Legality now; statistical detection as an async subsystem.)
- **Global** — multi-region for latency? (Yes $\rightarrow$ regional game servers, pair by region.)

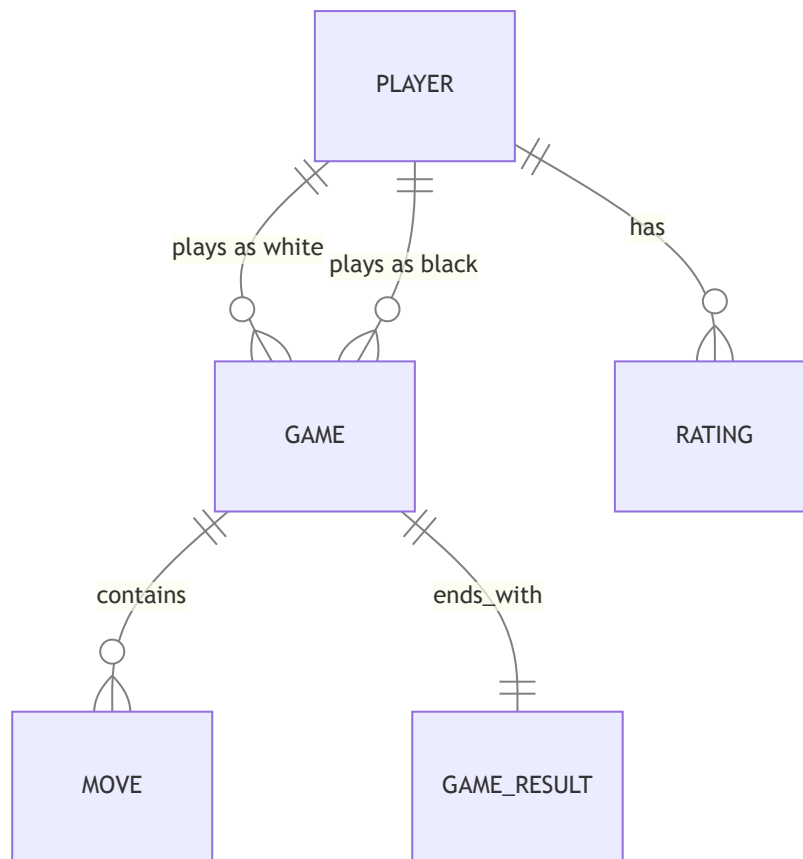
3. Back-of-the-envelope estimation

Quantity	Assumption	Result
DAU	—	10M
Concurrent players (peak)	~10% online & playing	~1M in games
Concurrent games	2 players/game	~500K live games
Games/day	~5 games/active user	~50M games/day
Move rate	blitz $\approx$ 40 full moves over ~6 min $\Rightarrow$ ~0.2 plies/s/game	~100–200K moves/sec peak
Move validation cost	rules engine per move	microseconds — CPU-cheap
Live game state	board + clocks + history	~a few KB/game $\Rightarrow$ ~GBs total in memory
Persisted record	moves + clocks + metadata as PGN	~5–10 KB/game $\Rightarrow$ ~0.5 TB/day
Matchmaking pool	waiting players per time control	thousands, sorted by rating

Takeaways that drive the design:

1. **Games are tiny and CPU-light**, but **stateful and long-lived** (minutes). $\rightarrow$ a **game-server tier that owns games in memory** (one authoritative owner per game), not a stateless request/response service.
2. **Real-time move exchange** for 1M connections $\rightarrow$ a **connection/gateway tier** with persistent WebSockets, decoupled from game logic.
3. **The clock is the hard correctness problem**, not throughput $\rightarrow$ invest in **server-authoritative timing + lag compensation**.
4. **Persistence is modest** (~0.5 TB/day) and **append-only** $\rightarrow$ cheap to store every game as PGN for replay.

4. Core entities & data model



Entity	Key fields	Notes
Player	player_id, name, rating, rating_deviation, country	Identity + strength; <code>rating</code> per time-control category
Rating	(player_id, category), value, RD, volatility, games	Glicko-2 state per bucket bullet/blitz/rapid
Game	game_id, white_id, black_id, time_control, status, start_ts, region	Owned by one game server while live
GameState (<i>in-memory</i>)	FEN, side_to_move, castling, ep_target, halfmove_clock, position_hashes	Authoritative live state; rebuildable from moves
Clock (<i>in-memory</i>)	white_ms, black_ms, increment_ms, turn_started_ts	Server-authoritative remaining time
Move	(game_id, ply), uci, san, clock_ms_after, ts	Ordered by ply (half-move index) — the ordering key
GameResult	game_id, winner, reason, white_rating_delta, black_rating_delta	checkmate / resign / draw / timeout / abort

Ordering key = ply. Chess is strictly turn-based, so each game has a single monotonic **half-move index (ply)**. Move `ply = n` is only legal from the position after `ply = n-1`. This makes ordering trivial (one writer per game) — unlike chat, there's no fan-out sequencing problem.

Time-control notation

```
base + increment    (minutes + seconds/move)    e.g., 5+3 = 5 min base, +3 s per move
category:  bullet (< 3 min)  ·  blitz (3–8)  ·  rapid (8–25)  ·  classical (> 25)
```

Rating is tracked **per category** (a strong bullet player may be an average rapid player). Matchmaking pools are keyed by exact time control.

5. APIs

Matchmaking & lifecycle (REST/HTTP)

```
POST /v1/matchmaking/queue      {time_control:"5+3"}           # enter the pool, returns ticket
DELETE /v1/matchmaking/queue    {ticket_id}                     # cancel search
GET /v1/games/{id}              -> {fen, clocks, moves, status} # snapshot (resume/replay)
POST /v1/games/{id}/resign      # resign
POST /v1/games/{id}/draw        {action:"offer|accept|decline"} # draw negotiation
GET /v1/players/{id}/games?before=<cursor> # finished-game history (replay list)
```

Real-time transport (WebSocket — the hot path)

Once matched, each player holds a persistent **WebSocket** to the game's server; moves and clock updates flow as framed events.

```
// server -> both: game starts, seats + clocks assigned
{ "type":"game_start", "game_id":"g_42", "color":"white", "opponent":{"name":"Mia","rating":1587}, "t:

// client -> server: submit a move (from-square, to-square, optional promotion)
{ "type":"move", "game_id":"g_42", "ply":7, "uci":"e7e8q", "client_ts": 1721659200123 }

// server -> both: accepted move + AUTHORITATIVE clocks after applying it
{ "type":"move_applied", "ply":7, "uci":"e7e8q", "san":"e8=Q+", "fen":"...", "clocks":{"white":297400,

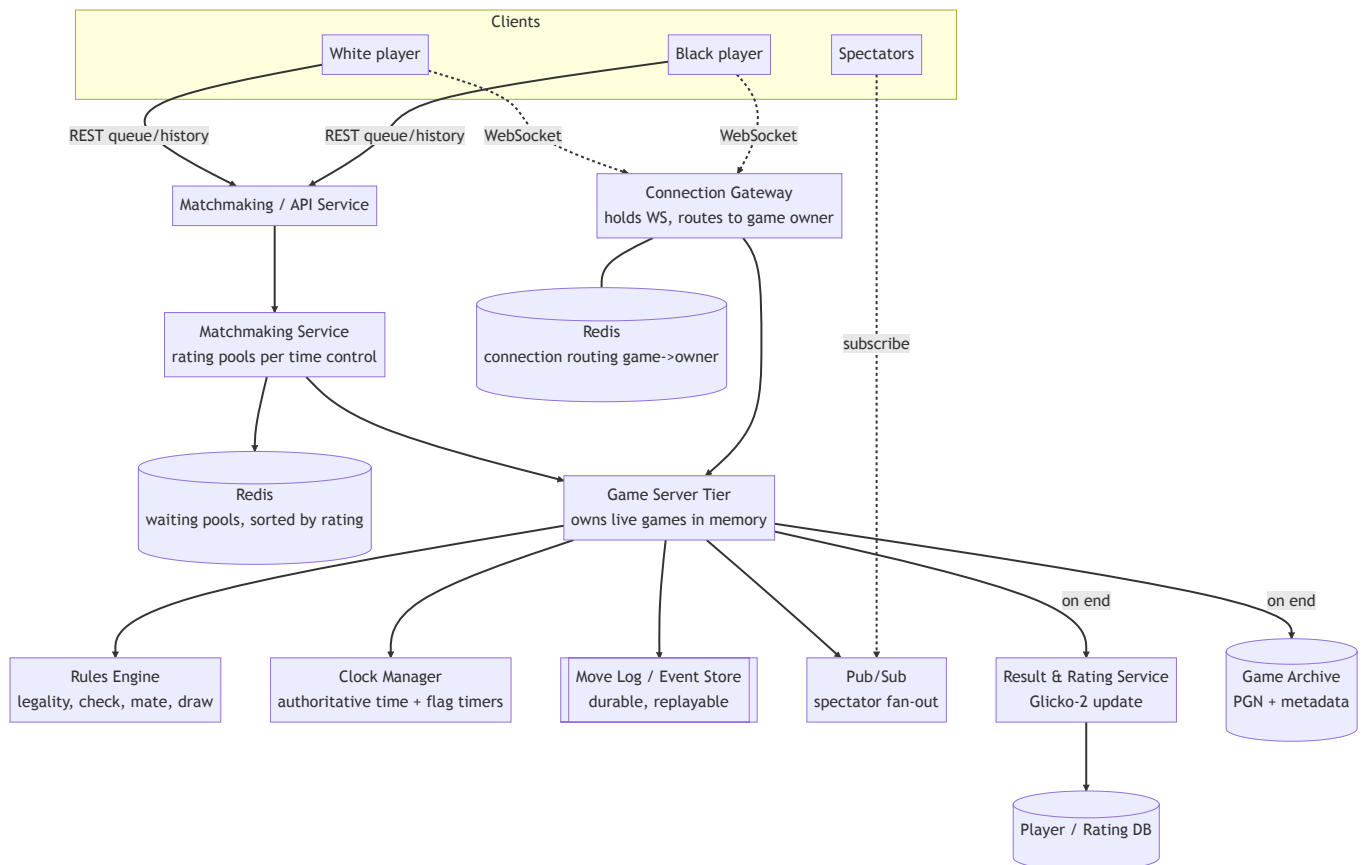
// server -> the mover: rejected (illegal or not your turn) -> client rolls back
{ "type":"move_rejected", "ply":7, "reason":"illegal_move" }

// server -> both: periodic authoritative clock sync (client interpolates between)
{ "type":"clock", "clocks":{"white":295800,"black":301300}, "turn":"white", "server_ts": 1721659203000

// server -> both: game over
{ "type":"game_over", "result":"black_wins", "reason":"timeout", "rating_delta":{"white":-8,"black":+8}}
```

Why WebSocket for moves, REST for the rest? Moves and clocks need **push, low latency, bidirectional** → WebSocket. Matchmaking, snapshots, history, and replay are **request/response, cacheable, retry-friendly** → REST. Moves carry a **ply** so the server can reject stale/duplicate submissions idempotently.

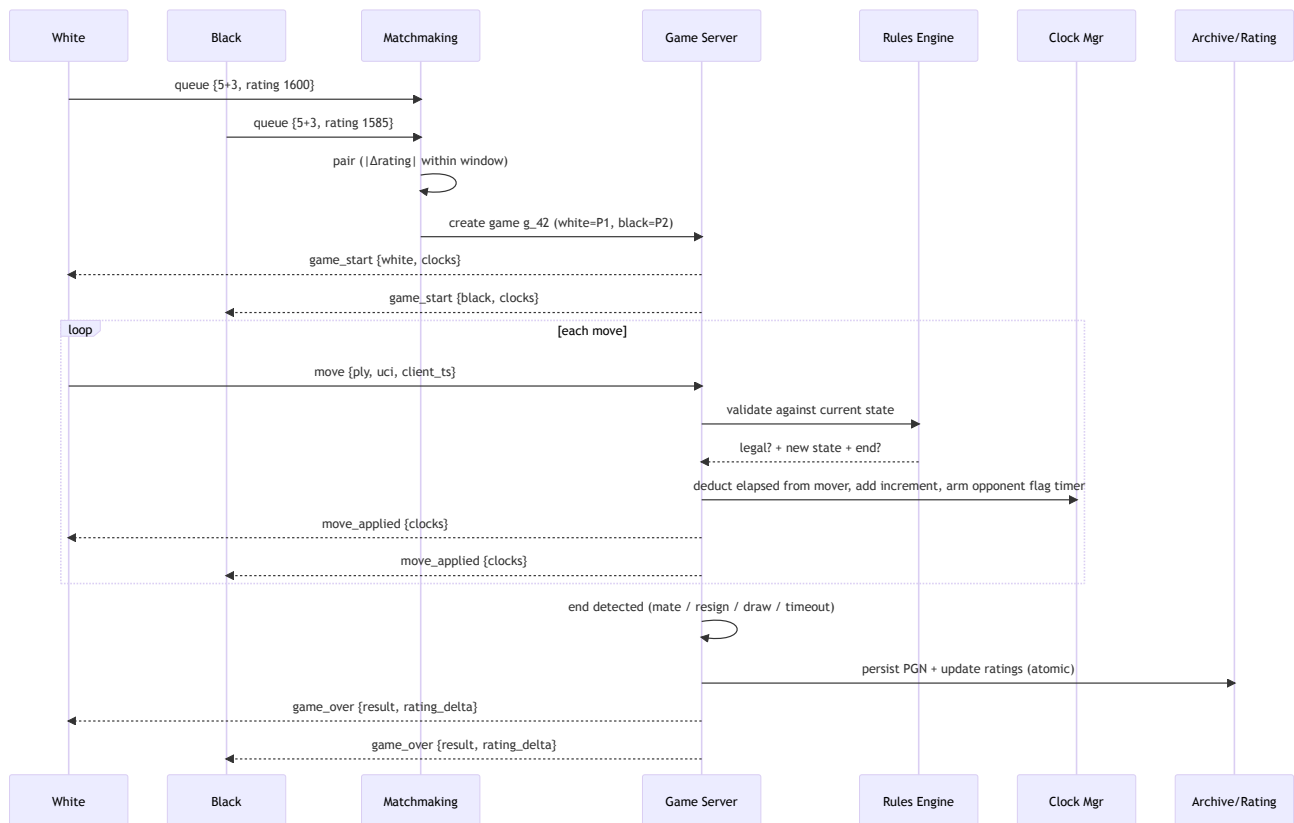
6. Architecture overview



Component responsibilities

- **Connection Gateway** — terminates player WebSockets; routes each player's frames to the **game-owner** node (via Redis `game_id` → `owner`). Stateful but disposable: on disconnect, the client reconnects (any gateway) and resumes.
- **Matchmaking Service** — maintains per-time-control **waiting pools sorted by rating** (Redis sorted sets); pairs players and asks a Game Server to create the game.
- **Game Server (the core)** — **owns the authoritative game** in memory: applies moves through the rules engine, drives the clock, broadcasts to both players + spectators, appends to the durable move log, and finalizes on end. **One owner per game** (single-writer ⇒ trivial move ordering).
- **Rules Engine** — pure chess logic: legal-move generation/validation, check/checkmate/stalemate detection, draw conditions.
- **Clock Manager** — per-game authoritative clocks; sets **flag timers** and resolves timeouts with lag compensation (§10).
- **Move Log / Event Store** — durable append-only log of moves so a crashed game is **reconstructable**.
- **Result & Rating Service** — computes Glicko-2/Elo deltas and updates both players atomically.
- **Game Archive** — finished games as **PGN + metadata** for replay; served via cache/CDN.
- **Pub/Sub** — fan-out live moves to spectators (read-only), keeping the hot player path lean.

7. Game lifecycle: queue → match → play → end → persist



Key points:

1. **Single-writer game owner** processes moves serially → no race between the two players' moves; `ply` increments cleanly.
2. **Validate before applying** — an illegal or out-of-turn move is rejected and never mutates state.
3. **Clock updated on every move** — deduct the mover's elapsed, add increment, re-arm the opponent's flag timer.
4. **End detection is server-side** — the engine reports mate/stalemate/draw; the clock reports timeout.
5. **Persist + rate atomically on end** — the finished game is durable and both ratings move together.

8. Matchmaking (focus area)

Goal: pair two players of similar strength, in the same time control, quickly. Rating is a number (Elo/Glicko, ~0–3000); pools are separate per exact time control (you don't mix 1+0 with 10+0).

The pool structure

- One **waiting pool per time control**, holding (`player_id`, `rating`, `joined_at`, `region`).
- Keep it **sorted by rating** (Redis **sorted set** `zADD pool:5+3 rating player`) so the nearest-rated opponent is a range query away.

Pairing strategies

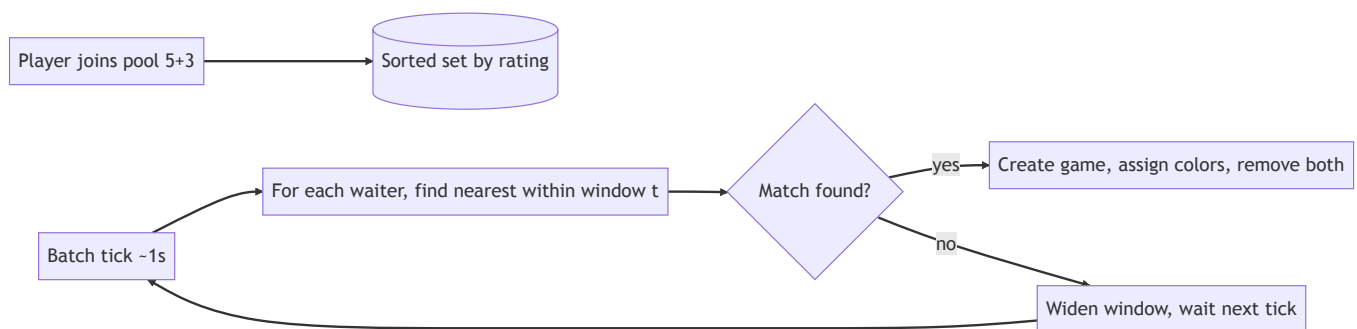
Strategy	How	Trade-off
Greedy on arrival	New player matches the closest waiting player within the window	Instant when pool is dense; can make globally sub-optimal pairs
Periodic batch (Lichess-style)	Every ~1 s, run a pass over the pool and pair globally (nearest-neighbor / min-cost matching)	Better pairings; adds up to ~1 s wait
Hybrid ✅	Try greedy immediately; if no in-window opponent, wait for the next batch tick	Fast when possible, optimal when busy

Widening window (the key idea)

Start strict, relax as the player waits so nobody waits forever:

```
window(t) = base ± (Δ0 + rate · seconds_waited)    # e.g., ±50, growing to ±100, ±200 ...
```

- **Provisional players** (few games, high rating deviation) → **wider** window immediately (we're unsure of their strength).
- **Region/latency** — prefer same-region opponents for low RTT; widen to cross-region only after waiting.
- **Avoid immediate rematches** / recently-played opponents when the pool allows.



Color assignment — balance white/black over a player's recent games; otherwise random.

One-liner: "Per-time-control pools sorted by rating in Redis; a batch tick pairs each waiter with the nearest-rated opponent inside a window that **widens with wait time** (and starts wide for provisional/high-RD players), preferring same-region — so matches are fast **and** fair."

9. Move validation & game state (focus area)

The server is the sole authority on legality — never trust the client. The client may validate locally for instant UI, but the server re-validates every move with a full rules engine.

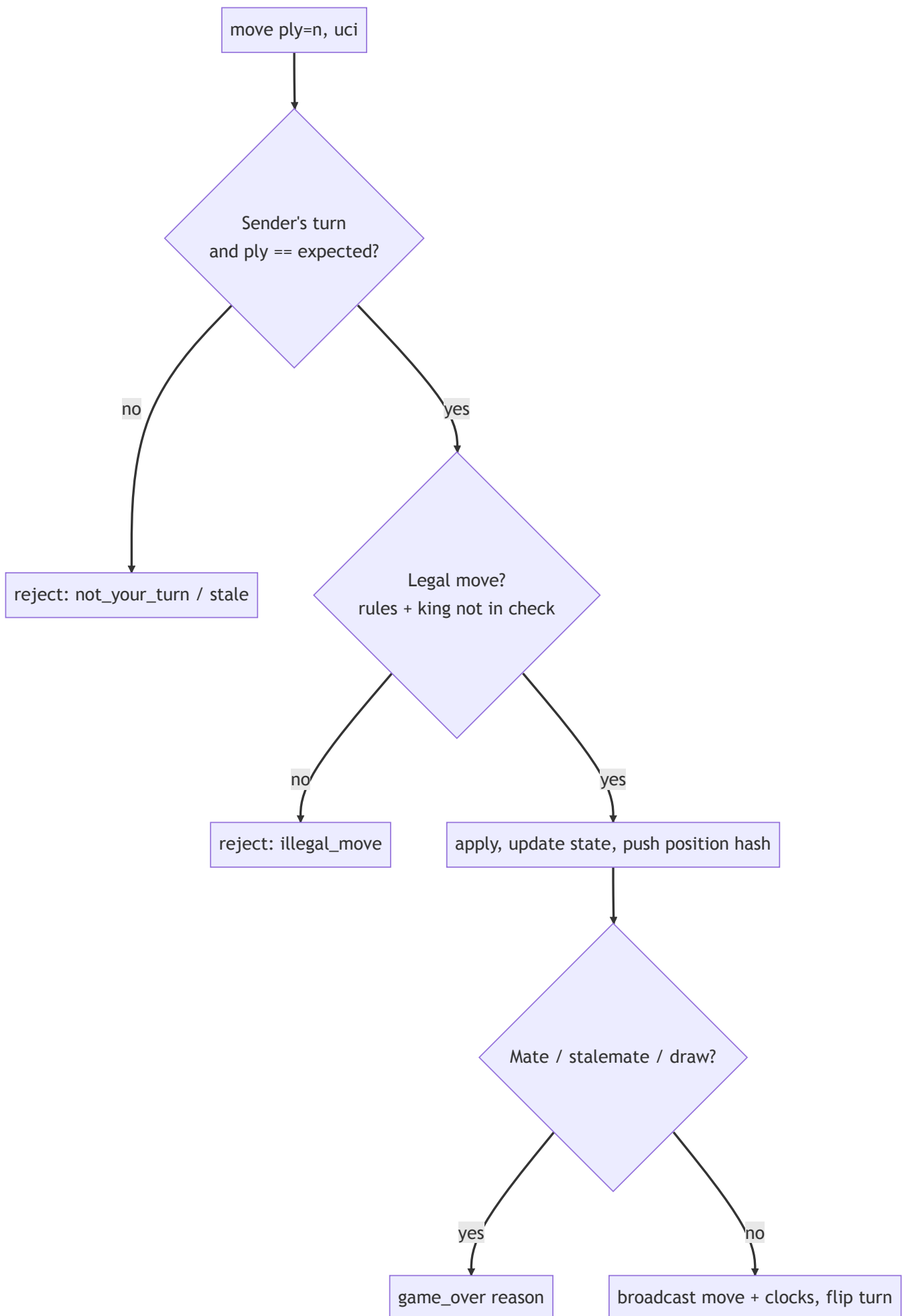
Authoritative game state (per live game, in memory)

```

FEN / board      : piece placement (8x8, or bitboards)
side_to_move     : white | black
castling_rights  : KQkq
en_passant_target : square or none
halfmove_clock   : plies since last capture/pawn move (50-move rule)
fullmove_number  : display
position_history  : hashes of every position (threefold repetition)
  
```

Validating a move

1. **Right player, right turn** — the sender owns `side_to_move`; the submitted `ply` matches the expected next ply (else reject stale/duplicate — **idempotent**).
2. **Pseudo-legal + legal** — the piece moves correctly; **special rules** (castling through/into check, en passant, promotion) handled; the move **does not leave the mover's king in check**.
3. **Apply** — mutate the board, update castling/ep/halfmove, push the position hash, flip `side_to_move`.
4. **Detect end** — checkmate, stalemate, **threefold repetition**, **50-move rule**, **insufficient material**; report to the game server.
5. **Broadcast** the accepted move + new FEN + authoritative clocks to both players (and spectators).



Move format — prefer **UCI** `from+to(+promo)` (e.g., `e7e8q`); derive **SAN** (`e8=Q+`) for display/PGN. Unambiguous and easy to validate.

Anti-cheat — illegal moves are impossible (structural). **Engine-assistance** (a human relaying an engine's moves) is a *separate* problem: an async subsystem flags suspicious accuracy/timing patterns; out of the hot path.

One-liner: *"The server keeps the authoritative FEN + castling/ep/halfmove/position-history and re-validates every move — right turn, legal, king-safe — then detects mate/stalemate/threefold/50-move/insufficient-material. Clients are optimistic views that roll back on move_rejected."*

10. The game clock (focus area — the hard part)

The server owns time. Clients only display an estimate. The subtle problem is **who flags first under network latency** — a move sent at the buzzer may arrive after the server thinks the clock hit zero.

The model

- Each side has **remaining milliseconds**; `increment_ms` is added after each completed move (Fischer).
- The clock **only ticks for the side to move**. The server records `turn_started_ts` = the instant it broadcast the opponent's move (i.e., when it became your turn).
- On receiving your move at server time `t_recv`:

```
elapsed          = t_recv - turn_started_ts - lag_credit
remaining[mover] = remaining[mover] - elapsed + increment_ms
if remaining[mover] <= 0 before the move arrived → the mover FLAGS (loses on time)
```

Detecting timeout when a player just sits there

- When it becomes X's turn, the Clock Manager **arms a flag timer** = `remaining[X]` ms.
- If the timer **fires before a move arrives** → declare **timeout**: X loses — *unless the opponent cannot possibly win* (e.g., opponent has only a king / insufficient material) → **draw** instead. If a mate/stalemate already occurred, that result stands.
- If a move **arrives just before** the timer, the game server (single-writer) processes it first and **cancels** the timer — the single-owner design makes this race deterministic (one thread decides order).

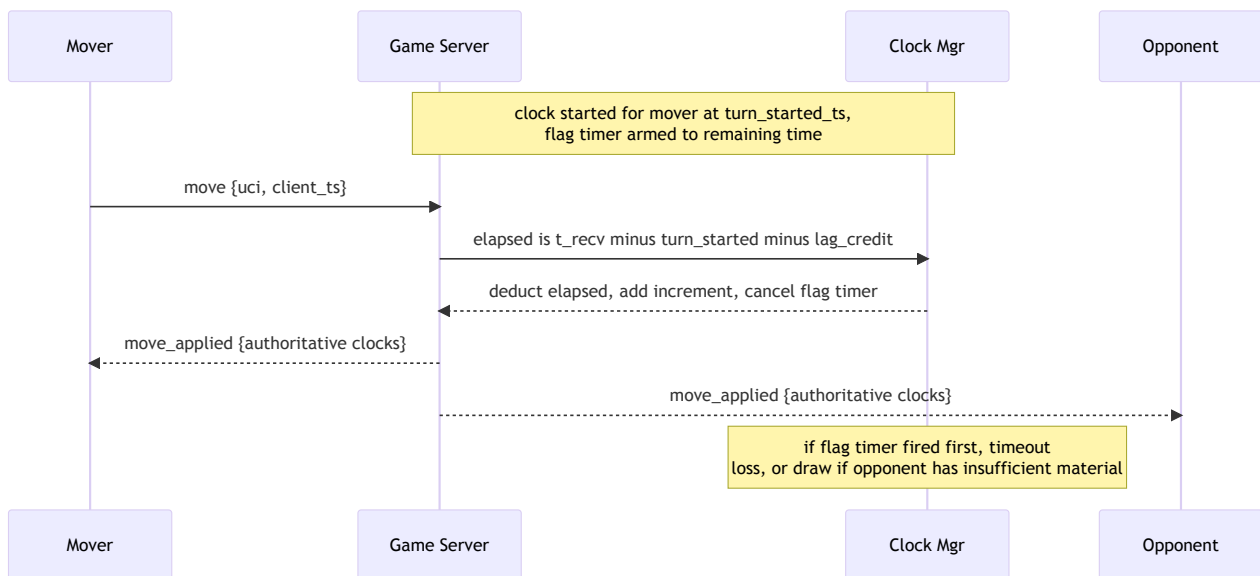
Lag compensation (fairness)

Naïve "server receive time" penalizes high-latency players — unfair. Mitigations:

- **Measure per-player network lag** continuously (WebSocket ping/pong RTT) and credit back an estimated one-way lag (`lag_credit`) so a player isn't flagged for their connection.
- **Cap the credit** (e.g., $\leq$ a few hundred ms) so it can't be abused to gain time.
- **Grace on the very last move** — many implementations don't flag if the move was clearly *sent* before expiry (`client_ts` + measured lag), trusting the measured connection profile.

Keeping both clients in sync

- The server periodically pushes an **authoritative clock snapshot** `{white_ms, black_ms, turn, server_ts}`.
- Clients **interpolate locally** (tick down the active side) between snapshots but **snap to the server** on every update — so drift never accumulates and both players agree.



One-liner: "The server is the clock authority: it deducts $t_{recv} - turn_started$ from the mover, adds increment, and arms a flag timer for the side to move. Timeout $\Rightarrow$ loss, unless the opponent has insufficient material $\Rightarrow$ draw. Network lag is measured via ping and **credited back (capped)** so players aren't flagged for their connection; clients interpolate but always snap to the server's authoritative snapshots."

11. Game session & real-time delivery (focus area)

One authoritative owner per game

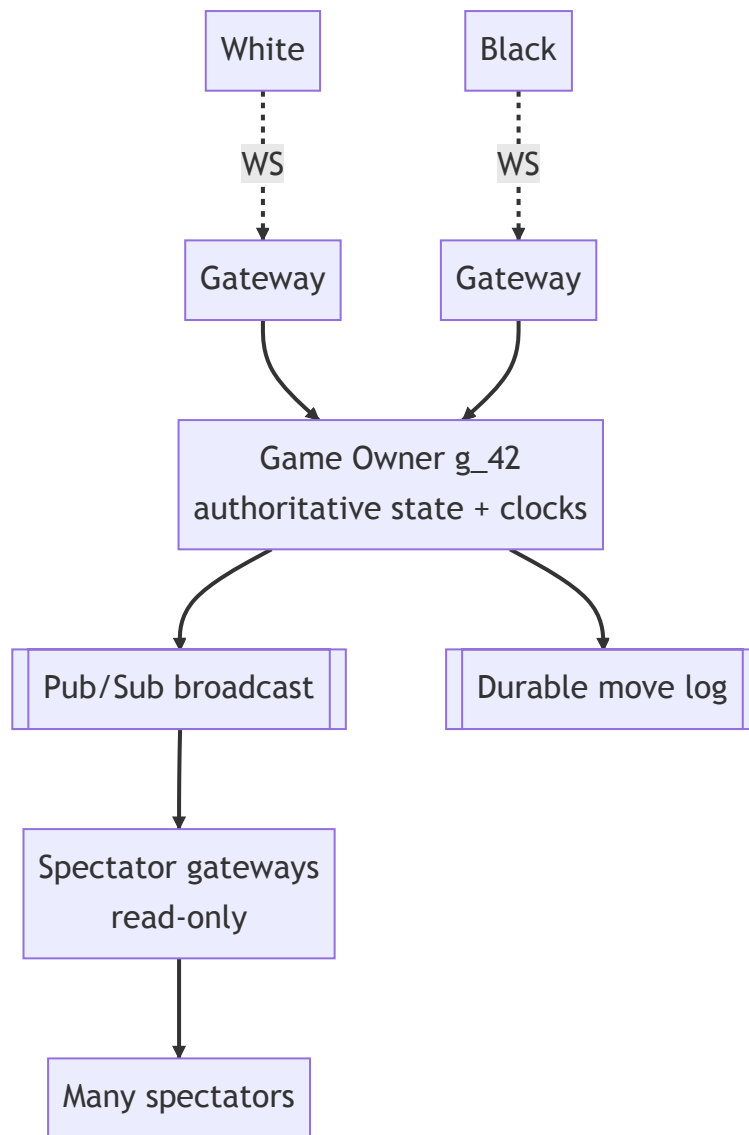
- Each live game is **owned by exactly one game-server node**, holding the state + clocks in memory (a **sharded actor**: $game_id \rightarrow owner$ via consistent hashing, recorded in Redis).
- Single-writer** $\Rightarrow$ moves are serialized naturally; there's no distributed consensus per move and `ply` ordering is automatic.
- Gateways forward each player's frames to the owner; the owner broadcasts results back through the gateways to both players.

Reconnection (clients drop constantly)

- Game state lives on the **server**, not the client. On reconnect, the client calls `GET /v1/games/{id}` (or a WS resume) and receives the **current FEN + clocks + move history**, then resumes live.
- Clock keeps running** during a disconnect (you can't gain time by pulling the cable), but a small **grace window** and "opponent disconnected" indicator are shown; if you don't return before your flag, you lose on time. Optional: auto-abort if a player never makes a first move.

Spectators

- Read-only subscribers to the game's move stream via **pub/sub**; they get the same `move_applied/clock` events but can't submit moves.
- Popular games (thousands of watchers) $\rightarrow$ the owner publishes **once** to a fan-out layer (Redis pub/sub / a broadcast tier) that cached spectator gateways read — the **celebrity/hot-game** pattern, keeping the player path lean.



One-liner: "Each game has a single authoritative owner node (sharded by `game_id`), so moves serialize without consensus. State lives server-side, so reconnects just re-fetch FEN + clocks and resume; the clock keeps running through disconnects with a grace indicator. Spectators are read-only pub/sub subscribers, and hot games fan out through a broadcast tier."

12. Game end & rating updates (focus area)

End conditions

Reason	Detected by	Result
Checkmate	rules engine	mover wins
Resignation	player action	opponent wins
Timeout (flag)	clock manager	flagged player loses (draw if opponent has insufficient material)
Stalemate	rules engine	draw
Threefold repetition / 50-move	position history / halfmove clock	draw (claimable/automatic)
Insufficient material	rules engine	draw
Draw agreement	offer + accept	draw
Abort / abandonment	no first move / long disconnect	no result, usually unrated

Rating update (Glicko-2 preferred; Elo as the simple story)

On completion, the **Result & Rating Service** computes both deltas **atomically**:

- **Elo (say this first, it's simple)**: expected score $E_A = 1 / (1 + 10^{((R_B - R_A)/400)})$; $\text{new } R_A' = R_A + K \cdot (S_A - E_A)$ with $S \in \{1, 0.5, 0\}$. K larger for provisional players.
- **Glicko-2 (production)**: tracks rating + **deviation (RD)** + **volatility**; uncertain/inactive players move more, confident players move less — better than Elo for confidence and inactivity. Ratings are **per time-control category**.
- Update **both players in one transaction** (they must move together); write a **rating-history** row for auditing/graphs. Only **rated** games count (casual/abort don't).

One-liner: "The engine or clock reports the end; the Result service updates both players' **per-category** ratings atomically — Elo $R \leftarrow R + K \cdot (S - E)$ as the simple model, **Glicko-2** in production for confidence/volatility — then the game is archived as PGN."

13. Persistence & replay (focus area)

Two very different durability needs: **surviving a crash mid-game** vs **archiving finished games**.

Live durability (don't lose a game in progress)

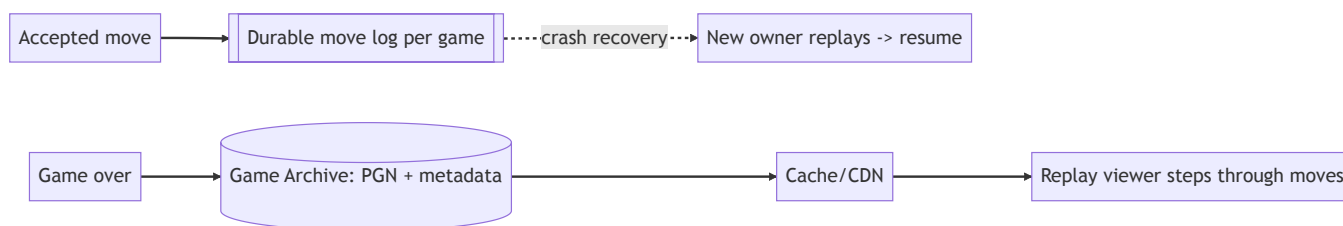
- The authoritative state is in memory (fast), but every accepted move is **appended to a durable move log** (event store) keyed by `game_id`, with the post-move clocks.
- If the owner node crashes, a new owner **replays the move log** to rebuild the exact position + clocks and resumes. (The log **is** the source of truth; the in-memory board is a cache.)

Finished-game archive (for later viewing)

- On end, write a single immutable record: **metadata** (players, ratings, time control, result, timestamps) + the **move list as PGN** (with clock annotations) + final FEN.
- **PGN** is the portable standard — compact, human-readable, replayable in any viewer:

```
[White "Ada"] [Black "Mia"] [Result "0-1"] [TimeControl "300+3"]
1. e4 e5 2. Nf3 Nc6 3. Bb5 a6 ... 0-1
```

- Serve replays from a **cache/CDN** (finished games are immutable $\Rightarrow$ perfectly cacheable). A replay is just "step through the moves," reconstructing each position client-side.



One-liner: "Every move is appended to a durable per-game log, so a crashed game is rebuilt by replay — the log is the truth, memory is the cache. On end, the game is frozen as **PGN + metadata** in an append-only archive and served immutably from a CDN for replay."

14. Storage choices

Data	Store	Why
Live game state + clocks	In-memory on the game owner (+ Redis backup)	Microsecond move application; single-writer authority
Move log / event store	Append-only log (Kafka / durable log / fast KV by <code>game_id</code>)	Crash recovery by replay; ordered by <code>ply</code>
Matchmaking pools	Redis sorted sets (by rating, per time control)	Range queries for nearest-rated opponent; ephemeral
Connection routing	Redis (<code>game_id</code> $\rightarrow$ owner, <code>player</code> $\rightarrow$ gateway)	Fast routing + cheap failover
Finished games (PGN)	Object store / wide-column, partition by <code>game_id</code> or <code>player_id</code>	Massive append-only volume; immutable; replay reads
Players & ratings	Relational (Postgres)	Transactions for atomic dual-rating updates + history
Rating history	Relational / time-series	Graphs, auditing
Replay delivery	CDN / cache	Immutable finished games $\Rightarrow$ edge-cacheable
Anti-cheat features	Analytics store (async from move log)	Off the hot path

Live vs archived is the key split: **hot, tiny, in-memory** while playing (backed by a replayable log); **cold, immutable, cheap** once finished (PGN in an archive + CDN).

15. Scalability considerations

- **Game-server tier scales by game count** — each game is a few KB and CPU-light; a node hosts **tens of thousands of games**. Shard by `game_id` (consistent hashing); rebalance as load grows.
- **Connection tier scales independently** — $\sim 100k$ WebSockets/node behind an L4/L7 LB; routing via Redis so any gateway can serve any player (cheap failover).
- **Matchmaking shards by time control** (and optionally region) — each pool is independent; hot pools (blitz) get more workers.

- **Stateless supporting services** — matchmaking pairing, rating, archive are horizontally scalable; durable state lives in Redis/log/DB.
- **Hot games (spectators)** — broadcast tier + cached game state fan out to many watchers without touching the player path.
- **Multi-region** — regional game servers + gateways for low RTT; **pair players by region** first; a game's owner lives in one region (both players routed there); archives replicate globally.
- **Graceful degradation** — under load, shed non-critical (spectator streams, presence) before player moves; the clock and move path are sacred.

16. Failure scenarios — "what if X fails?"

Failure	Impact	Mitigation
Player disconnects	Can't see/make moves	State is server-side; reconnect re-fetches FEN + clocks; clock keeps running + grace indicator; auto-abort if no first move
Game-owner node dies	Live games on it stall	New owner replays the move log to rebuild exact state + clocks and resumes; routing updated in Redis
Gateway node dies	Its connections drop	Clients reconnect to another gateway (routing via Redis) and resume
Move lost / retried	Duplicate/stale submit	Idempotent on <code>(game_id, ply)</code> : re-applying the same ply is a no-op; wrong ply rejected
Clock race at the buzzer	Mis-flag risk	Single-writer owner orders "move vs timer" deterministically; lag credit (capped) for fairness
Client clock drift	Displayed time wrong	Authoritative <code>clock</code> snapshots; client interpolates but snaps to server
Rating update fails	Inconsistent ratings	Atomic dual-update transaction; retry from the persisted result (idempotent by <code>game_id</code>)
Archive write fails	Game not saved	Move log is already durable → re-derive and retry the PGN archive
Cheating (engine use)	Unfair games	Structural legality always enforced; async statistical detection flags accounts
Region outage	Games unreachable	Owner + both players re-homed on reconnect; replay log to resume; archives replicated

Guiding principle: the **durable move log is the source of truth** (rebuild any live game), the **single-writer owner** makes move/clock ordering deterministic, and **idempotency on `(game_id, ply)`** heals retries. Correctness of state and time is non-negotiable; spectators/presence are best-effort.

17. Trade-off analysis (the money section)

Axis	Choice A	Choice B	Guidance
Authority	Trust client moves/clock (fast, cheating)	Server-authoritative legality + time	Server authority — always ✓
Clock truth	Client-timed (unfair, hackable)	Server-timed + lag credit	Server clock; measure lag, credit capped ✓
Game state	Distributed/replicated per move (consensus)	Single-writer owner in memory + replayable log	Single owner — no per-move consensus ✓
Matchmaking	Greedy instant (sub-optimal)	Batch optimal (slower)	Hybrid : greedy, fall back to batch tick
Rating	Elo (simple)	Glicko-2 (confidence/volatility)	Glicko-2 in production; Elo to explain
Move ordering	Sequencer/consensus (chat-style)	ply from single writer	Turn-based $\Rightarrow$ ply is free ordering ✓
Persistence	Snapshot state periodically	Event-sourced move log	Log = truth + crash recovery + PGN ✓
Disconnect	Pause clock (exploitable)	Keep clock running + grace	Keep running; grace indicator
Spectators	Push through player path	Separate pub/sub broadcast tier	Isolate hot games from player latency

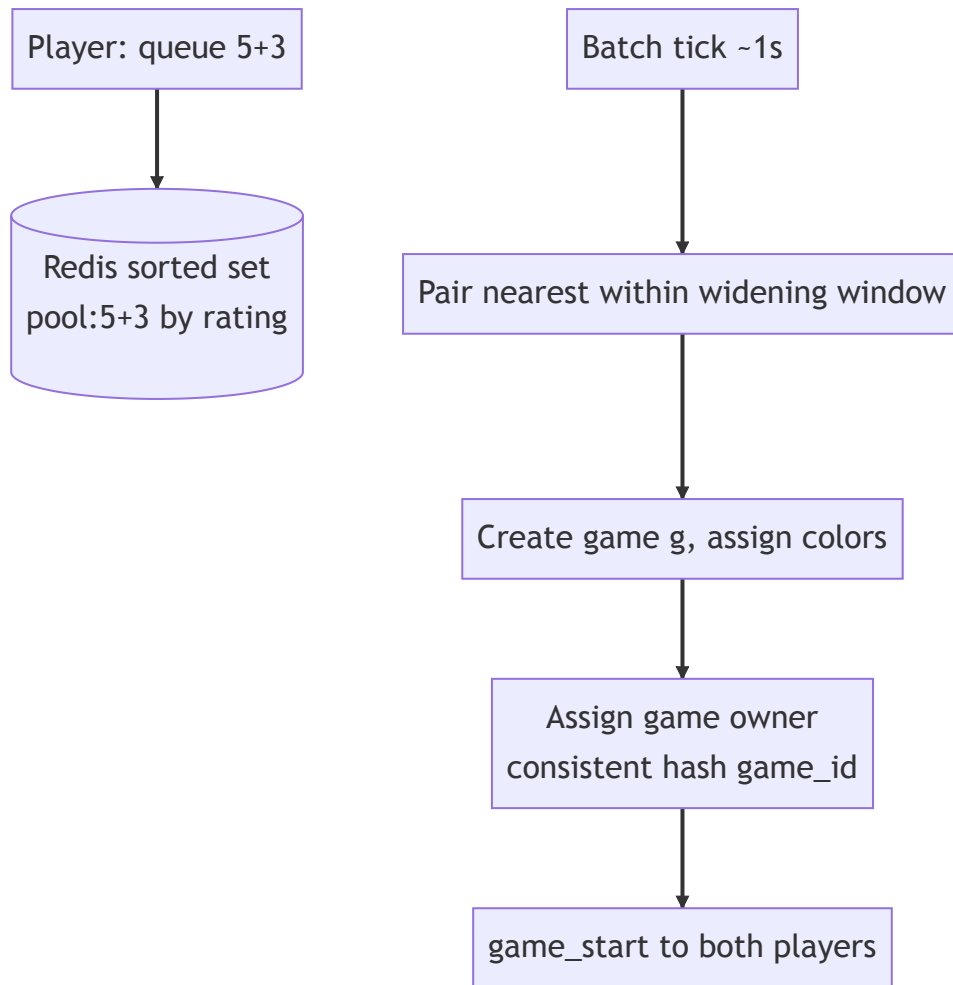
CAP framing: each game is a **CP island** — one authoritative owner serializes moves and time (consistency + correctness over availability *for that game*), rebuildable from a durable log after failover. Matchmaking, spectating, and presence lean **AP** (stay available, reconcile). Acked/rated results are **durable, non-negotiable**.

One-liner to say out loud: "Players queue by time control and are **paired by rating** with a window that widens over time. Each game gets a **single authoritative owner** that validates every move with a rules engine and owns the clock — deducting $t_{recv} - turn_started$ and arming a flag timer, with **capped lag credit** so nobody is flagged for their connection. Moves are ordered for free by **ply** (single writer), every move is appended to a **durable log** (so a crashed game is replayed and resumed), and on end both **Glicko-2 ratings update atomically** and the game is frozen as **PGN** in an immutable, CDN-served archive. Clients are optimistic views that roll back on rejection and snap to authoritative clock snapshots."

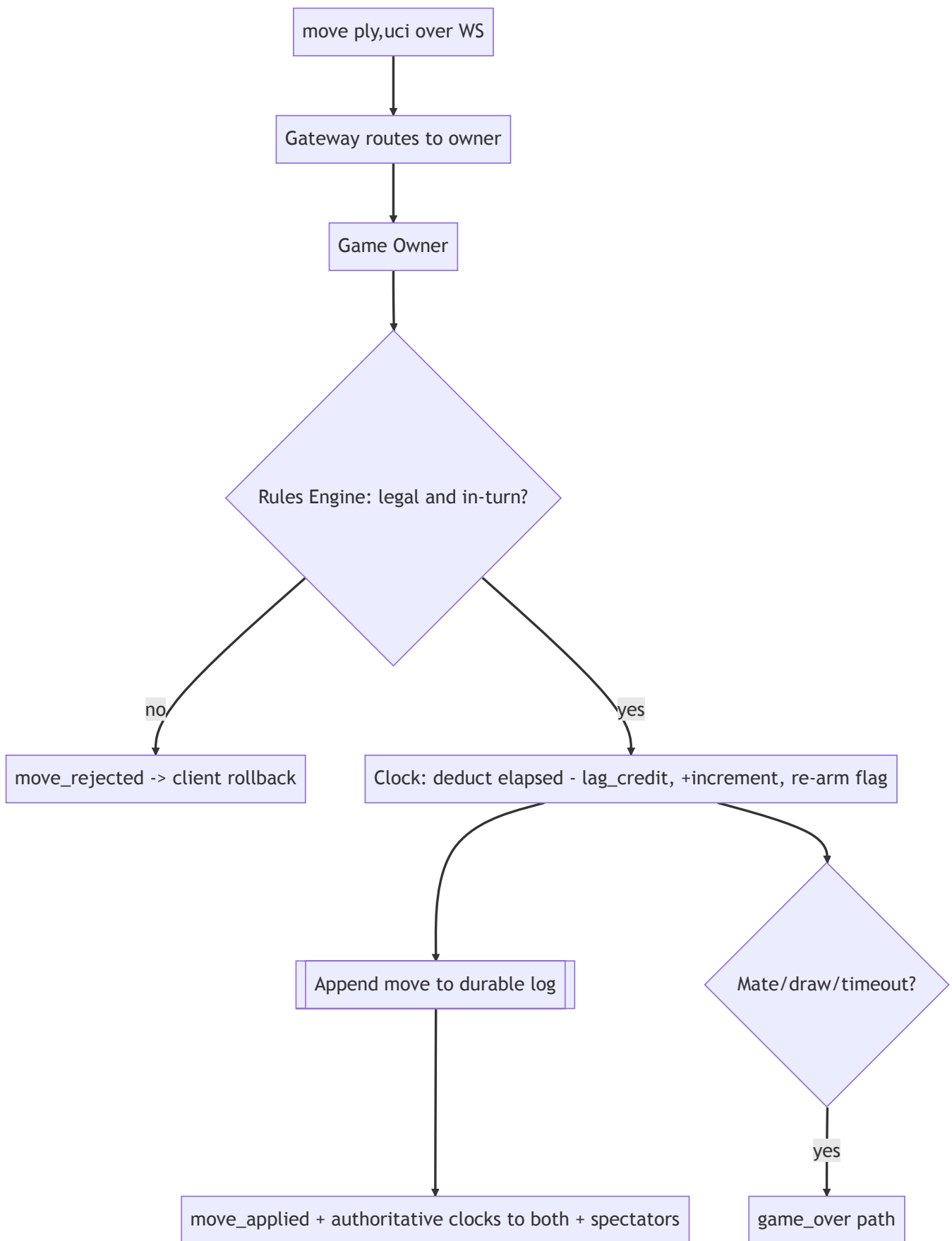
18. Full system design (detailed)

End-to-end, split into **(A)** matchmaking, **(B)** the live move/clock loop, and **(C)** end/persist/rate + replay.

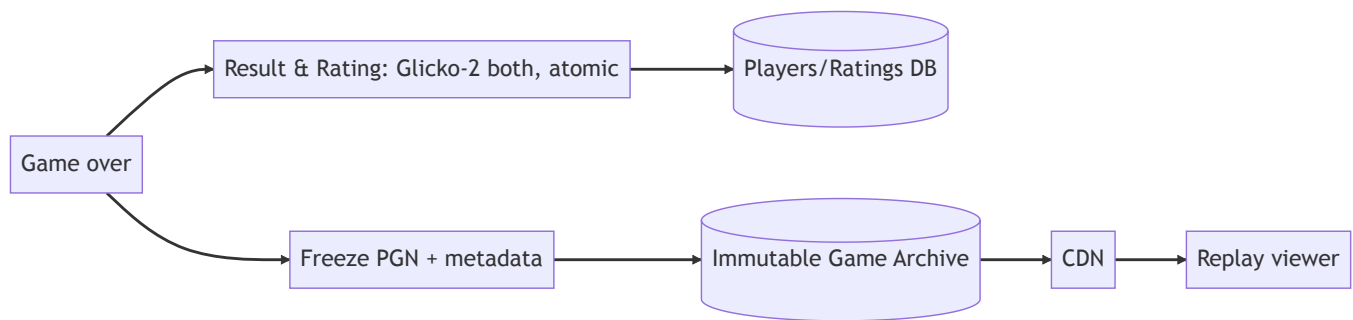
18A. Matchmaking $\rightarrow$ game creation



18B. Live move & clock loop



18C. End → persist → rate → replay



19. Networking, security & performance best practices

Networking

- **Persistent WebSocket per player** with ping/pong heartbeats (also the **lag measurement**), auto-reconnect with backoff + jitter.
- **Independent gateway tier** (~100k conns/node) behind an L4/L7 LB; **routing via Redis** (`game_id` → `owner`, `player` → `gateway`) for cheap failover.
- **TLS at the edge**; **HTTP/3 / QUIC** for mobile resilience; **gRPC** internal; both players of a game routed to the **same owner** (and, multi-region, the same region).

Security & fairness

- **AuthN/Z** on connect and per action (only the two seated players can move/resign; spectators are read-only).
- **Server-authoritative legality** blocks illegal-move exploits; **rate-limit** move spam and queue abuse.
- **Anti-cheat**: async statistical **engine-detection** (accuracy/timing) on the move log; flag & review — never trust client "analysis."
- **Abuse**: sandbag/boost detection via rating anomalies; fair-play bans; audit result-affecting events.

Performance

- **In-memory authoritative state** + microsecond validation ⇒ sub-ms server processing per move.
- **Single-writer owner** ⇒ no locks/consensus on the hot path; `ply` ordering for free.
- **Authoritative clock snapshots + client interpolation** ⇒ exact-feeling clocks without chatty updates.
- **Immutable finished games** ⇒ CDN-cacheable replays; **hot games** fan out via a broadcast tier.

20. Staying current — modern & emerging approaches

- **Reference systems**: **Lichess** (open-source; Scala, **Glicko-2**, actor-per-game, lag compensation) and **Chess.com**.
 - **Ratings**: **Glicko-2** / TrueSkill over classic Elo for confidence + inactivity handling.
 - **Transport**: WebSocket, **HTTP/3/QUIC**, gRPC streaming; actor models (Akka/Erlang/Elixir) for per-game ownership.
 - **State**: event-sourced move logs (the game **is** its move list); Redis for pools/routing; object store + CDN for PGN.
 - **Anti-cheat**: ML/statistical engine-move detection, timing analysis, centipawn-loss profiling.
 - **Patterns**: **sharded actors** (one owner per game), **event sourcing**, **optimistic client + server reconciliation**, **lag compensation** (borrowed from real-time gaming/netcode).
 - **How I stay current**: the Lichess engineering blog & open-source repo, real-time game netcode talks, distributed-systems and rating-theory papers.
-

21. Likely follow-up questions (rehearse these)

- **How do you decide who flagged first under lag?** (*server-authoritative clock, $t_{recv} - turn_started$, capped lag credit; single-writer owner orders move-vs-timer deterministically*)
 - **How do you stop illegal-move cheating?** (*server-side rules engine validates every move; client is just a view*)
 - **A player disconnects mid-game — what happens?** (*state is server-side; reconnect re-fetches FEN + clocks; clock keeps running with a grace indicator*)
 - **The game server crashes — is the game lost?** (*no — replay the durable move log to rebuild exact state + clocks and resume*)
 - **How do you match players fast but fairly?** (*rating-sorted pools per time control; window widens with wait; wider for provisional/high-RD*)
 - **Elo vs Glicko-2?** (*Elo $R \propto K(S-E)$ simple; Glicko-2 adds deviation + volatility → better confidence/inactivity*)
 - **How is move ordering guaranteed?** (*strictly turn-based; single-writer owner increments ply ; no sequencer needed*)
 - **Timeout but the opponent only has a king?** (*insufficient material ⇒ draw, not a loss*)
 - **Handling thousands of spectators on a top game?** (*broadcast tier + cached state, isolated from the player path*)
-

22. Summary checklist (whiteboard recap)

- **Entities** — players (+ per-category rating/RD), games, moves (ordered by ply), in-memory state + clocks; PGN archive.
- **Transport** — WebSocket for moves/clocks; REST for queue/history/replay; moves idempotent on ($game_id$, ply).
- **Matchmaking** — Redis rating-sorted pools **per time control**; **window widens with wait**; wider for provisional; pair by region.
- **Move validation** — **server-authoritative** rules engine (legality, check/mate, stalemate, threefold, 50-move, insufficient material); client optimistic + rollback.
- **Clock** — **server owns time**; deduct $t_{recv} - turn_started$, add increment, arm flag timer; **timeout ⇒ loss (draw if insufficient material)**; **capped lag credit**; snapshots + client interpolation.
- **Session** — **one authoritative owner per game** (sharded by $game_id$, single-writer ⇒ free ordering); reconnection re-fetches state; spectators via pub/sub.
- **End & rating** — engine/clock detect end; **Glicko-2** (or Elo) update **both players atomically**.
- **Persistence** — every move appended to a **durable log** (crash recovery by replay); finished game frozen as **PGN + metadata**, served immutably via CDN.
- **Scale** — game tier by game count (tens of k/node), connection tier ~100k WS/node, matchmaking sharded by time control, multi-region pairing.
- **CAP** — each game is a **CP island** (single-writer owner, replayable log); matchmaking/spectating/presence lean AP; rated results durable and non-negotiable.